

Comparing Categories - Refinements

Maithreyi Gopalan
Week 5

Data viz in Wild

Febe

Erika

Chen and Tongle later on today;
Everett and Dodjivi on deck for next
week

Agenda

- Visualization for comparing categories - more considerations
- Aspect ratios, scales, and labels
- Annotations (most of the day)
- Saving plots (pretty quick)
- Compound figures
- Themes refresher (if time permits)
- Review Lab 3 and Lab 4/Lab PS-2
- Introduce Lab-PS3 quickly
- Lab 5

Learning Objectives

- Additional Considerations when visualizing/comparing categories including over time
- Understand how to make a wide variety of tweaks to ggplot.
- Understand common modifications to plots to make them more clear and reduce cognitive load
 - And ways to implement them

Comparing Categories: Core Concepts

Why Compare Categories?

- Categories are everywhere in data
 - Demographics (race, gender, education level)
 - Geographic units (states, countries, regions)
 - Time periods (years, quarters, months)
 - Experimental conditions or groups
- Key questions when comparing categories:
 - Which category has the highest/lowest value?
 - How different are the categories from each other?
 - Are there patterns or trends across categories?
 - How do distributions differ across categories?

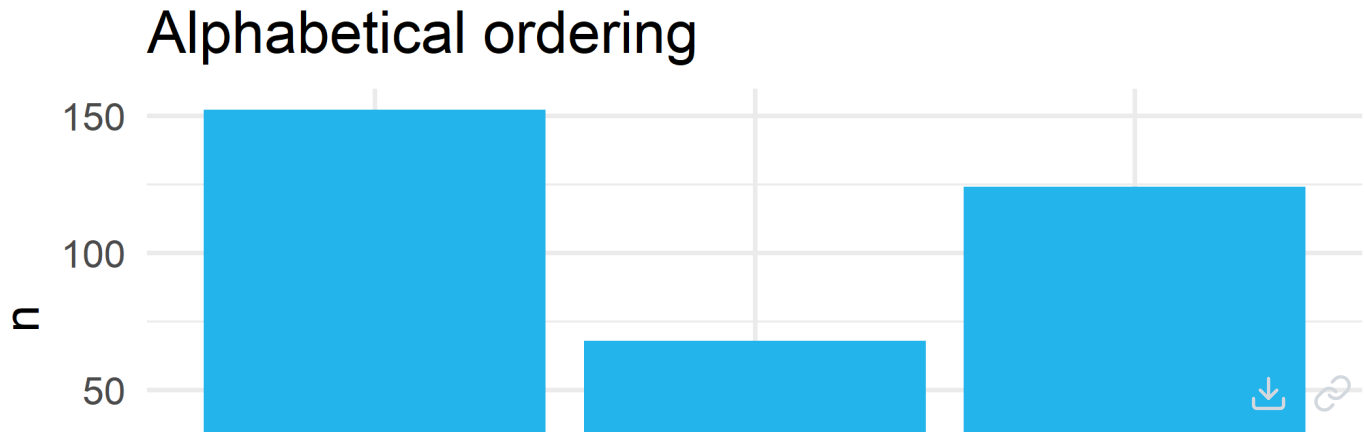
Strategies for Comparing Categories

1. **Direct comparison:** Bar charts, dot plots, lollipop charts
2. **Showing distributions:** Boxplots, violin plots, ridgeline plots
3. **Multiple dimensions:** Faceting, small multiples
4. **Proportions:** Stacked bars, treemaps, waffle charts
5. **Ordering:** Alphabetical, by value, by time, by logic

Ordering Categories Effectively

```
library(palmerpenguins)

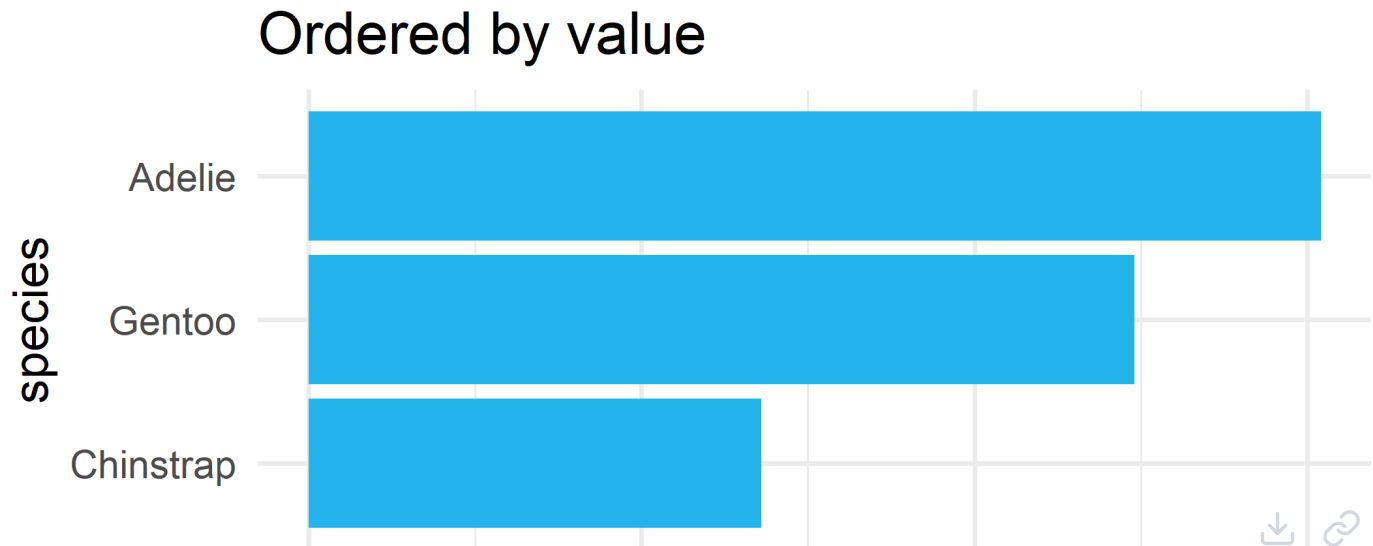
# Alphabetical (default)
p1 <- penguins %>%
  drop_na(species) %>%
  count(species) %>%
  ggplot(aes(x = species, y = n)) +
  geom_col(fill = "#25B6EE") +
  labs(title = "Alphabetical ordering")
p1
```



Ordering by Value

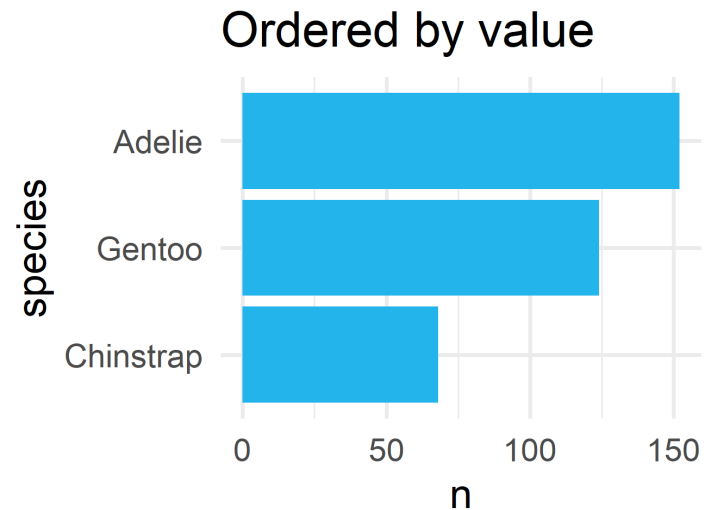
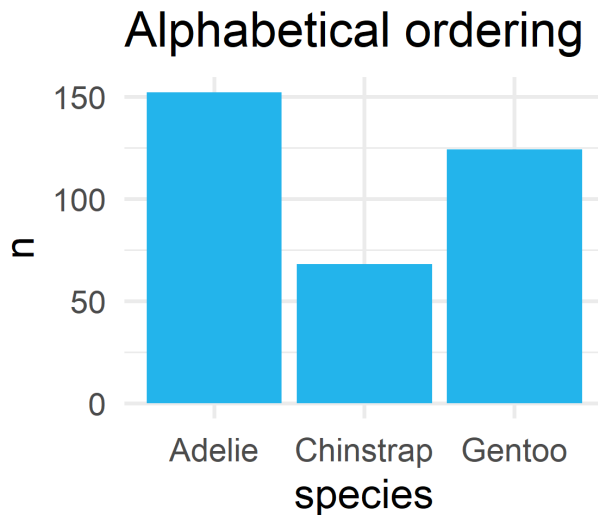
```
# Ordered by count
p2 <- penguins %>%
  drop_na(species) %>%
  count(species) %>%
  mutate(species = fct_reorder(species, n)) %>%
  ggplot(aes(x = n, y = species)) +
  geom_col(fill = "#25B6EE") +
  labs(title = "Ordered by value")
```

p2



Side by Side Comparison

```
library(patchwork)
p1 + p2
```



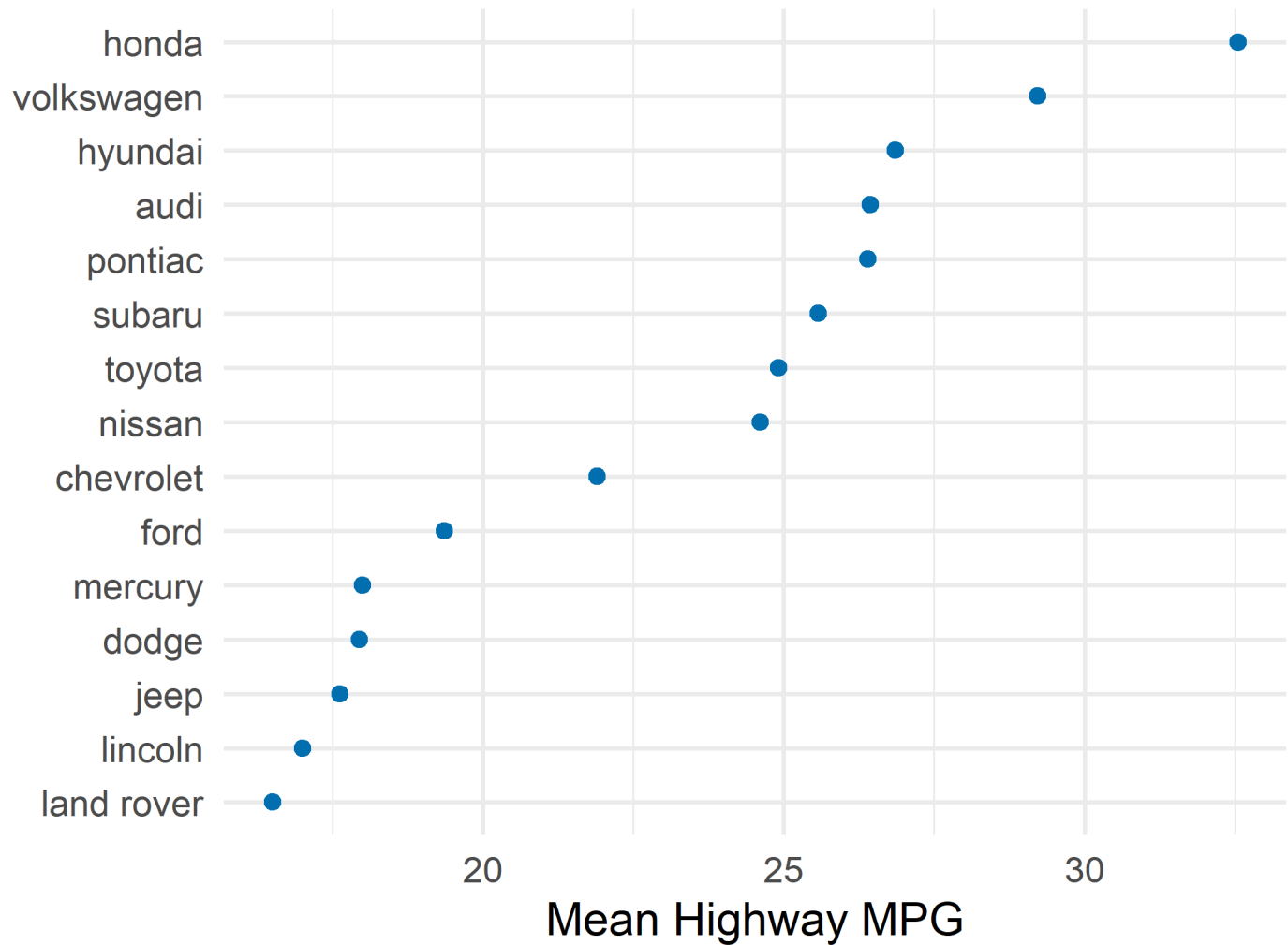
Which is easier to read?

Cleveland Dot Plots

Great alternative to bar charts when you have many categories

```
mpg %>%  
  group_by(manufacturer) %>%  
  summarize(mean_hwy = mean(hwy)) %>%  
  mutate(manufacturer = fct_reorder(manufacturer, mean_hwy)) %>%  
  ggplot(aes(x = mean_hwy, y = manufacturer)) +  
  geom_point(size = 4, color = "#0072B2") +  
  labs(x = "Mean Highway MPG",  
       y = NULL,  
       title = "Average Fuel Efficiency by Manufacturer")
```

Average Fuel Efficiency by Manufacturer

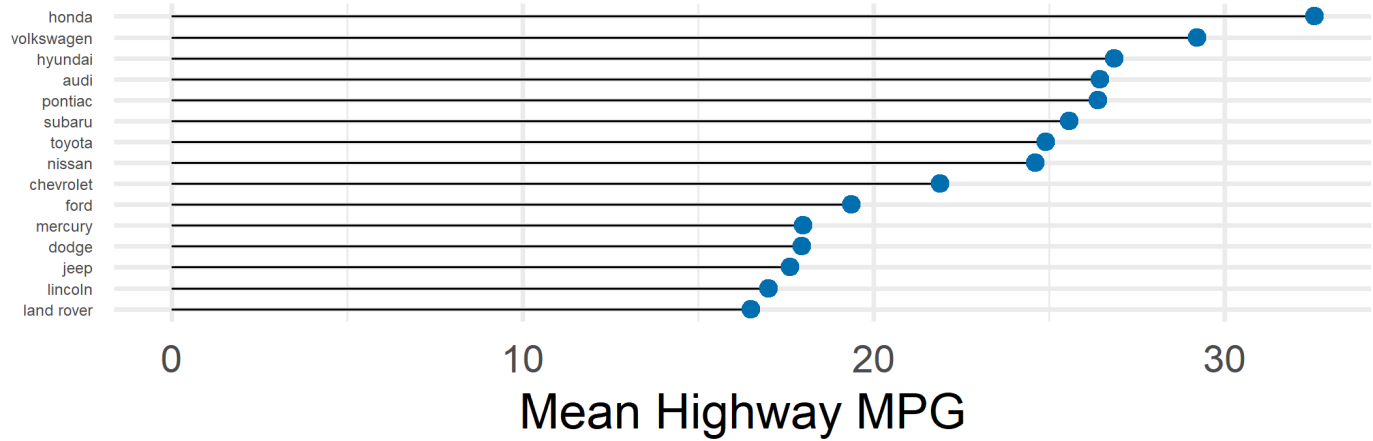


Lollipop Charts

Add visual connection to baseline

```
mpg %>%
  group_by(manufacturer) %>%
  summarize(mean_hwy = mean(hwy)) %>%
  mutate(manufacturer = fct_reorder(manufacturer, mean_hwy)) %>%
  ggplot(aes(x = mean_hwy, y = manufacturer)) +
  geom_segment(aes(x = 0, xend = mean_hwy,
                  y = manufacturer, yend = manufacturer),
              color = "black") +
  geom_point(size = 4, color = "#0072B2") +
  labs(x = "Mean Highway MPG",
       y = NULL,
       title = "Average Fuel Efficiency by Manufacturer") +
  theme(axis.text.y = element_text(size = 8, hjust = 1))
```

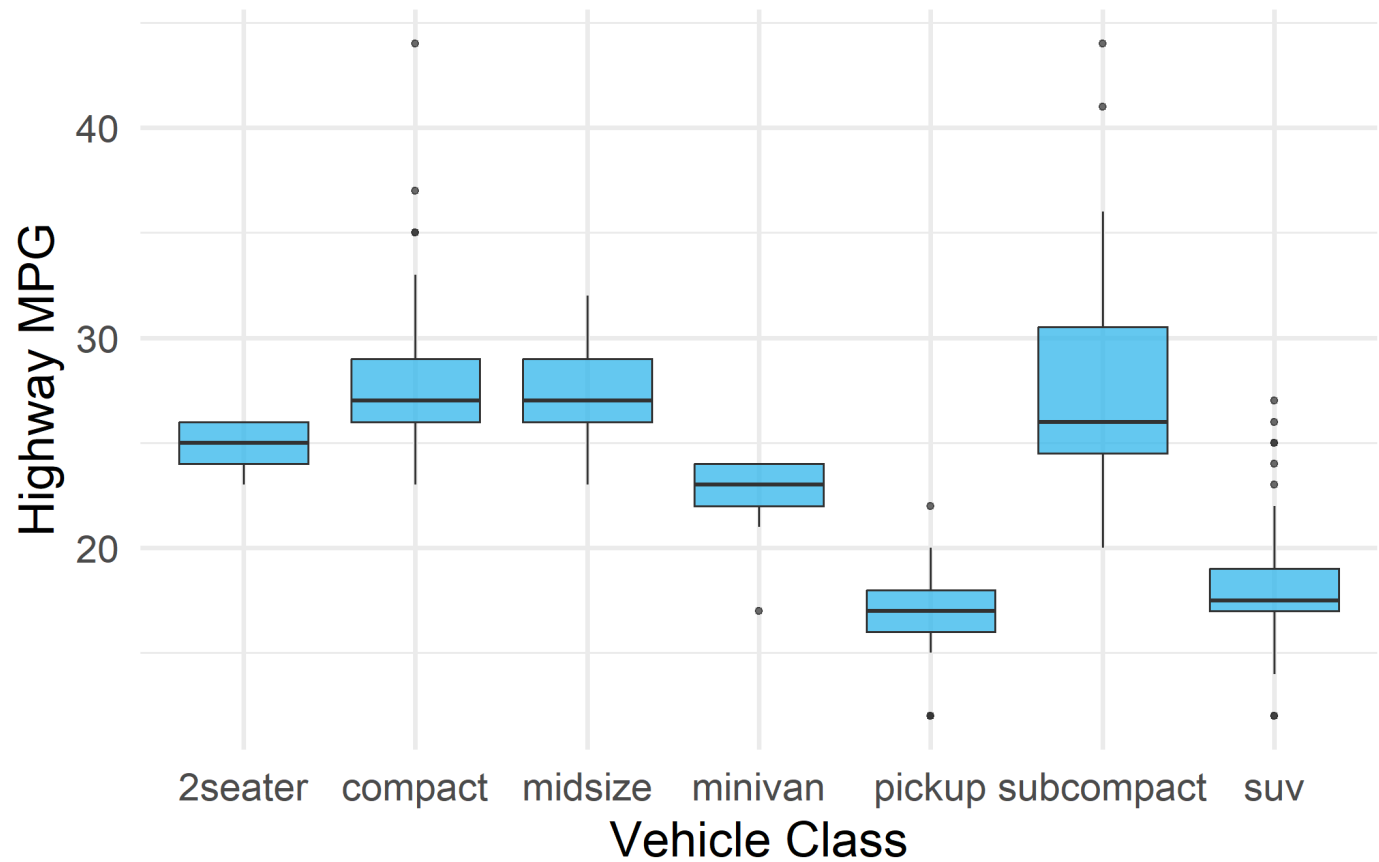
Average Fuel Efficiency by Manufacturer



Comparing Distributions: Boxplots

```
ggplot(mpg, aes(x = class, y = hwy)) +  
  geom_boxplot(fill = "#25B6EE", alpha = 0.7) +  
  labs(x = "Vehicle Class",  
       y = "Highway MPG",  
       title = "Fuel Efficiency by Vehicle Class")
```

Fuel Efficiency by Vehicle Class

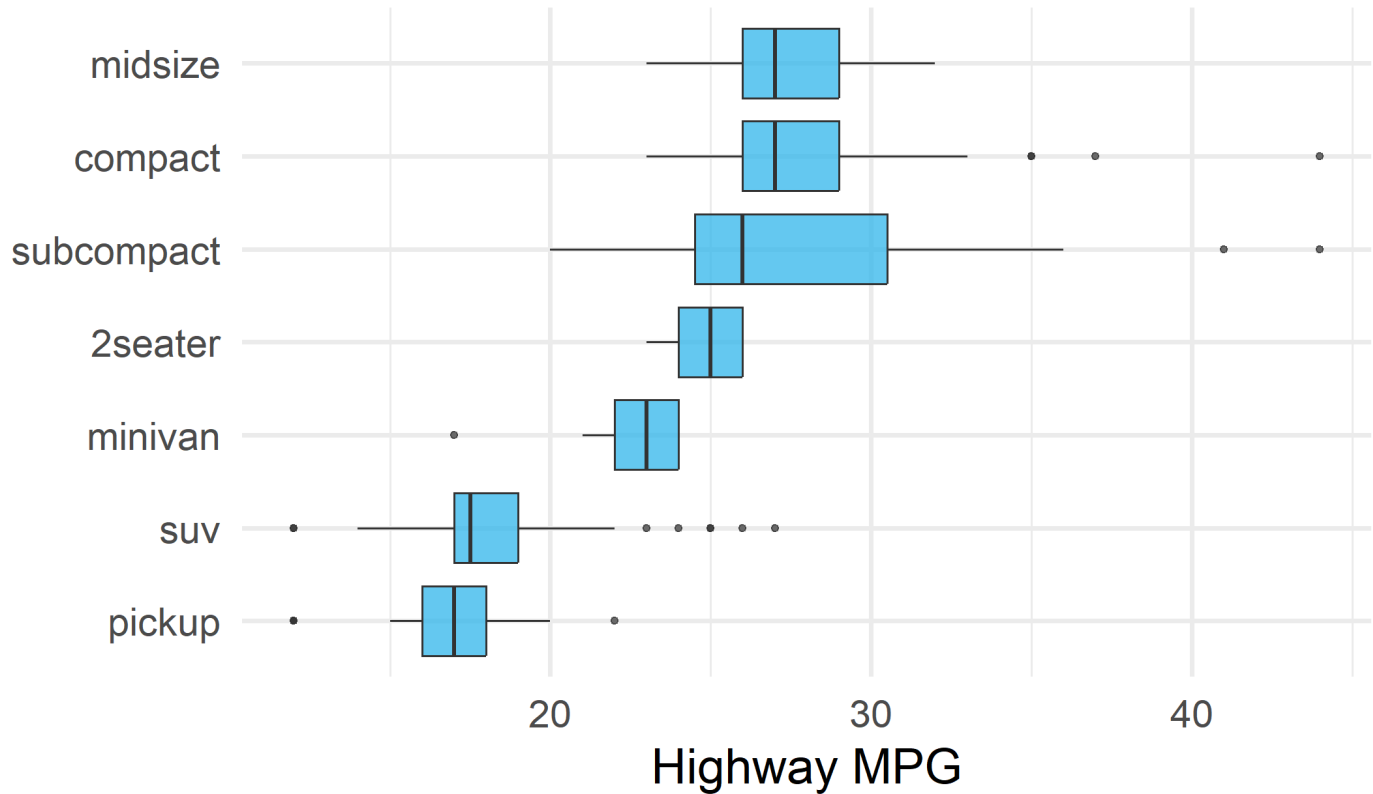


Better: Ordered Boxplots

```
mpg %>%  
  mutate(class = fct_reorder(class, hwy, .fun = median)) %>%  
  ggplot(aes(x = class, y = hwy)) +  
  geom_boxplot(fill = "#25B6EE", alpha = 0.7) +  
  coord_flip() +  
  labs(y = "Highway MPG",  
       x = NULL,  
       title = "Fuel Efficiency by Vehicle Class",  
       subtitle = "Ordered by median MPG")
```

Fuel Efficiency by Vehicle Class

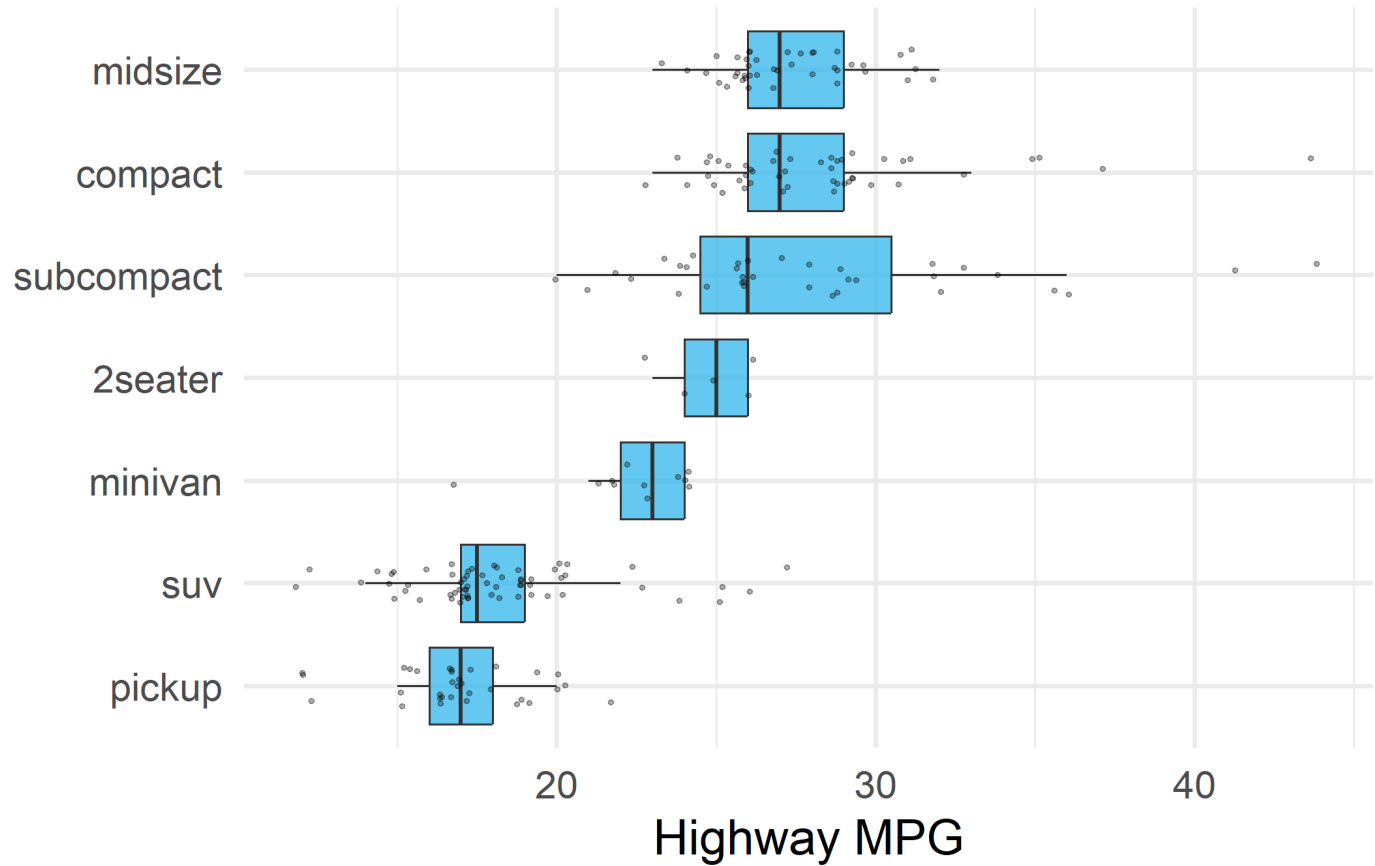
Ordered by median MPG



Add Individual Points

```
mpg %>%  
  mutate(class = fct_reorder(class, hwy, .fun = median)) %>%  
  ggplot(aes(x = class, y = hwy)) +  
  geom_boxplot(fill = "#25B6EE", alpha = 0.7, outlier.shape = NA) +  
  geom_jitter(width = 0.2, alpha = 0.3, size = 1) +  
  coord_flip() +  
  labs(y = "Highway MPG",  
        x = NULL,  
        title = "Fuel Efficiency by Vehicle Class")
```

Fuel Efficiency by Vehicle Class

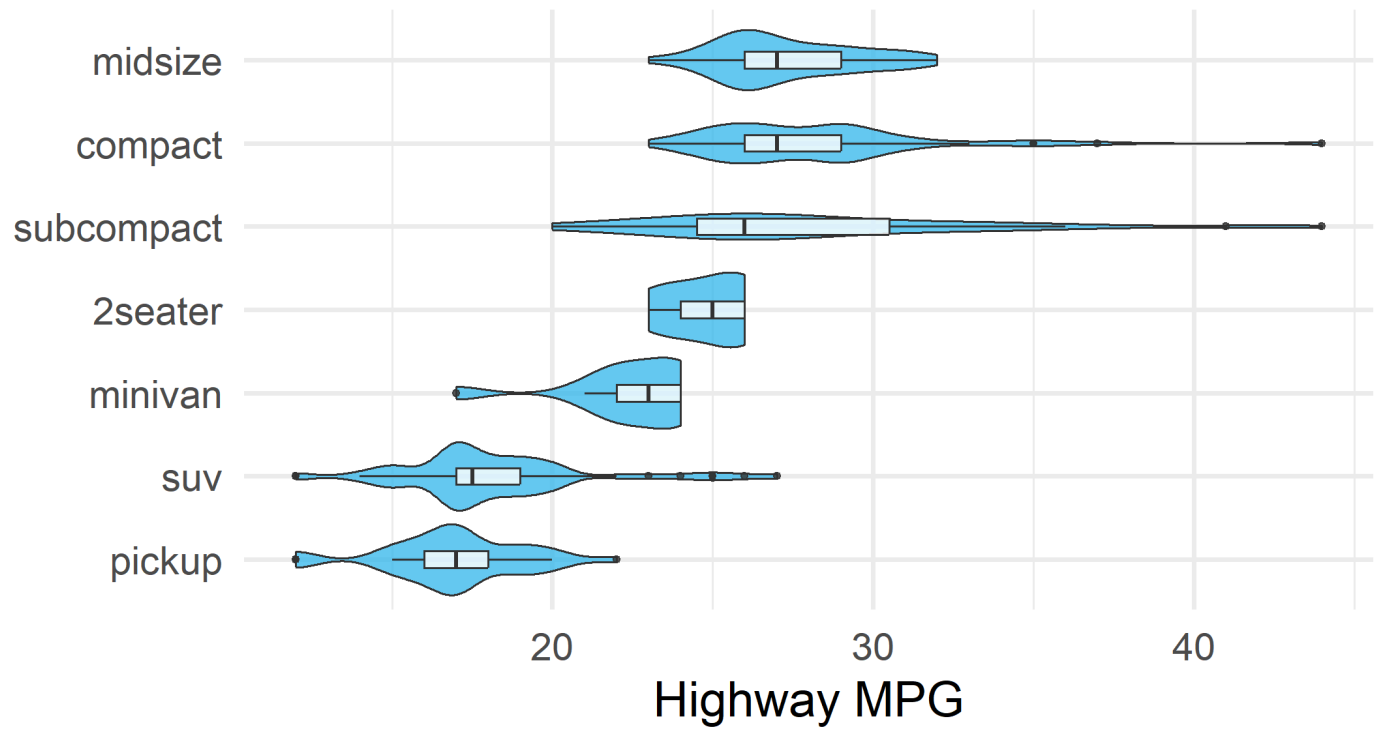


Violin Plots

Show the full distribution

```
mpg %>%  
  mutate(class = fct_reorder(class, hwy, .fun = median)) %>%  
  ggplot(aes(x = class, y = hwy)) +  
  geom_violin(fill = "#25B6EE", alpha = 0.7) +  
  geom_boxplot(width = 0.2, fill = "white", alpha = 0.8) +  
  coord_flip() +  
  labs(y = "Highway MPG",  
        x = NULL,  
        title = "Distribution of Fuel Efficiency by Vehicle Class")
```

Distribution of Fuel Efficiency by Vehicle Class

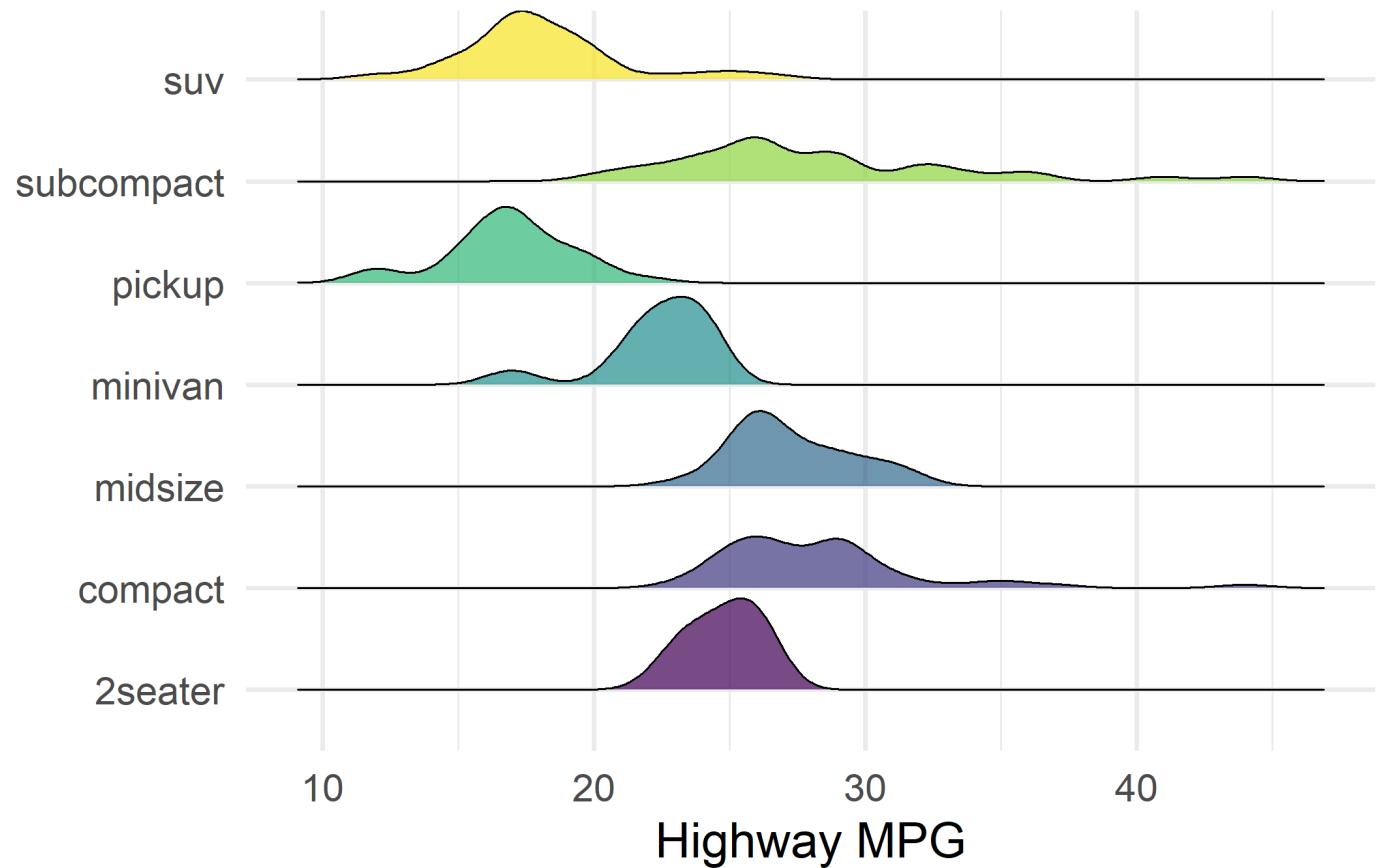


Ridgeline Plots

```
library(ggribes)

ggplot(mpg, aes(x = hwy, y = class, fill = class)) +
  geom_density_ridges(alpha = 0.7, scale = 0.9) +
  scale_fill_viridis_d() +
  labs(x = "Highway MPG",
       y = NULL,
       title = "Distribution of Fuel Efficiency by Vehicle Class")
theme(legend.position = "none")
```

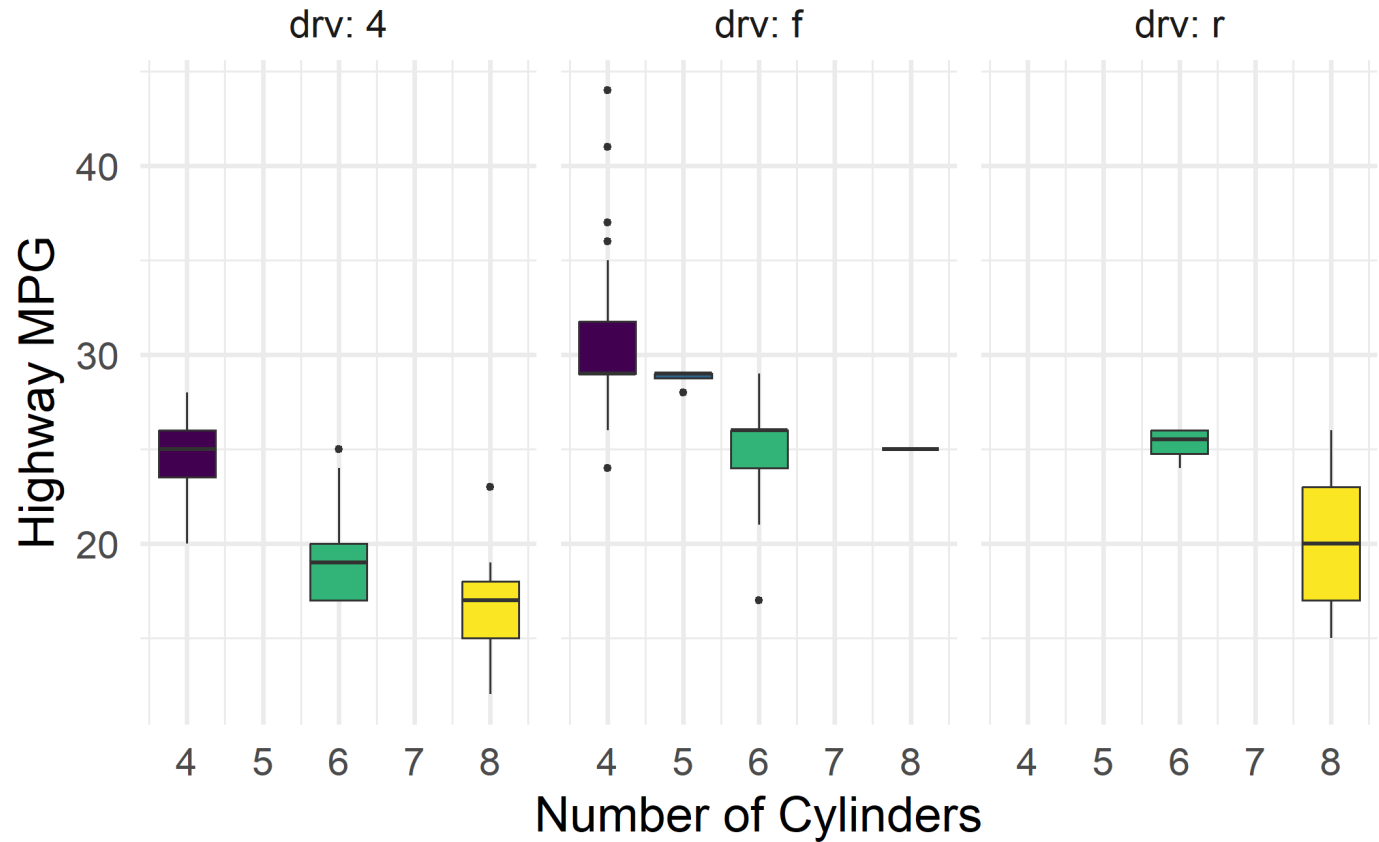
Distribution of Fuel Efficiency by Vehicle Class



Faceting for Multiple Categories

```
ggplot(mpg, aes(x = cyl, y = hwy)) +  
  geom_boxplot(aes(group = cyl, fill = factor(cyl))) +  
  facet_wrap(~ drv, labeller = label_both) +  
  scale_fill_viridis_d() +  
  labs(x = "Number of Cylinders",  
       y = "Highway MPG",  
       title = "Fuel Efficiency by Cylinders and Drive Type") +  
  theme(legend.position = "none")
```

Fuel Efficiency by Cylinders and Drive Type

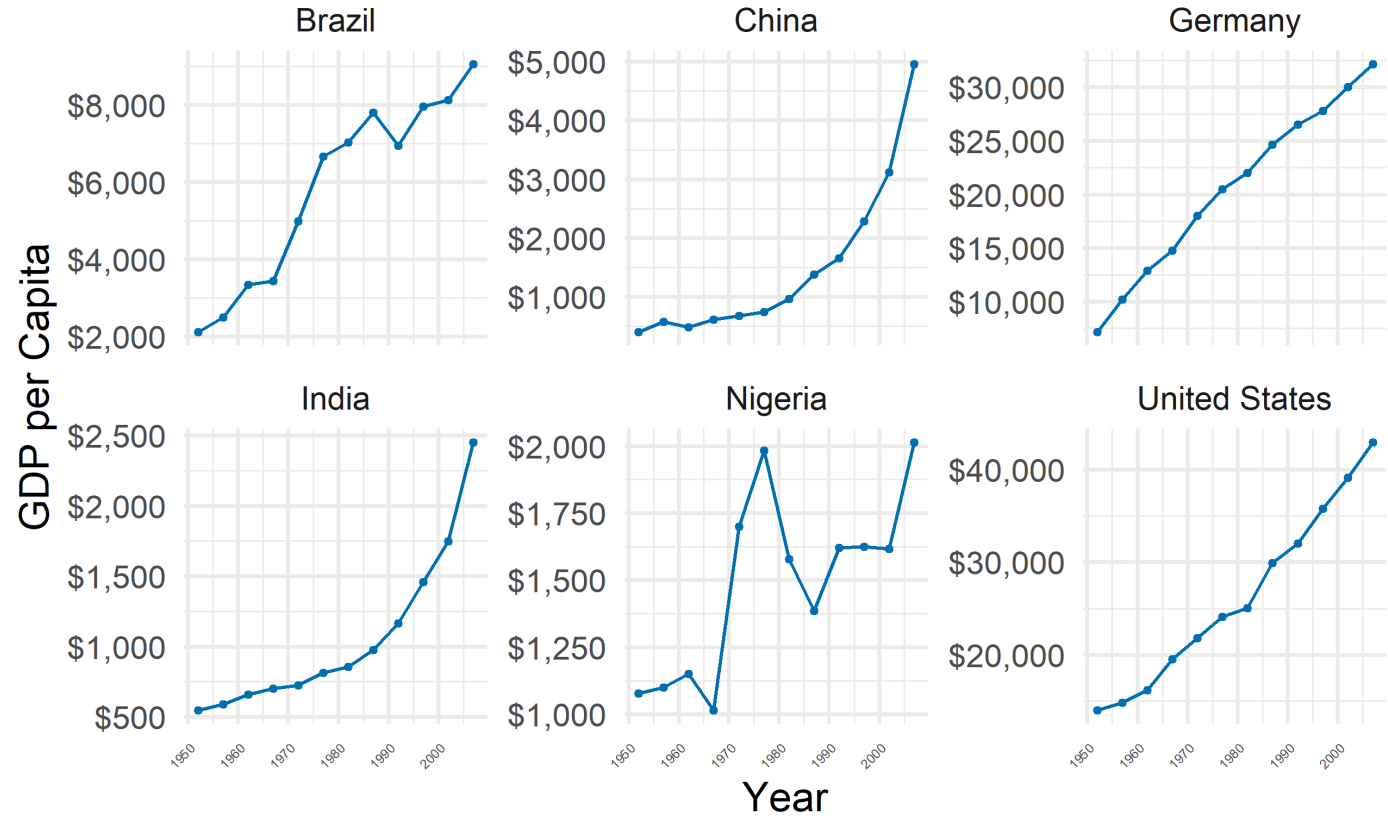


Small Multiples: Time Series

```
library(gapminder)

gapminder %>%
  filter(country %in% c("United States", "China", "India",
                        "Germany", "Brazil", "Nigeria")) %>%
  ggplot(aes(x = year, y = gdpPercap)) +
  geom_line(color = "#0072B2", size = 1) +
  geom_point(size = 2, color = "#0072B2") +
  facet_wrap(~ country, scales = "free_y") +
  scale_y_continuous(labels = scales::dollar) +
  labs(x = "Year",
       y = "GDP per Capita",
       title = "Economic Growth Across Countries")+
  theme(axis.text.x = element_text(size = 8, angle = 45, hjust = 1)
```

Economic Growth Across Countries

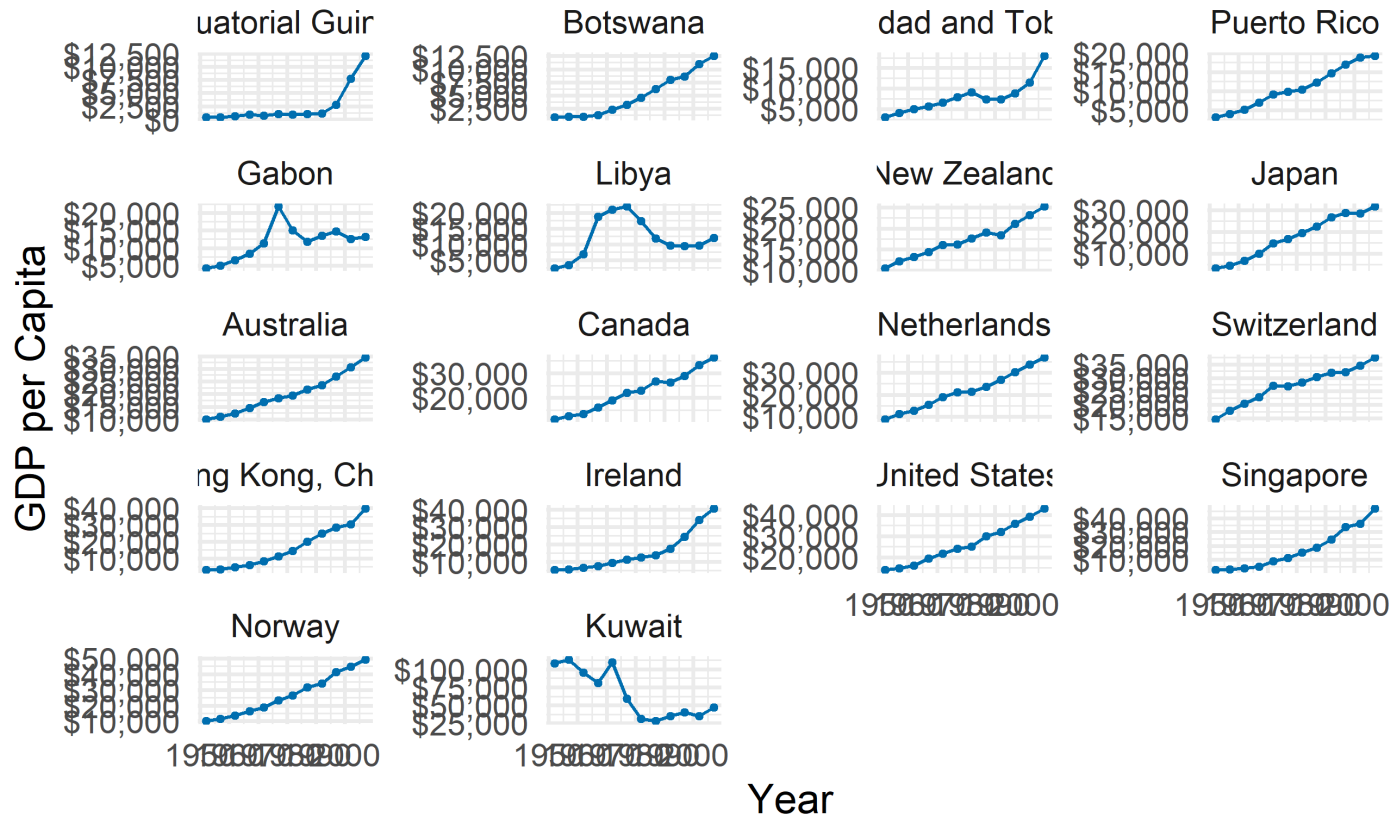


Small Multiples: Better Layout

```
# First, identify top 4 countries per continent in 2007
top_countries <- gapminder %>%
  filter(year == 2007) %>%
  group_by(continent) %>%
  slice_max(gdpPercap, n = 4) %>%
  pull(country)

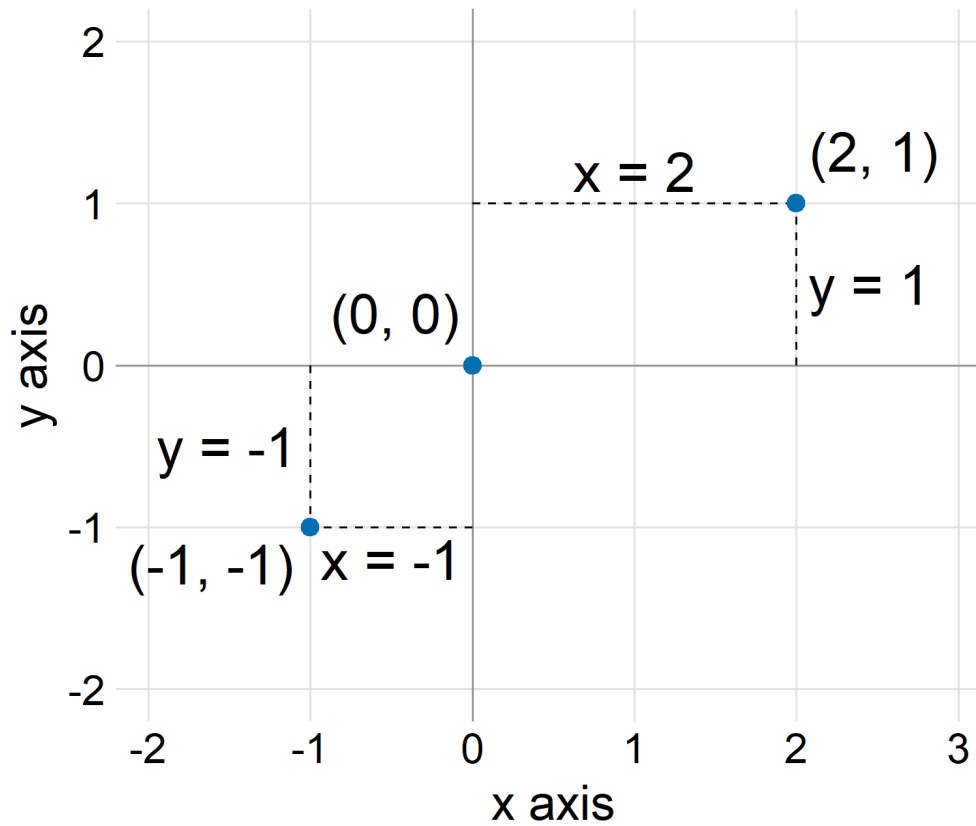
# Then filter gapminder to just those countries across all years
gapminder %>%
  filter(country %in% top_countries) %>%
  mutate(country = fct_reorder(country, gdpPercap, .fun = max)) %>%
  ggplot(aes(x = year, y = gdpPercap)) +
  geom_line(color = "#0072B2", size = 1) +
  geom_point(size = 2, color = "#0072B2") +
  facet_wrap(~ country, scales = "free_y", ncol = 4) +
  scale_y_continuous(labels = scales::dollar) +
  labs(x = "Year",
       y = "GDP per Capita",
       title = "Top 4 Countries by GDP per Capita in Each Continent")
```

Top 4 Countries by GDP per Capita in Each Continent (1950-2000)

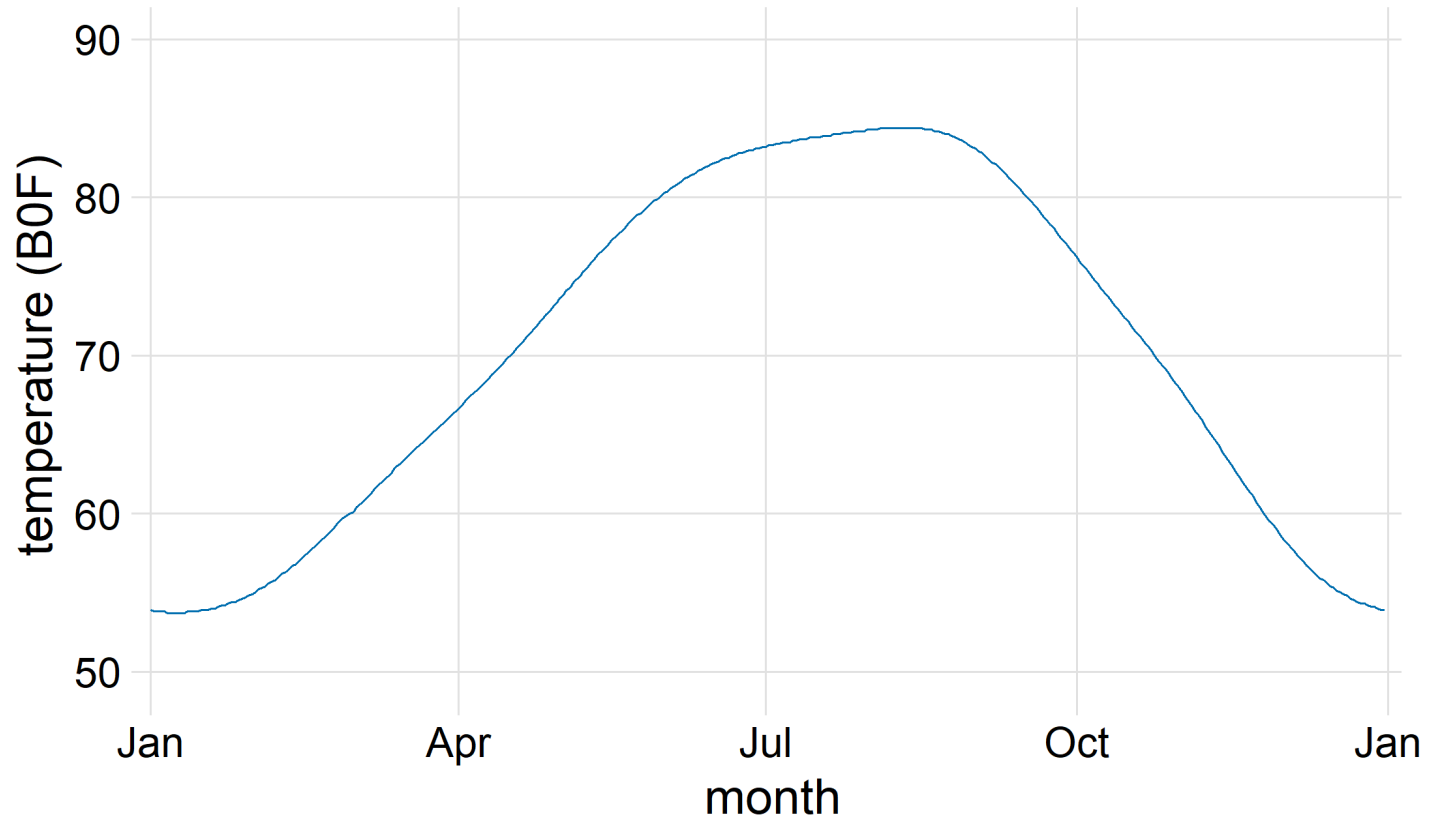


Axes

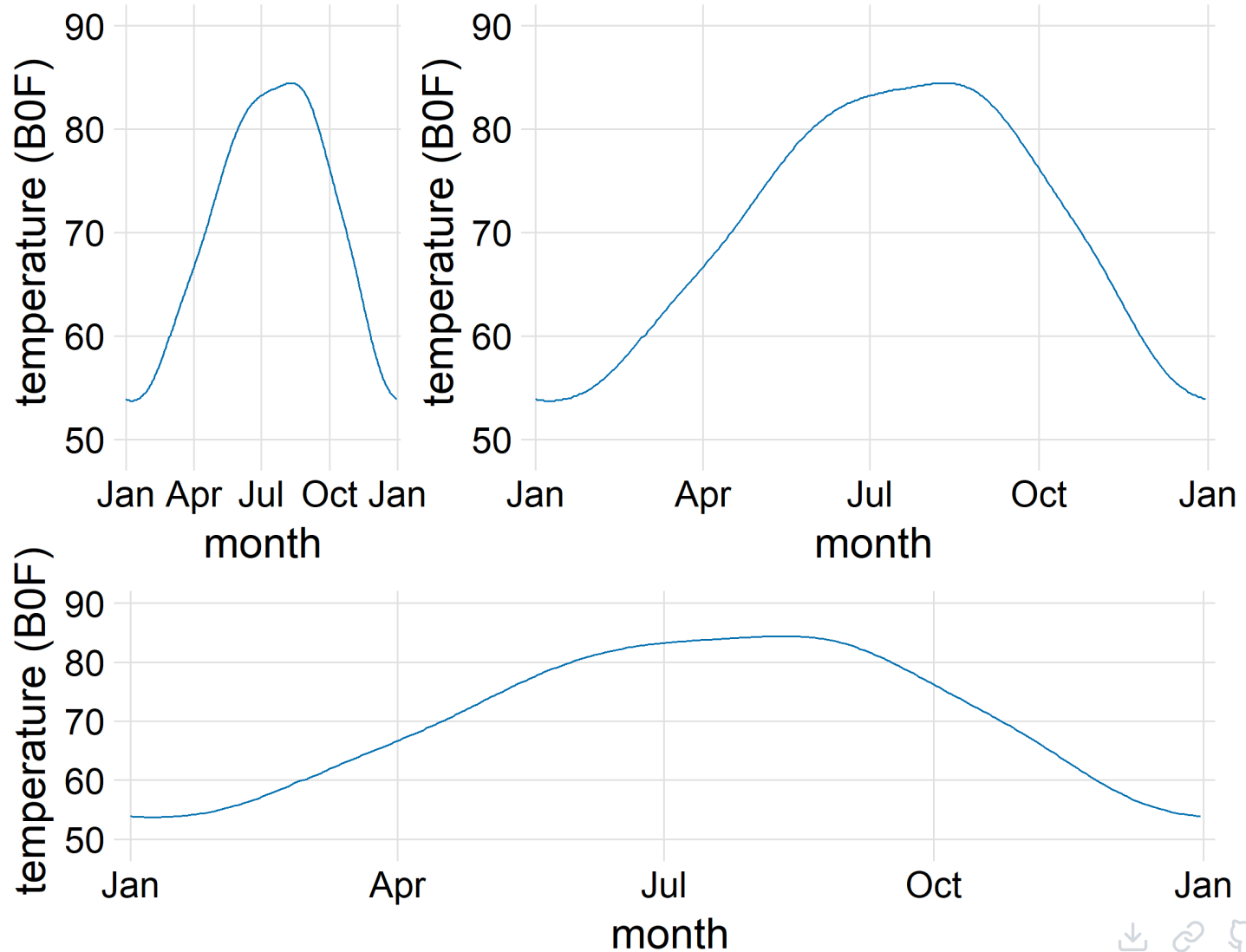
- Cartesian coordinates - what we generally use

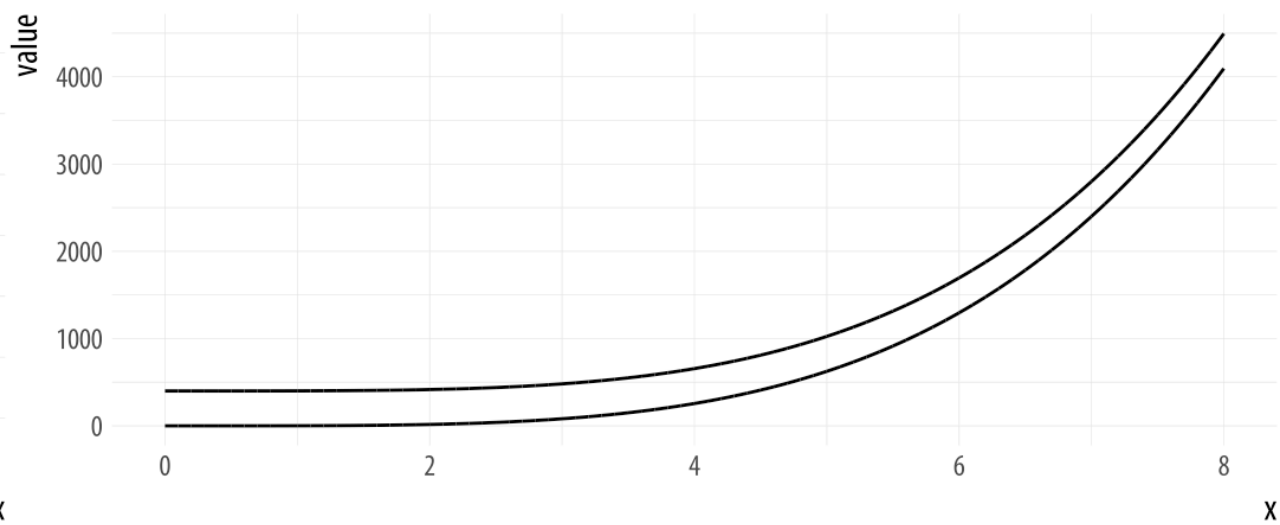
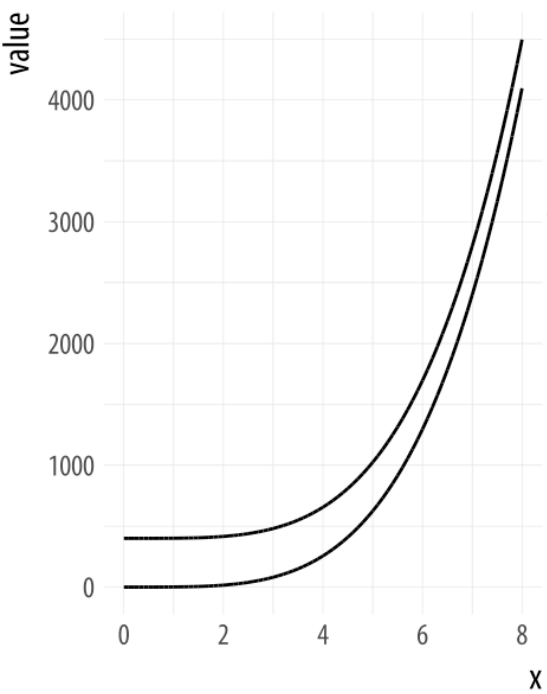


Different units



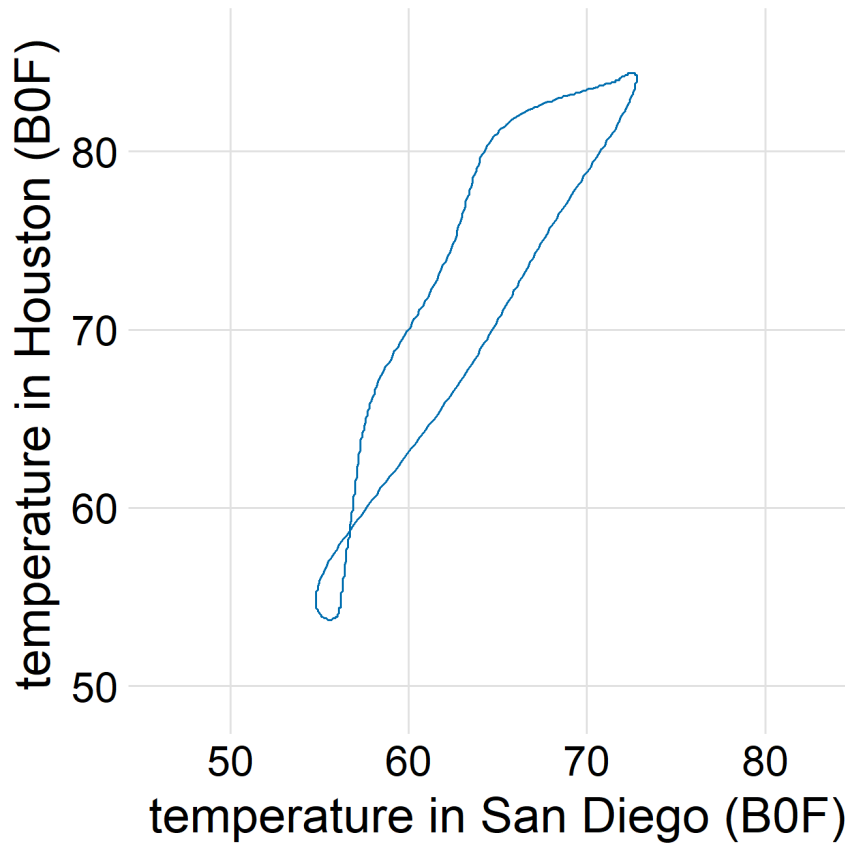
Aspect ratio





Same scales

Use `coord_fixed()`



Note - I think this is a weird plot, but the point remains

Changing aspect ratio

- Explore how your plot will look in its final size
- No hard/fast rules (if on different scales)
- Not even really rules of thumb
- Keep visual perception in mind
- Try your best to be truthful - show the trend/relation, but don't exaggerate/hide it

Gist

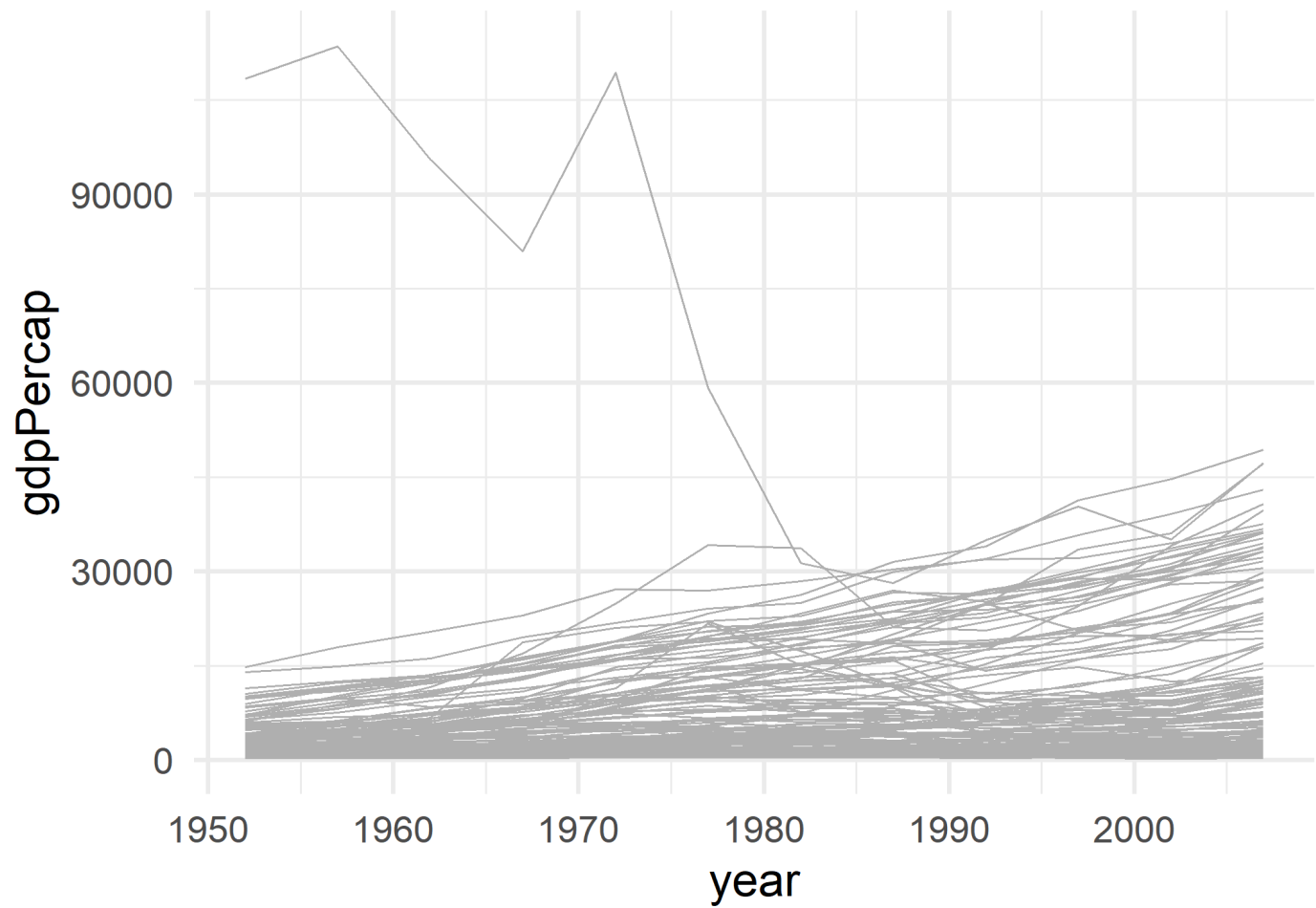
(side note: gists are a good way to share things)

- See the full code/example [here](#)
- You can use that link to play around:
 - Create a plot (could even be the example in the gist)
 - Try different aspect ratios by changing the width/length

Scale transformation

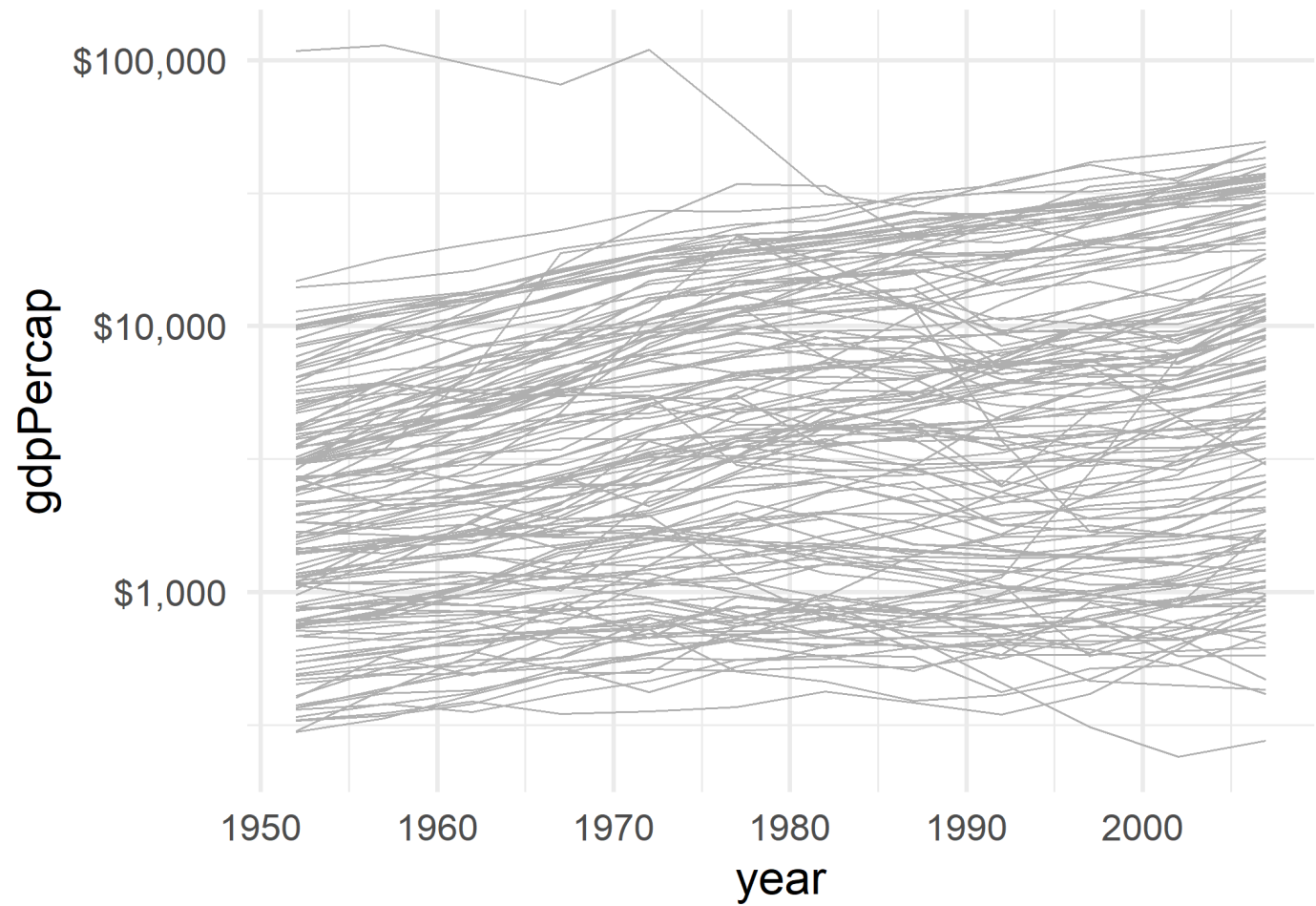
Raw scale

```
library(gapminder)
ggplot(gapminder, aes(year, gdpPercap)) +
  geom_line(aes(group = country),
            color = "gray70")
```

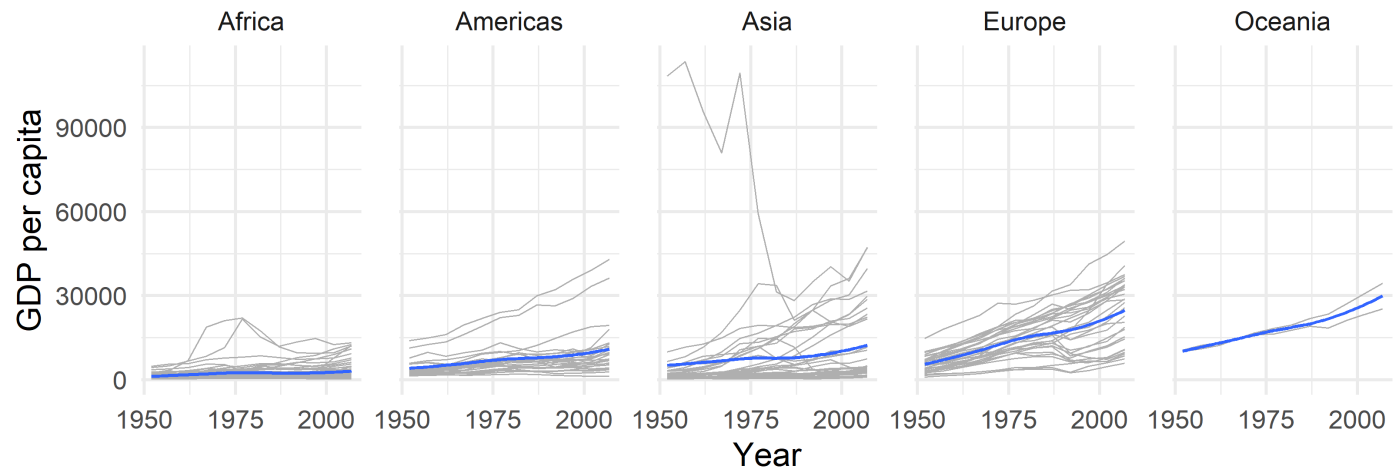


Log10 scale

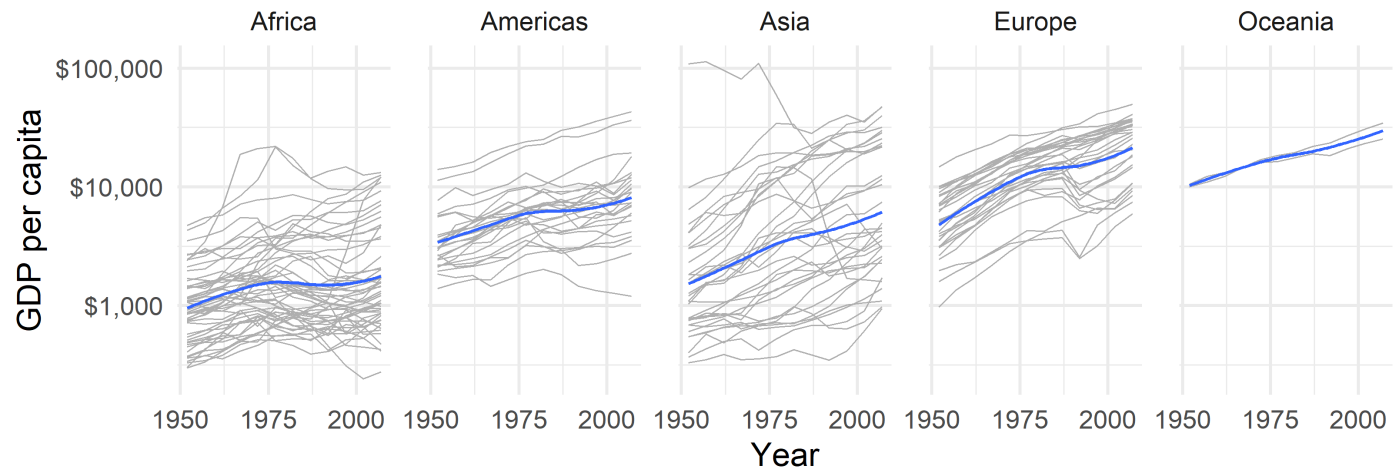
```
ggplot(gapminder, aes(year, gdpPercap)) +  
  geom_line(aes(group = country),  
            color = "gray70") +  
  scale_y_log10(labels = scales::dollar)
```



GDP per capita on Five Continents



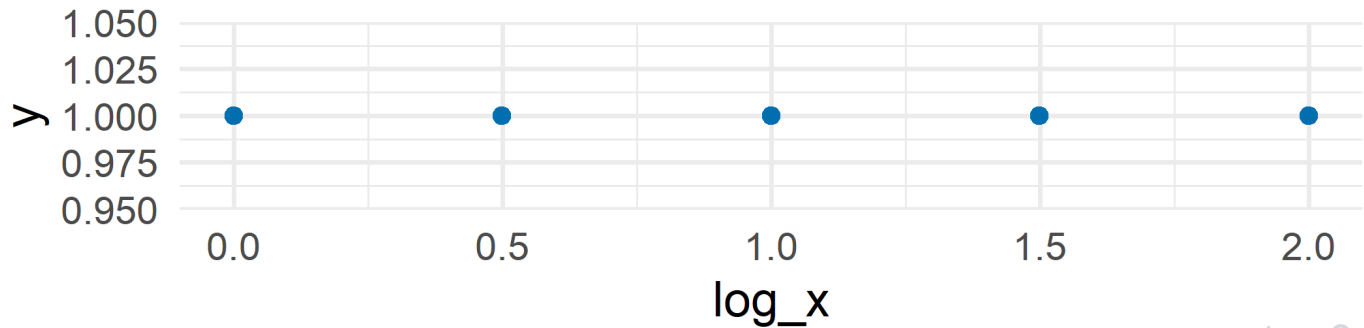
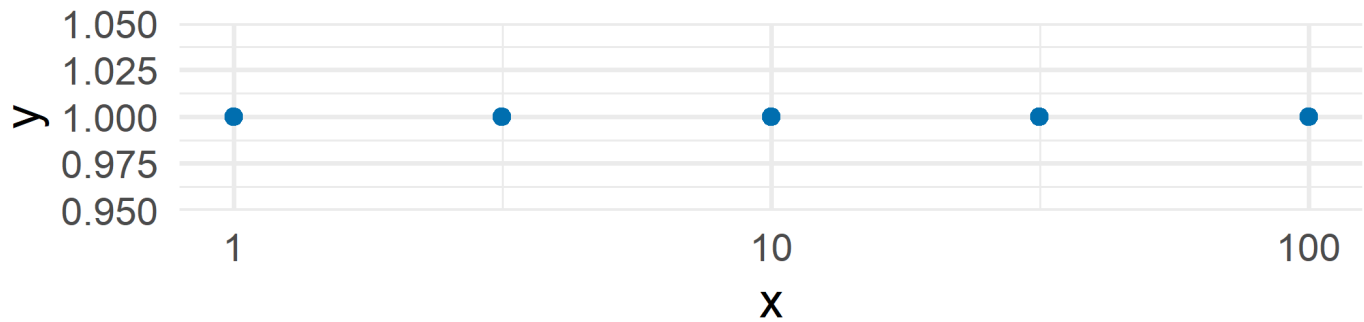
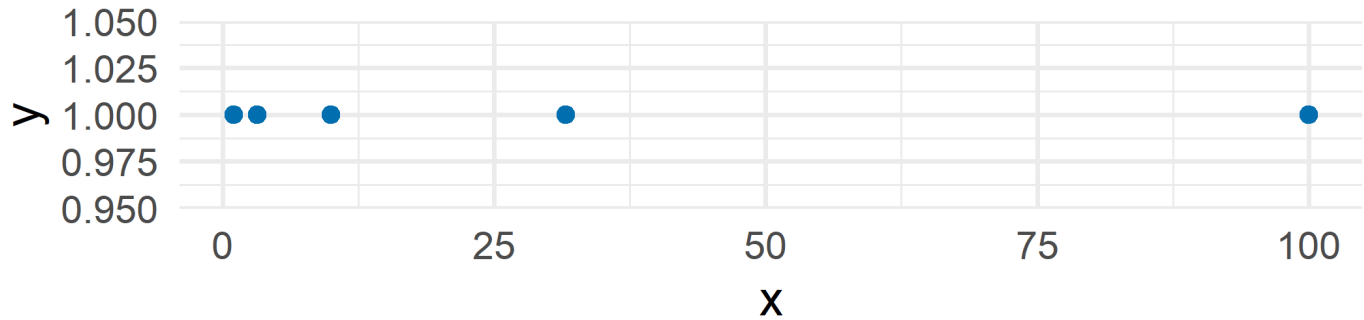
GDP per capita on Five Continents



Scales

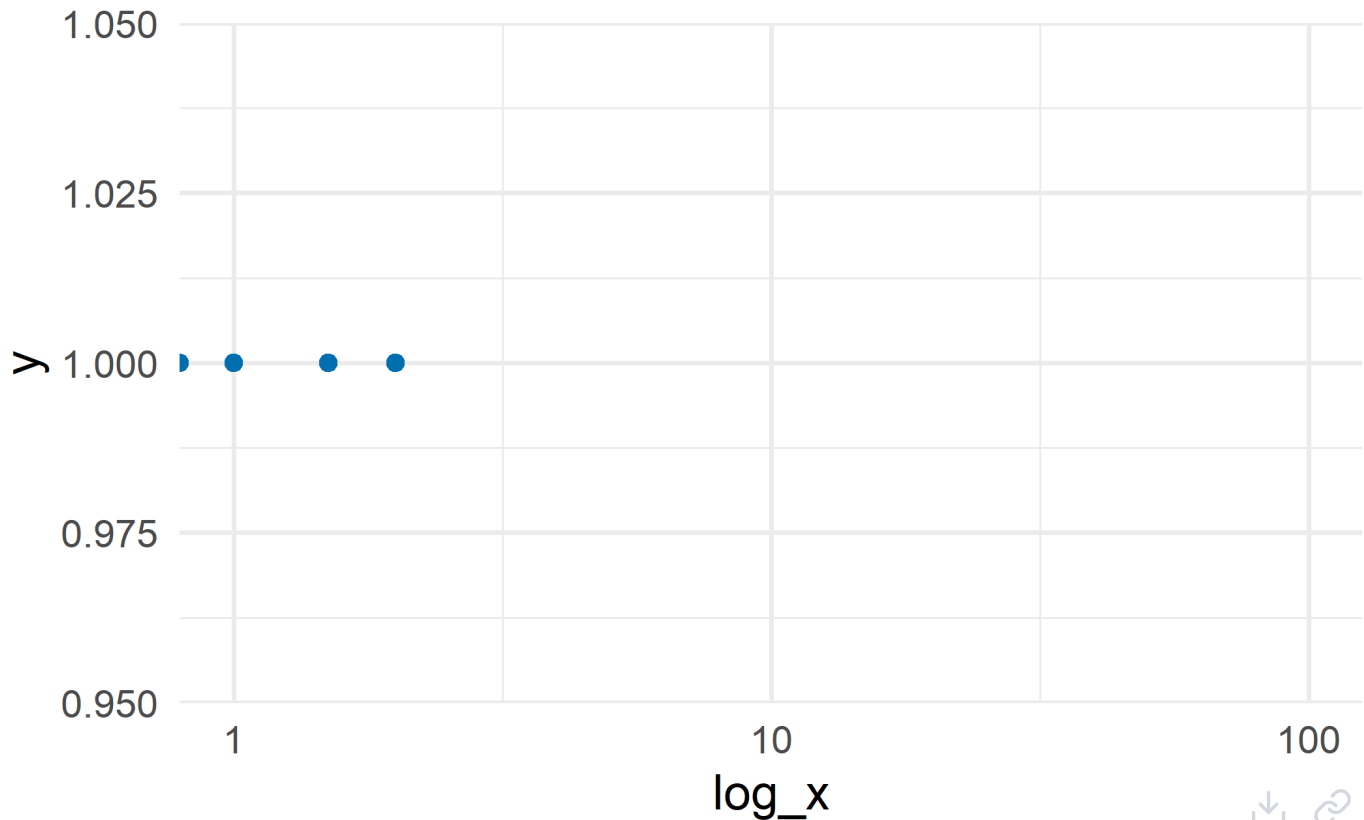
```
d <- tibble(x = c(1, 3.16, 10, 31.6, 100),  
            log_x = log10(x))  
  
ggplot(d, aes(x, 1)) +  
  geom_point(color = "#0072B2")  
  
ggplot(d, aes(x, 1)) +  
  geom_point(color = "#0072B2") +  
  scale_x_log10()  
  
ggplot(d, aes(log_x, 1)) +  
  geom_point(color = "#0072B2")
```

Scales



Don't transform twice

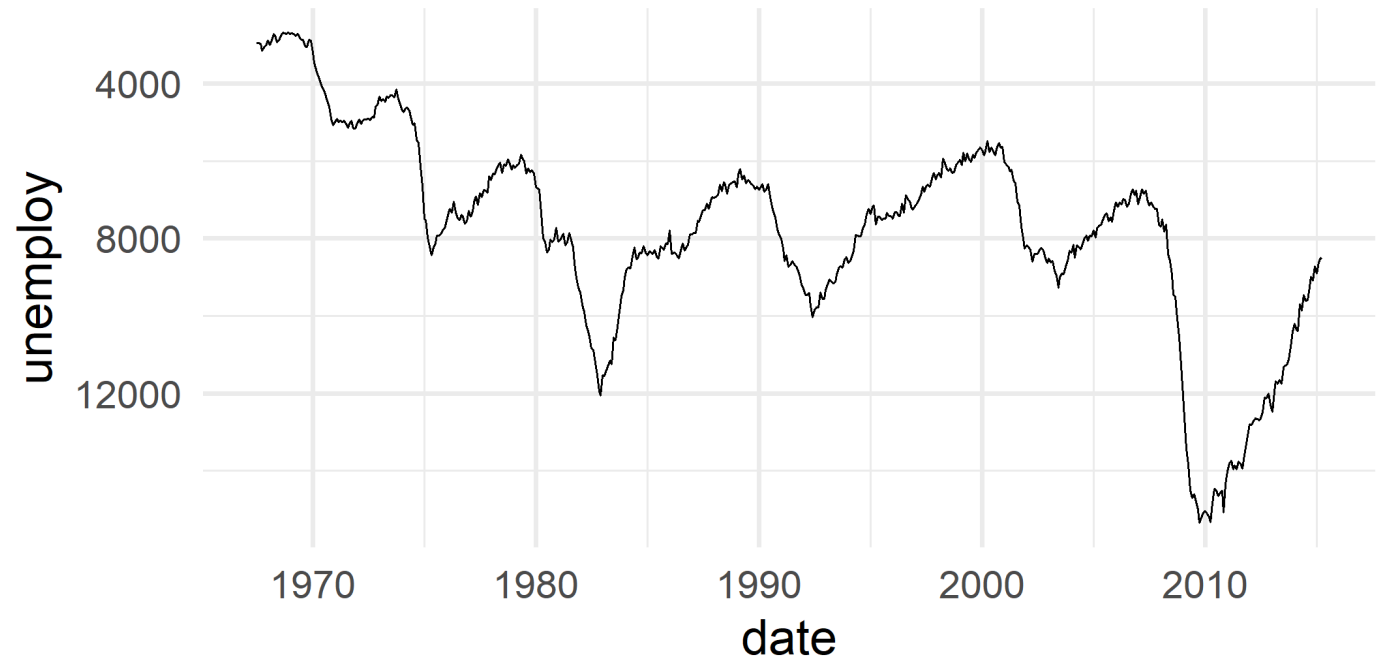
```
ggplot(d, aes(log_x, 1)) +  
  geom_point(color = "#0072B2") +  
  scale_x_log10() +  
  coord_cartesian(xlim = c(1, 100))
```



Other common transformations

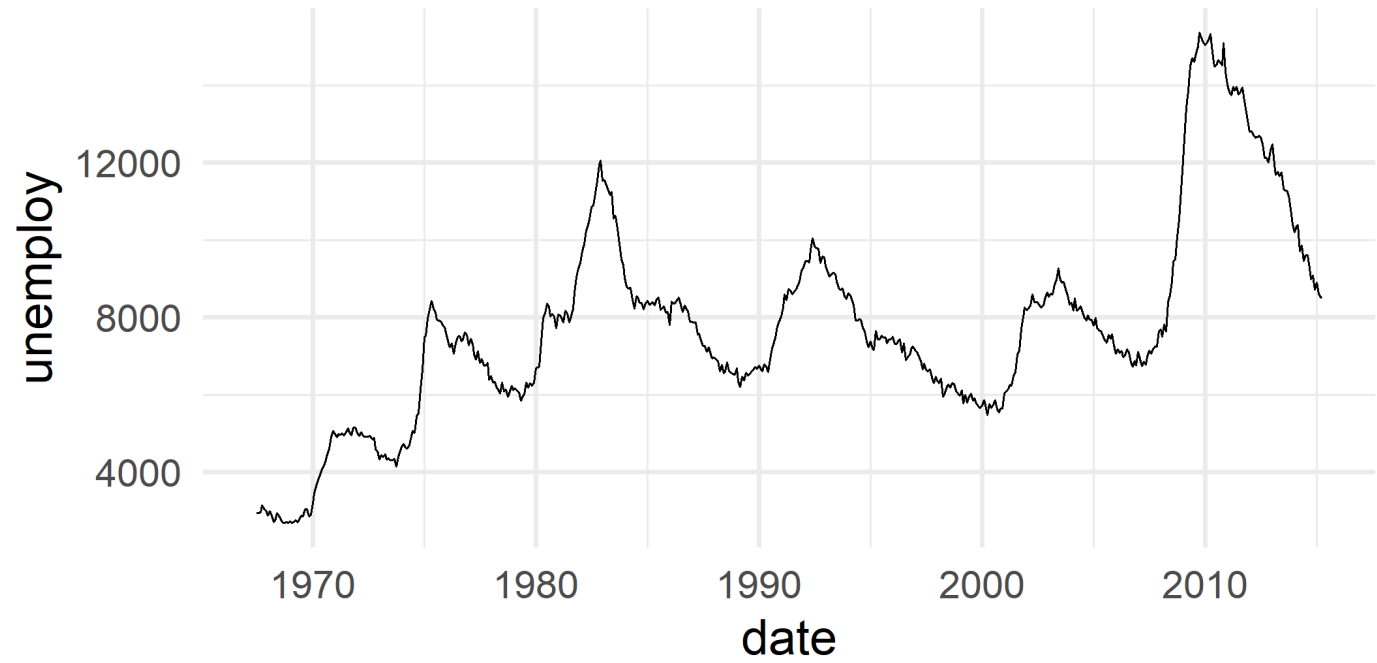
- `scale*_sqrt()`: Square-root scale
- `scale*_reverse()`: Reverse scale

```
ggplot(economics, aes(date, unemploy)) +  
  geom_line() +  
  scale_y_reverse()
```

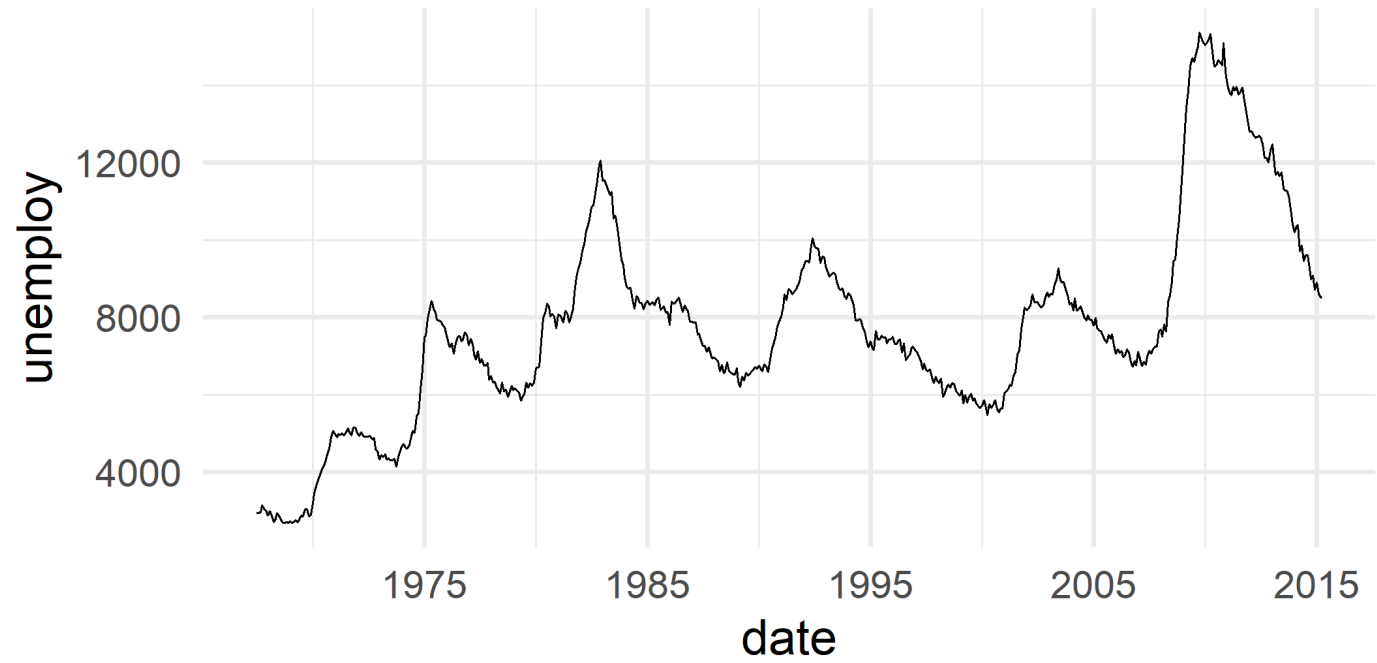
Date scales

```
ggplot(economics, aes(date, unemploy)) +  
  geom_line()
```



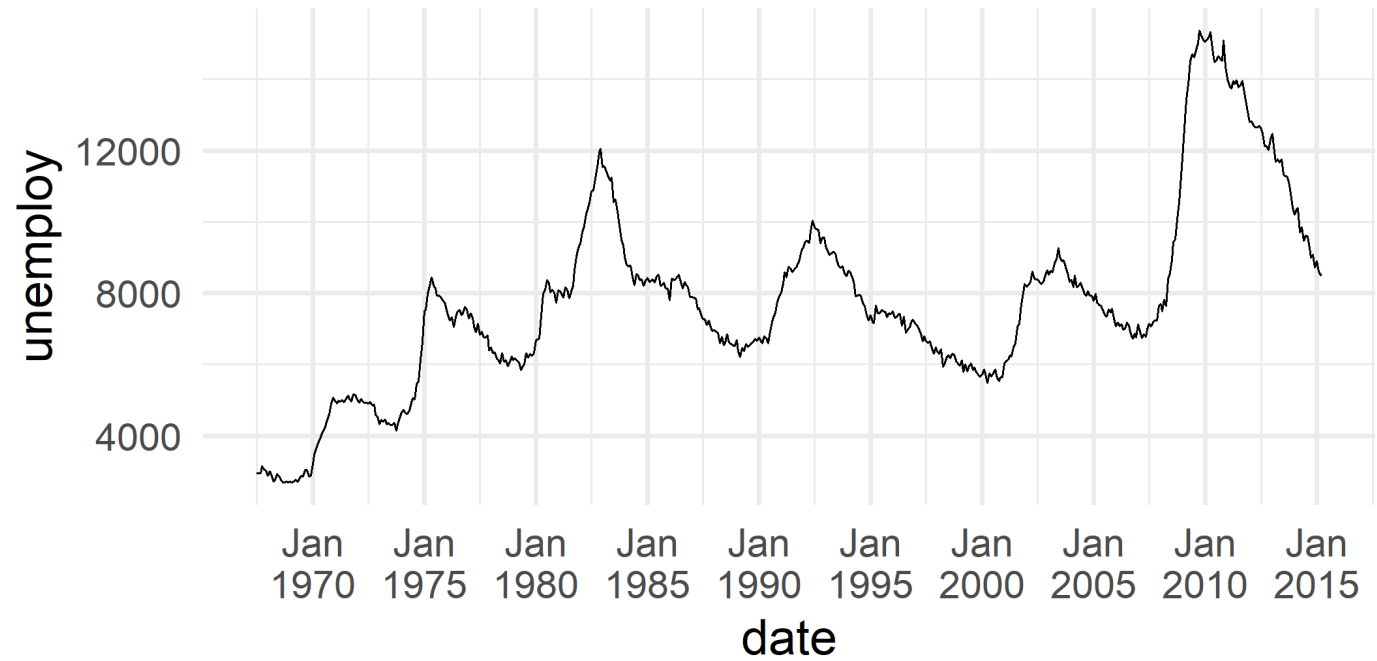
Customize date breaks

```
ggplot(economics, aes(date, unemploy)) +  
  geom_line() +  
  scale_x_date(date_breaks = "10 years",  
               date_labels = "%Y")
```



More date customization

```
ggplot(economics, aes(date, unemploy)) +  
  geom_line() +  
  scale_x_date(date_breaks = "5 years",  
              date_labels = "%b\n%Y")
```



Color Scales for Comparing Categories

Choosing Colors for Categories

Key principles:

- **Distinct:** Colors should be easily distinguishable
- **Accessible:** Consider colorblindness (8% of males, 0.5% of females)
- **Meaningful:** Colors can carry meaning (red = danger, green = success)
- **Limited:** Use 3-7 colors maximum for discrete categories

Annotations

Please follow along with code

Why annotate?

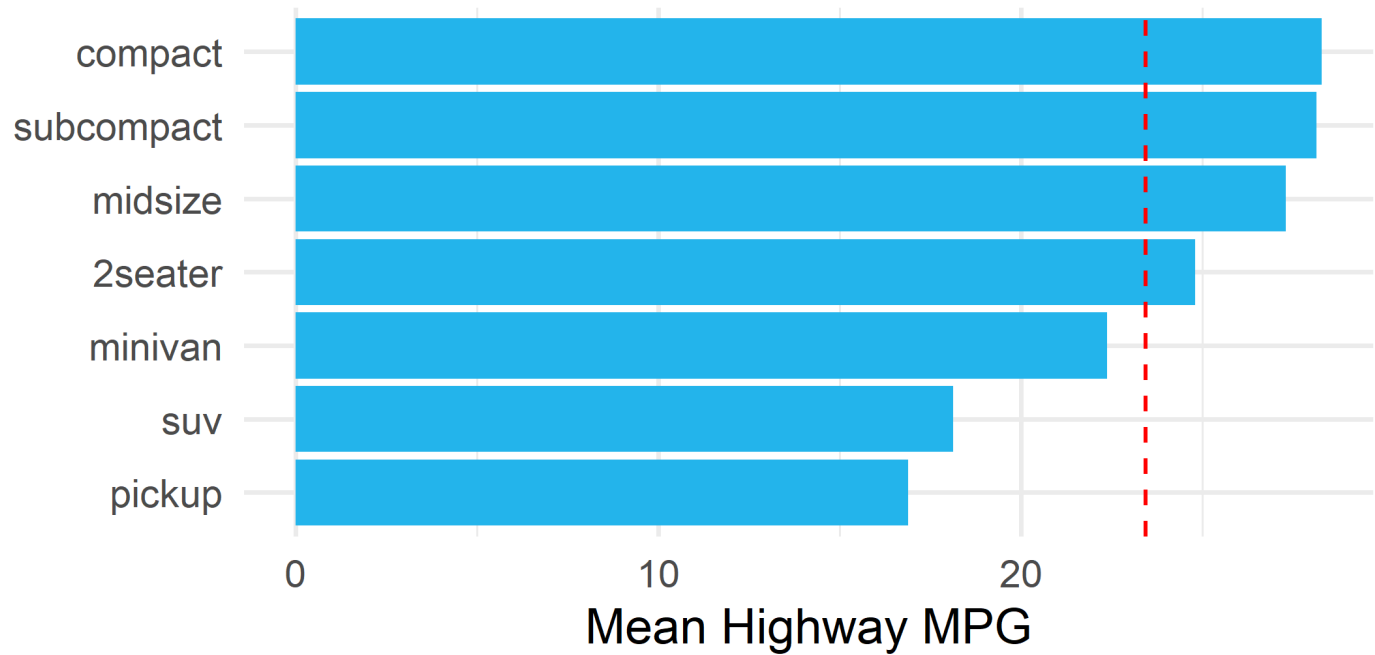
- Direct labels often better than legends
- Highlight important features
- Provide context
- Tell the story in your data
- Guide the reader's attention

Reference lines

```
mpg %>%
  group_by(class) %>%
  summarize(mean_hwy = mean(hwy)) %>%
  mutate(class = fct_reorder(class, mean_hwy)) %>%
  ggplot(aes(x = mean_hwy, y = class)) +
  geom_col(fill = "#25B6EE") +
  geom_vline(xintercept = mean(mpg$hwy),
             linetype = "dashed",
             color = "red",
             size = 1) +
  labs(x = "Mean Highway MPG",
       y = NULL,
       title = "Fuel Efficiency by Vehicle Class",
       subtitle = "Red line shows overall average")
```

Fuel Efficiency by Vehicle Class

Red line shows overall average

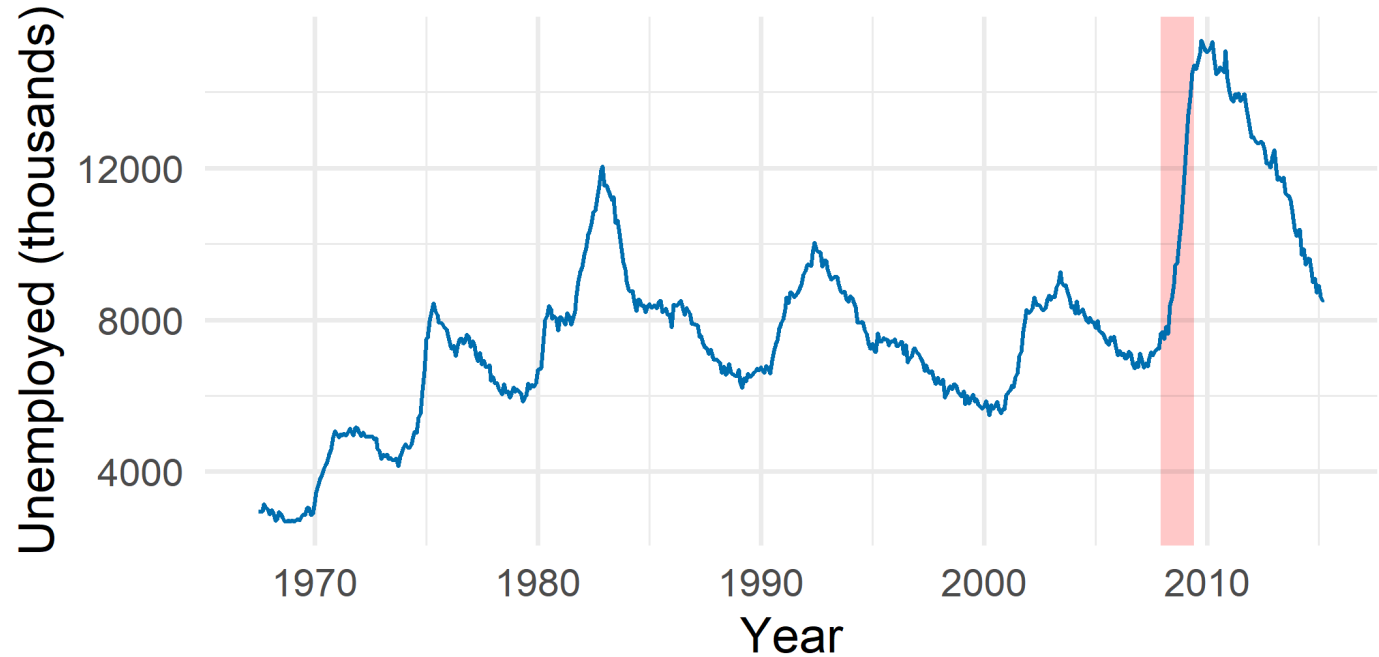


Shaded regions

```
ggplot(economics, aes(x = date, y = unemploy)) +  
  annotate("rect",  
    xmin = as.Date("2007-12-01"),  
    xmax = as.Date("2009-06-01"),  
    ymin = -Inf,  
    ymax = Inf,  
    fill = "red",  
    alpha = 0.2) +  
  geom_line(size = 1, color = "#0072B2") +  
  labs(x = "Year",  
    y = "Unemployed (thousands)",  
    title = "US Unemployment Over Time",  
    subtitle = "Shaded region shows Great Recession (Dec 2007
```

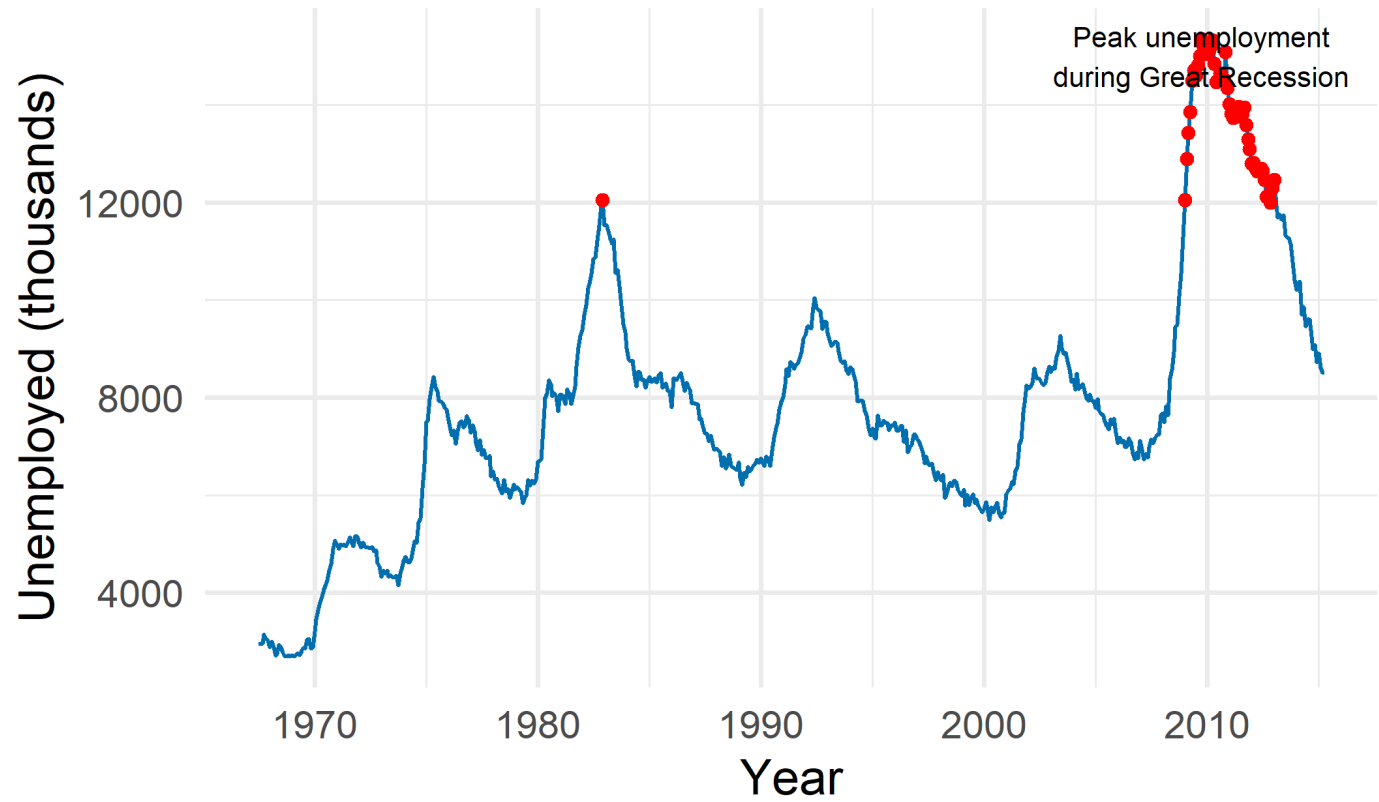
US Unemployment Over Time

Shaded region shows Great Recession (Dec 2007 - Jul 2009)



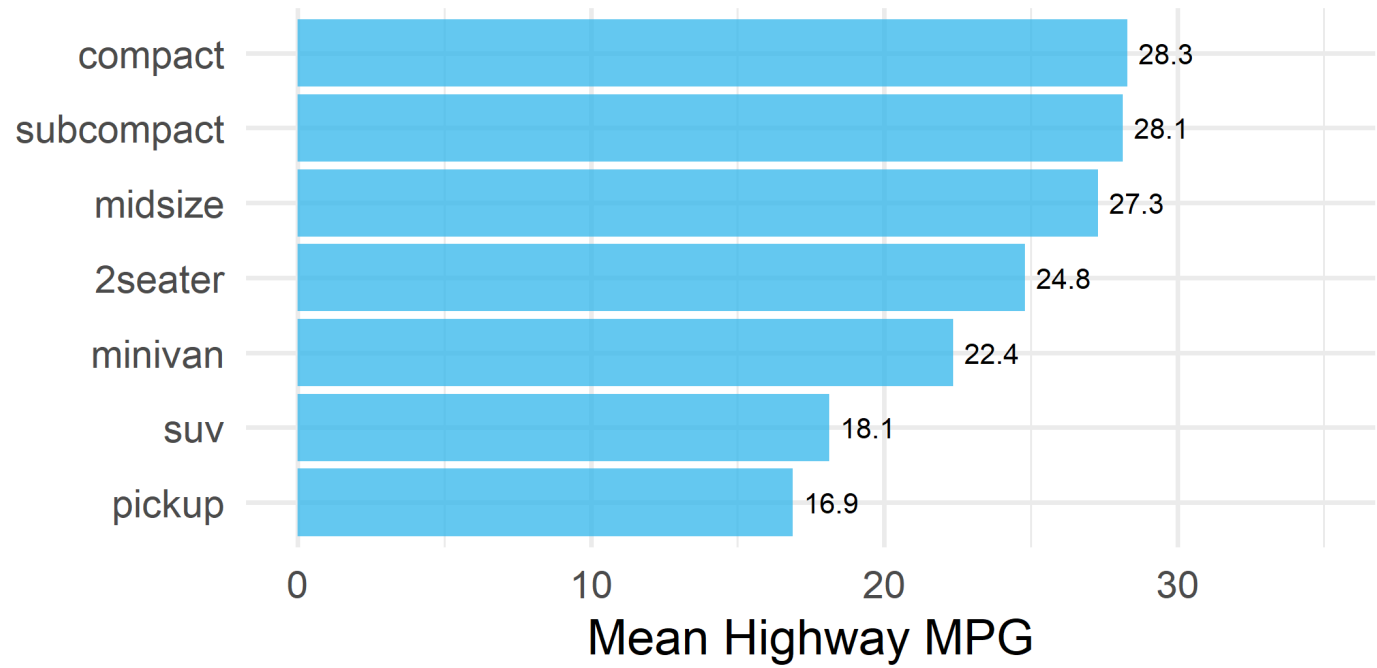
Text annotations

```
high_points <- economics %>%  
  filter(unemploy > 12000)  
  
ggplot(economics, aes(x = date, y = unemploy)) +  
  geom_line(size = 1, color = "#0072B2") +  
  geom_point(data = high_points,  
            size = 3,  
            color = "red") +  
  annotate("text",  
         x = as.Date("2009-10-01"),  
         y = 15000,  
         label = "Peak unemployment\nduring Great Recession",  
         hjust = 0.5,  
         size = 5) +  
  labs(x = "Year",  
       y = "Unemployed (thousands)")
```

Using geom_text

```
mpg %>%  
  group_by(class) %>%  
  summarize(mean_hwy = mean(hwy),  
            n = n()) %>%  
  mutate(class = fct_reorder(class, mean_hwy)) %>%  
  ggplot(aes(x = mean_hwy, y = class)) +  
  geom_col(fill = "#25B6EE", alpha = 0.7) +  
  geom_text(aes(label = round(mean_hwy, 1)),  
            hjust = -0.2,  
            size = 5) +  
  xlim(0, 35) +  
  labs(x = "Mean Highway MPG",  
       y = NULL)
```



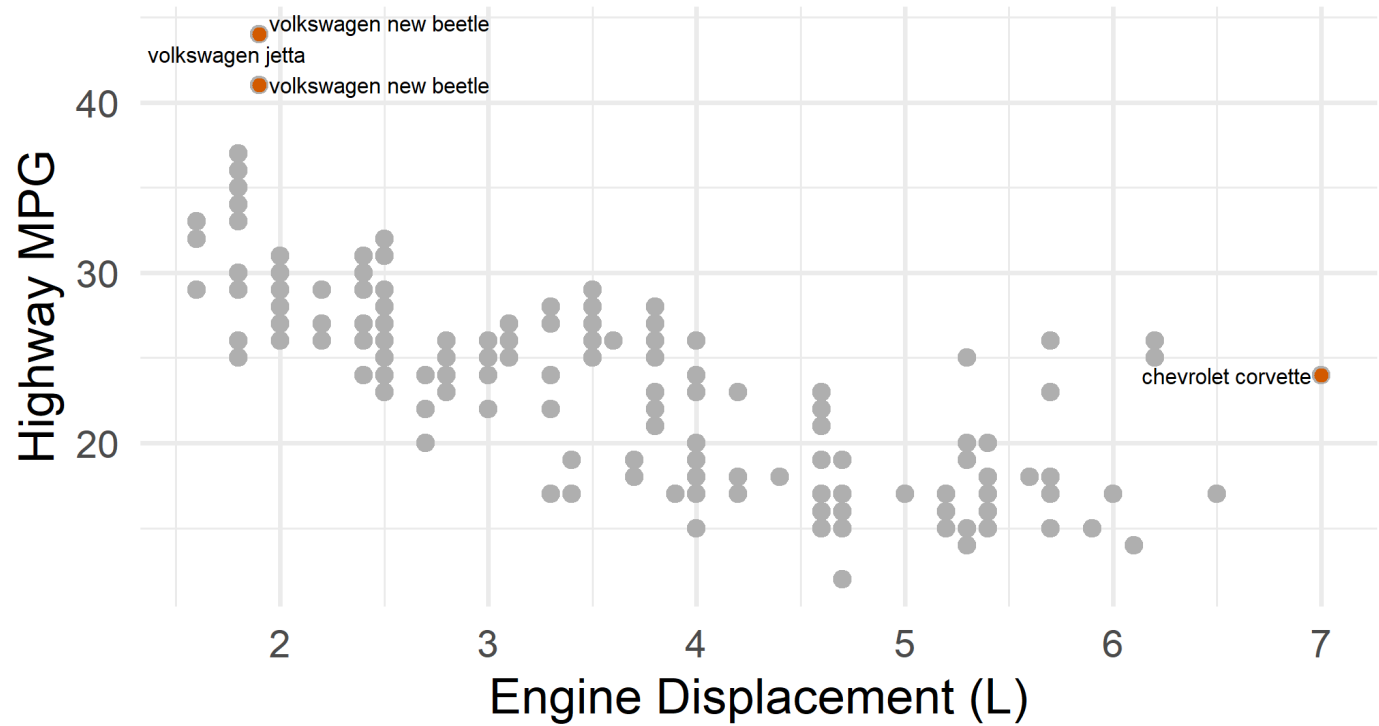
ggrepel for non-overlapping labels

```
library(ggrepel)

mpg_labeled <- mpg %>%
  filter(hwy > 40 | displ > 6.5)

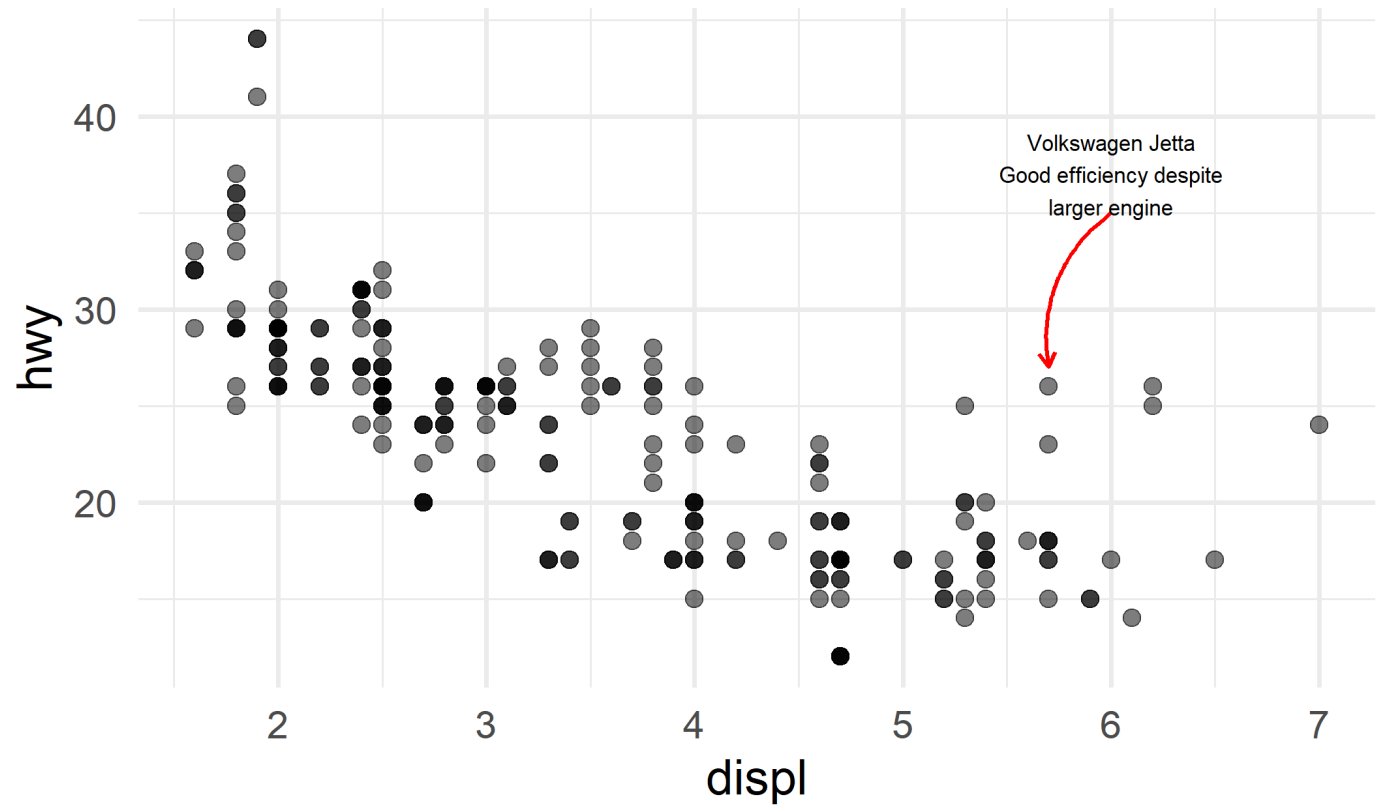
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(color = "gray70") +
  geom_point(data = mpg_labeled, color = "#D55E00", size = 3) +
  geom_text_repel(data = mpg_labeled,
                  aes(label = paste(manufacturer, model)),
                  size = 4) +
  labs(x = "Engine Displacement (L)",
       y = "Highway MPG",
       title = "Highlighting Extreme Vehicles")
```

Highlighting Extreme Vehicles



Arrows and segments

```
annotation_data <- tibble(  
  x = 6, y = 35,  
  xend = 5.7, yend = 27,  
  label = "Volkswagen Jetta"  
)  
  
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(alpha = 0.5) +  
  geom_curve(data = annotation_data,  
            aes(x = x, y = y, xend = xend, yend = yend),  
            arrow = arrow(length = unit(0.3, "cm")),  
            curvature = 0.3,  
            color = "red",  
            size = 1) +  
  annotate("text",  
          x = 6,  
          y = 37,  
          label = "Volkswagen Jetta\nGood efficiency despite\nl",  
          hjust = 0.5,  
          size = 4)
```

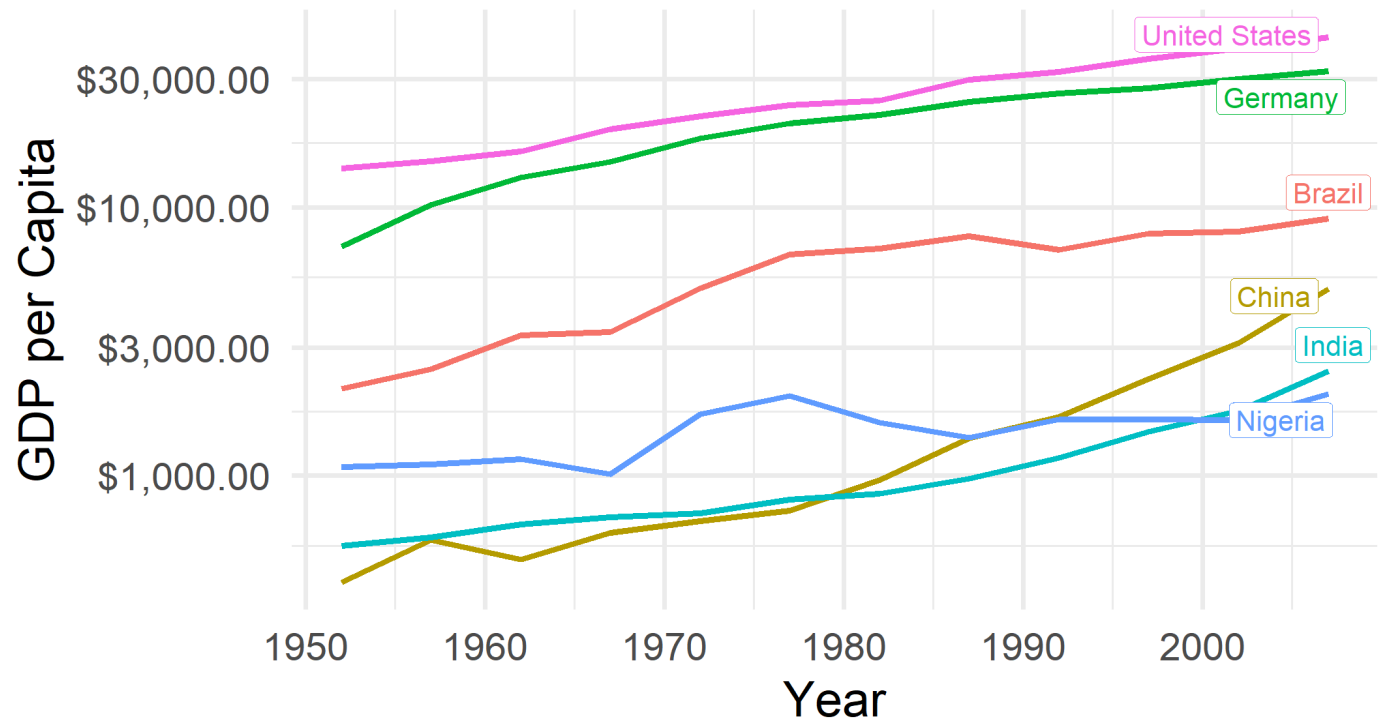


Direct labels with gghighlight

```
library(gghighlight)

gapminder %>%
  filter(country %in% c("United States", "China", "India",
                        "Germany", "Brazil", "Nigeria")) %>%
  ggplot(aes(x = year, y = gdpPercap, color = country)) +
  geom_line(size = 1.5) +
  gghighlight(use_direct_label = TRUE,
              label_params = list(size = 5)) +
  scale_y_log10(labels = scales::dollar) +
  labs(x = "Year",
       y = "GDP per Capita",
       title = "Economic Growth in Six Countries") +
  theme(legend.position = "none")
```


Economic Growth in Six Countries



Themes in Detail

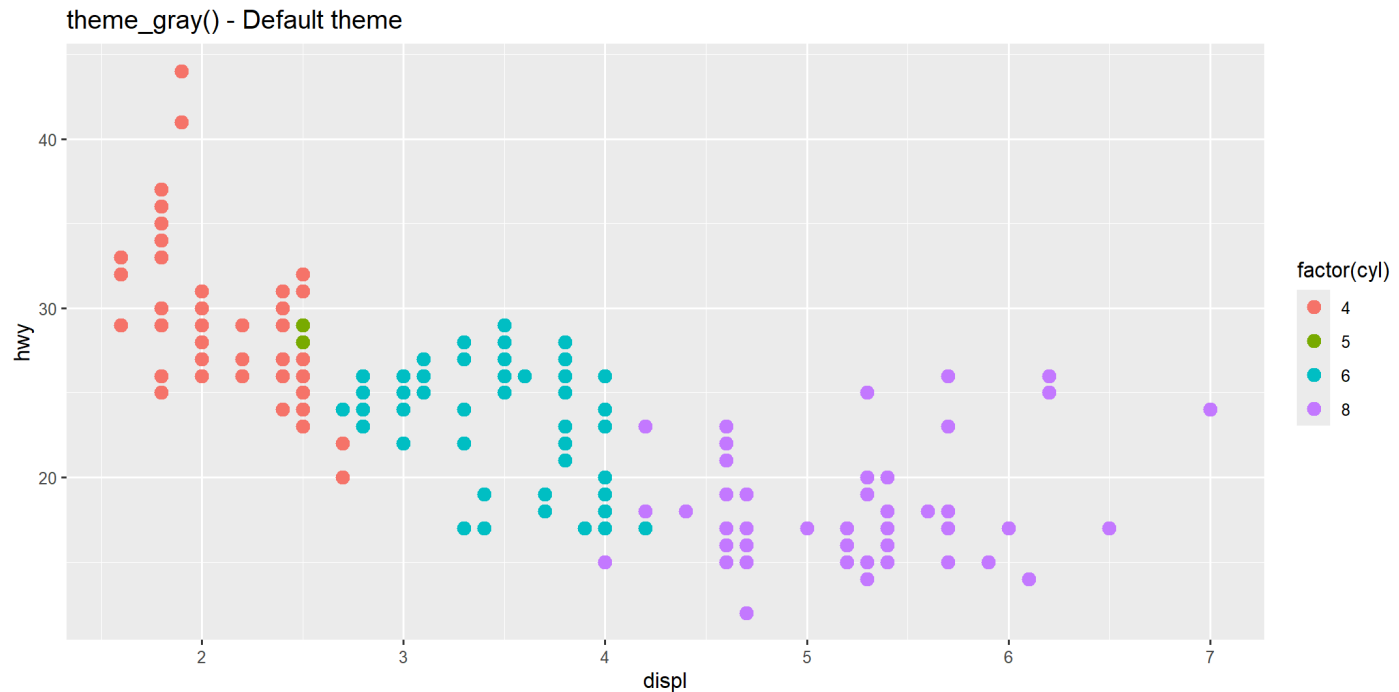
Built-in Themes

ggplot2 comes with 8 complete themes:

```
theme_gray()      # Default
theme_bw()        # Black and white
theme_minimal()   # Minimal
theme_classic()   # Classic
theme_light()     # Light
theme_dark()      # Dark
theme_void()      # Void (no axes)
theme_test()      # For visual unit tests
```

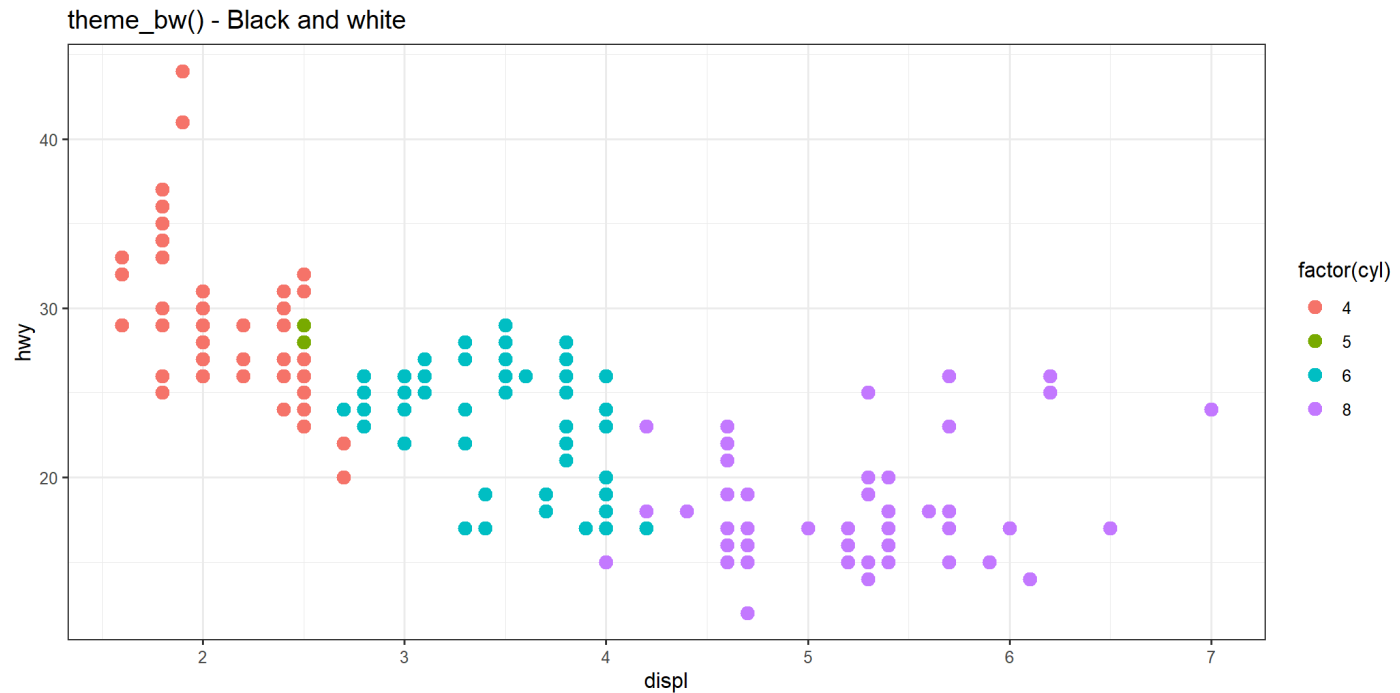
theme_gray() - Default

```
p <- ggplot(mpg, aes(x = displ, y = hwy, color = factor(cyl))) +  
  geom_point(size = 3) +  
  labs(title = "theme_gray() - Default theme")  
  
p + theme_gray()
```



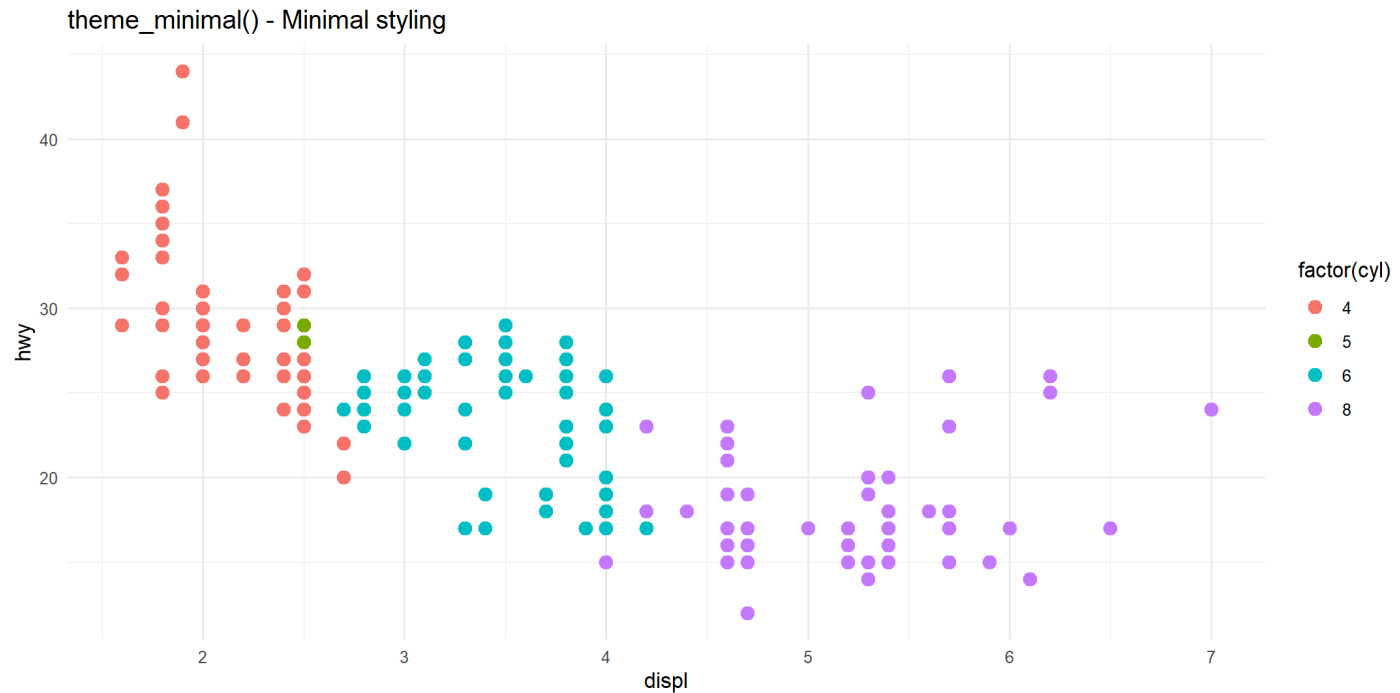
theme_bw()

```
p + theme_bw() +  
  labs(title = "theme_bw() - Black and white")
```



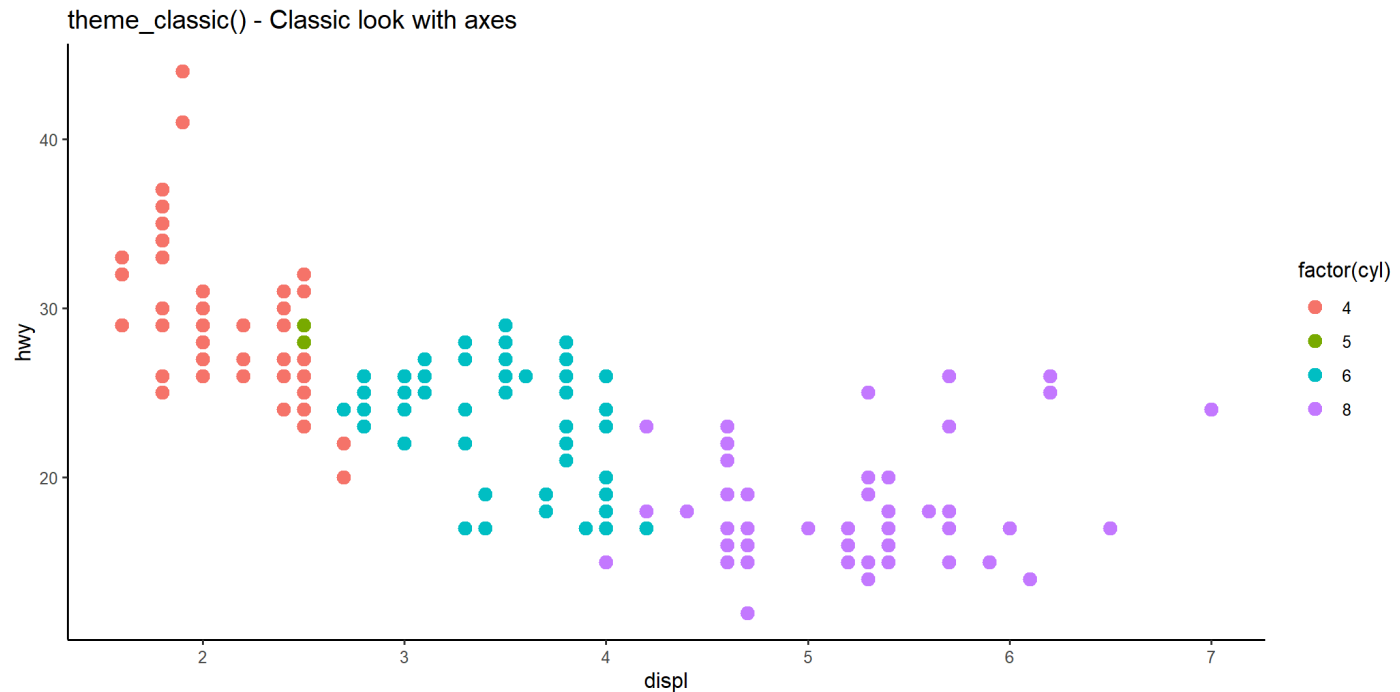
theme_minimal()

```
p + theme_minimal() +  
  labs(title = "theme_minimal() - Minimal styling")
```



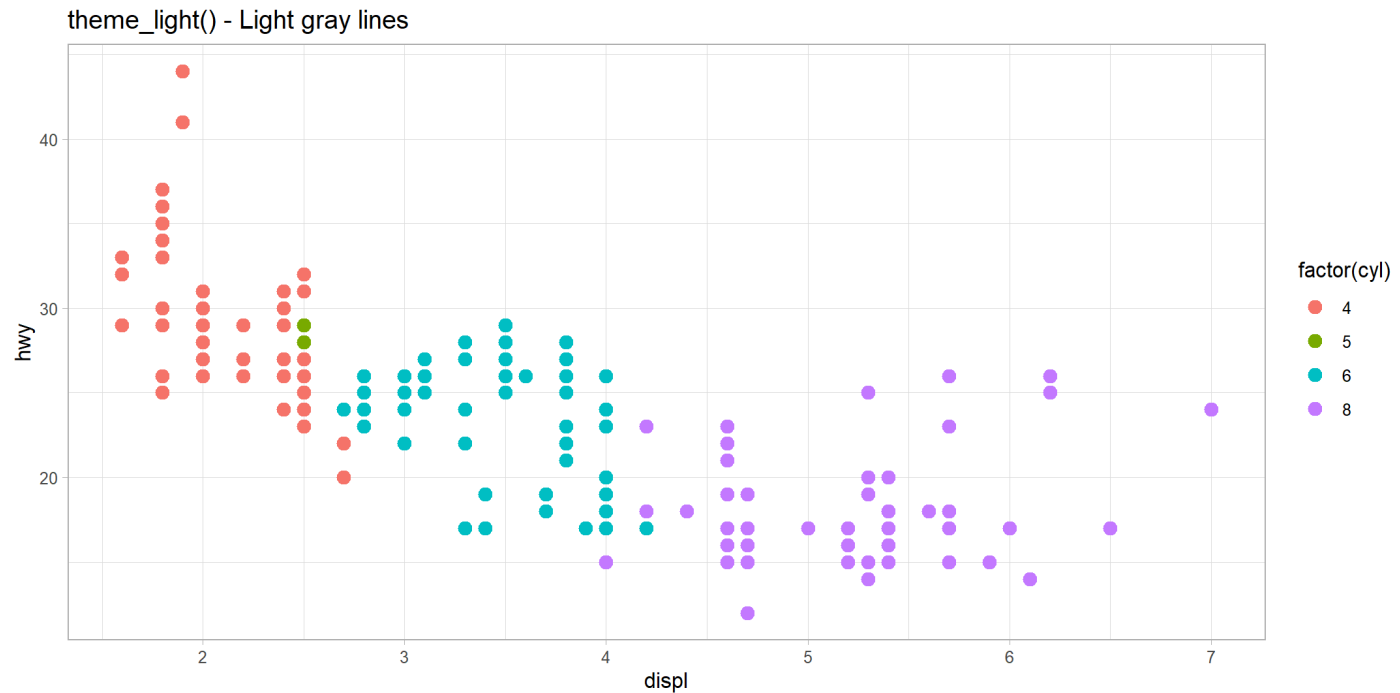
theme_classic()

```
p + theme_classic() +  
  labs(title = "theme_classic() - Classic look with axes")
```



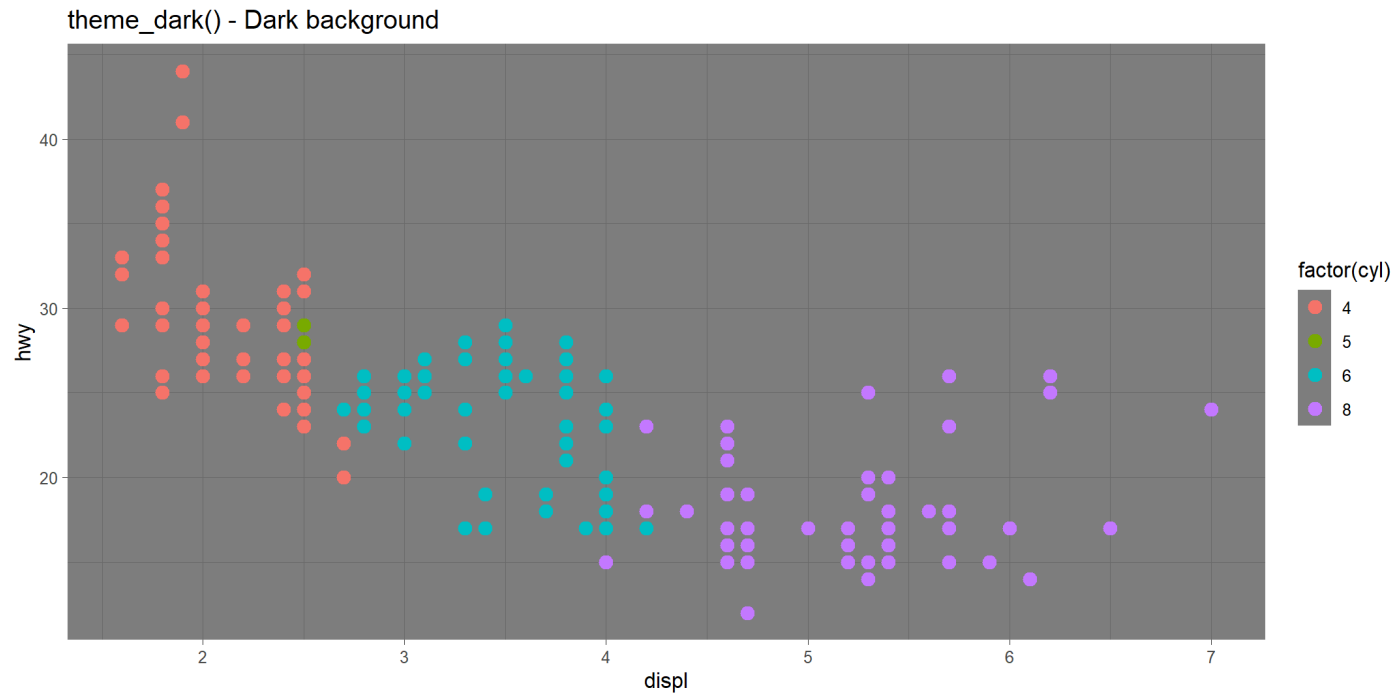
theme_light()

```
p + theme_light() +  
  labs(title = "theme_light() - Light gray lines")
```



theme_dark()

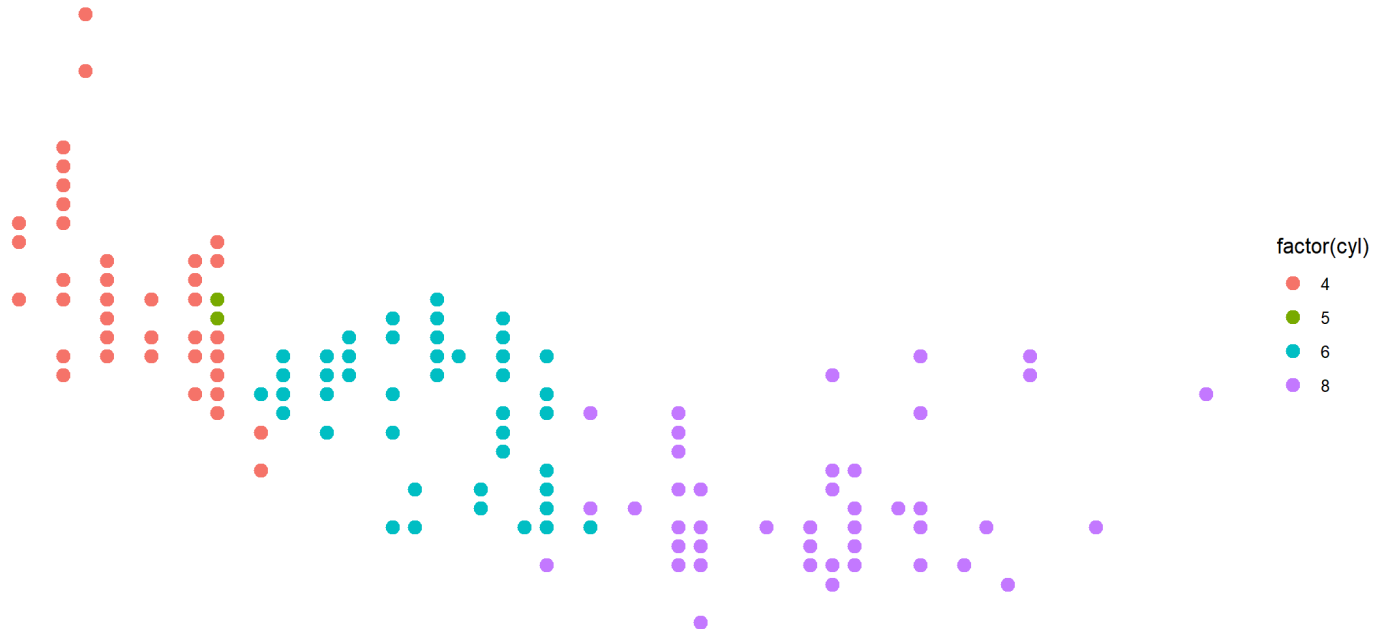
```
p + theme_dark() +  
  labs(title = "theme_dark() - Dark background")
```



theme_void()

```
p + theme_void() +  
  labs(title = "theme_void() - No axes or background")
```

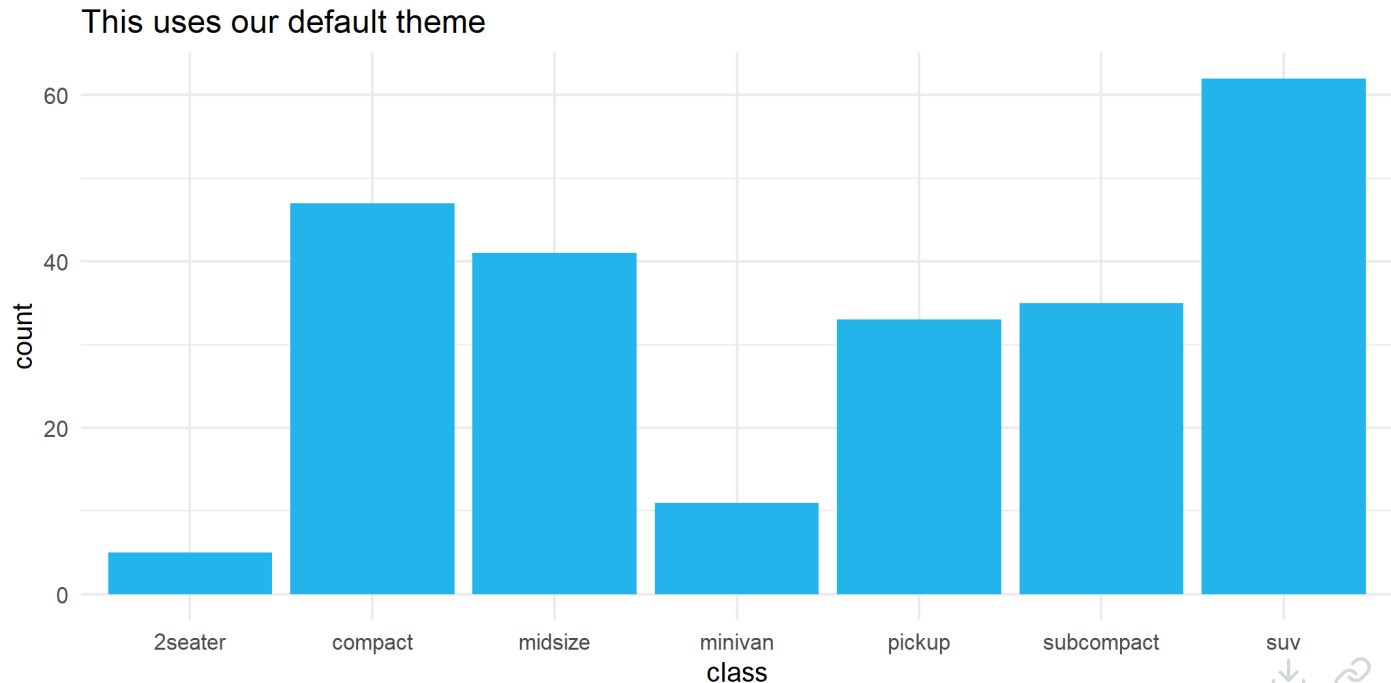
theme_void() - No axes or background



Setting a Default Theme

```
# At the beginning of your script
theme_set(theme_minimal(base_size = 14))

# Now all plots will use this theme
ggplot(mpg, aes(x = class)) +
  geom_bar(fill = "#25B6EE") +
  labs(title = "This uses our default theme")
```



Theme Element Functions

Four main functions for theme elements:

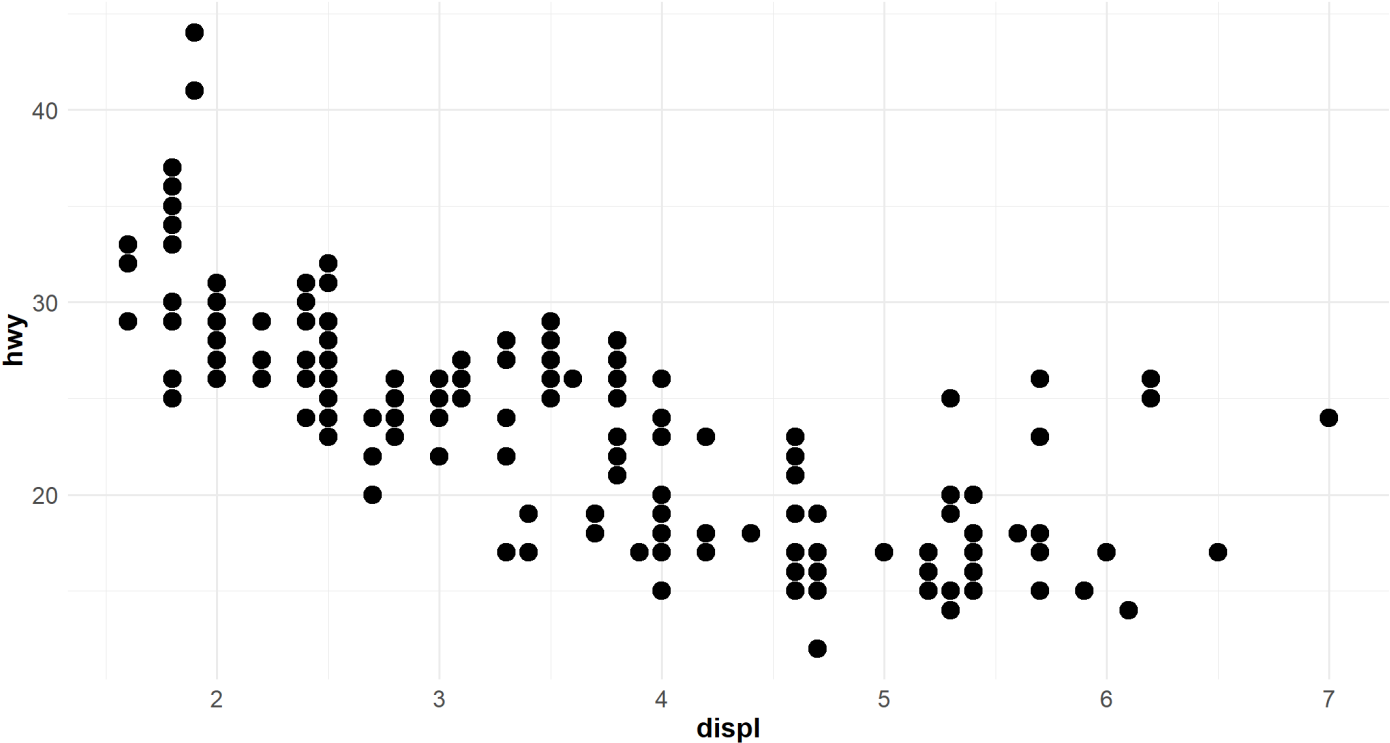
- `element_text()`: Text elements (axis labels, titles, etc.)
- `element_line()`: Line elements (axis lines, grid lines)
- `element_rect()`: Rectangle elements (plot background, panel background)
- `element_blank()`: Remove element completely

Customizing Text Elements

```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point() +  
  labs(title = "Customized Text Elements",  
        subtitle = "Engine displacement vs. Highway MPG") +  
  theme_minimal() +  
  theme(  
    plot.title = element_text(size = 20, face = "bold", color = "teal"),  
    plot.subtitle = element_text(size = 14, face = "italic", color = "teal"),  
    axis.title = element_text(size = 14, face = "bold"),  
    axis.text = element_text(size = 12, color = "gray30")  
  )
```

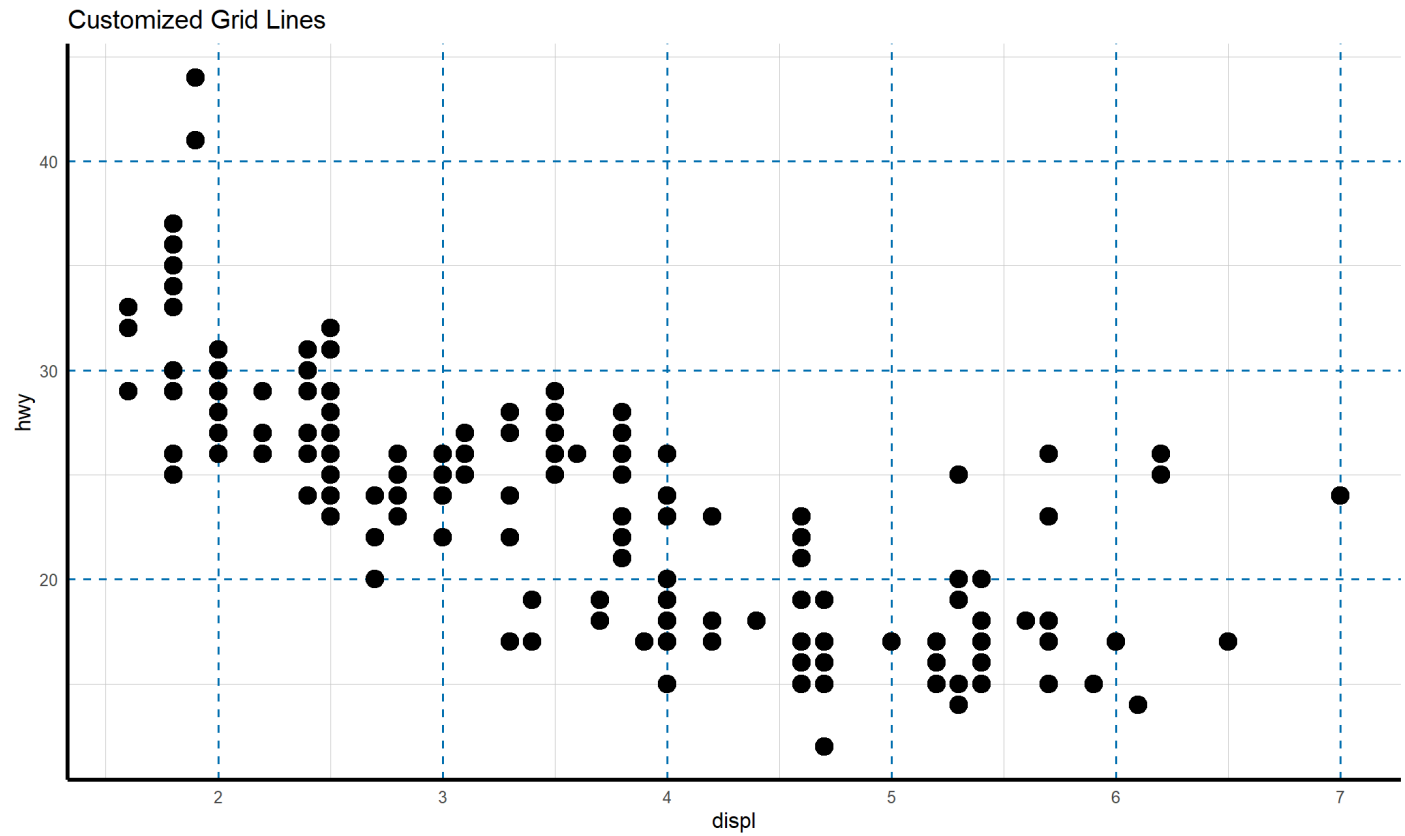
Customized Text Elements

Engine displacement vs. Highway MPG



Customizing Lines

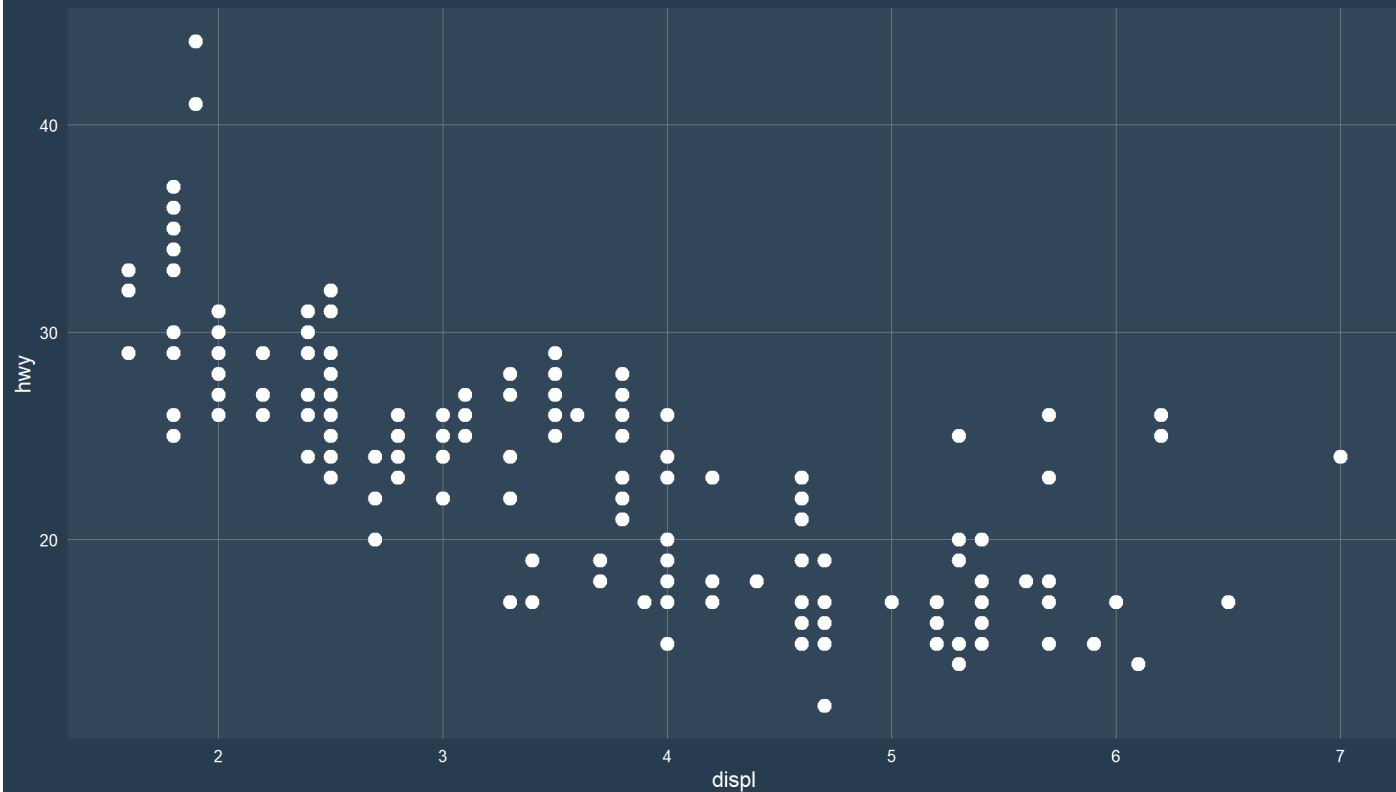
```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point() +  
  theme_minimal() +  
  theme(  
    panel.grid.major = element_line(color = "#0072B2", size = 0.5),  
    panel.grid.minor = element_line(color = "gray80", size = 0.25),  
    axis.line = element_line(color = "black", size = 1)  
  ) +  
  labs(title = "Customized Grid Lines")
```



Customizing Rectangles

```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(color = "white", size = 3) +  
  labs(title = "Dark Theme with Custom Background") +  
  theme_minimal() +  
  theme(  
    plot.background = element_rect(fill = "#2c3e50", color = NA),  
    panel.background = element_rect(fill = "#34495e", color = NA),  
    panel.grid.major = element_line(color = "#7f8c8d", size = 0.5),  
    panel.grid.minor = element_blank(),  
    text = element_text(color = "white"),  
    axis.text = element_text(color = "white"),  
    plot.title = element_text(size = 16, face = "bold", color = "white"),  
  )
```

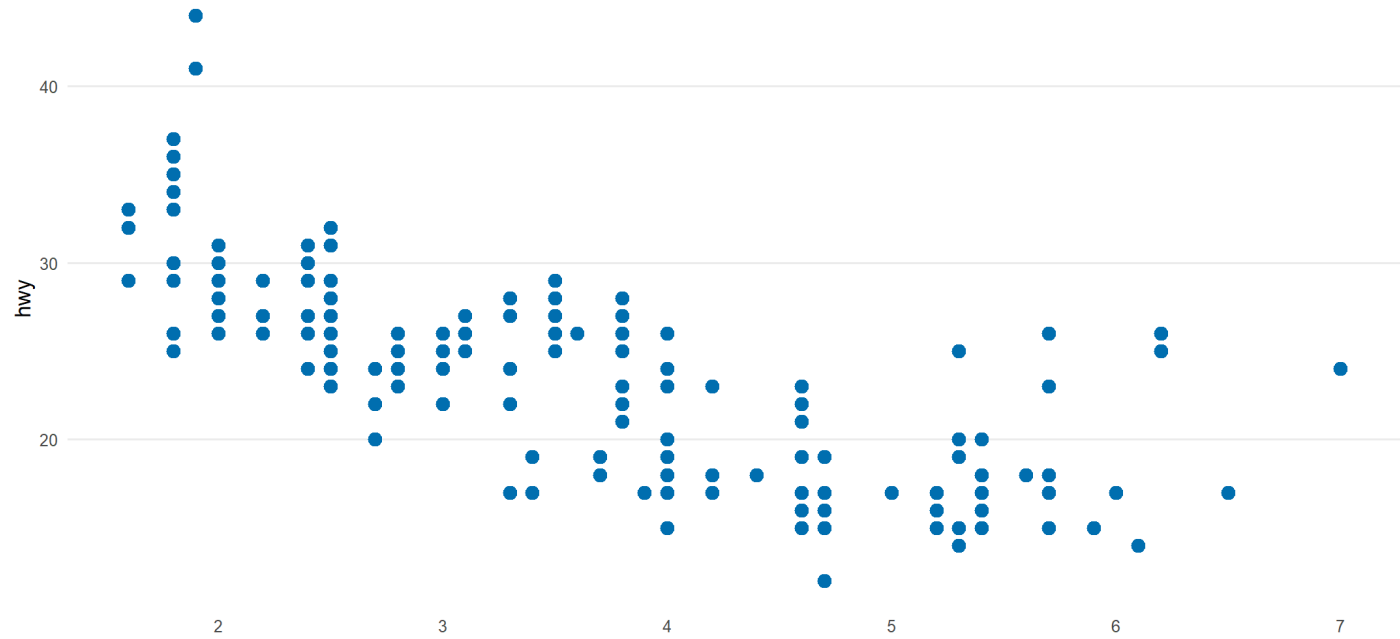
Dark Theme with Custom Background



Removing Elements

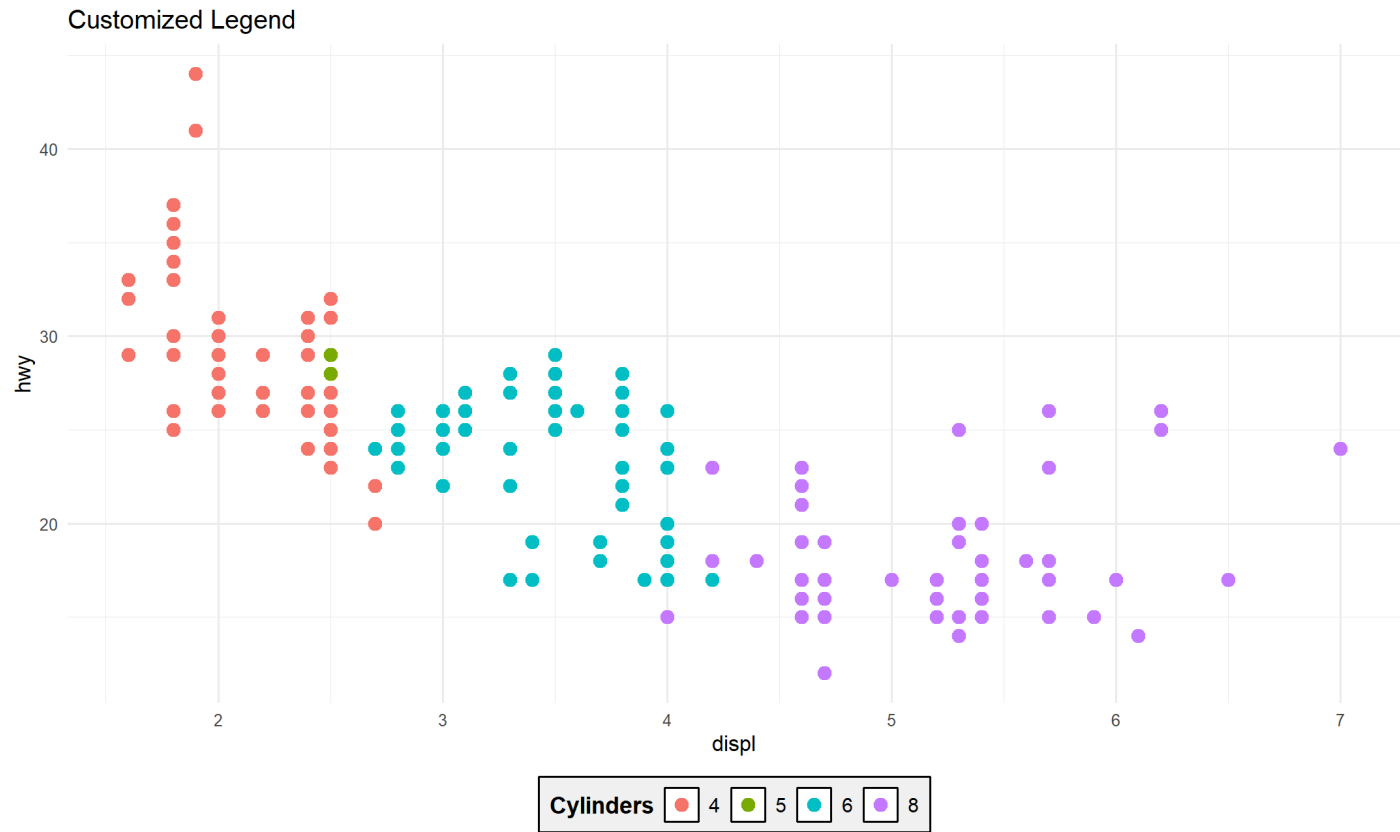
```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(size = 3, color = "#0072B2") +  
  labs(title = "Minimal Theme - Elements Removed") +  
  theme_minimal() +  
  theme(  
    panel.grid.minor = element_blank(), # Remove minor grid  
    panel.grid.major.x = element_blank(), # Remove vertical grid  
    axis.title.x = element_blank(), # Remove x-axis title  
    legend.position = "none" # Remove legend  
  )
```

Minimal Theme - Elements Removed



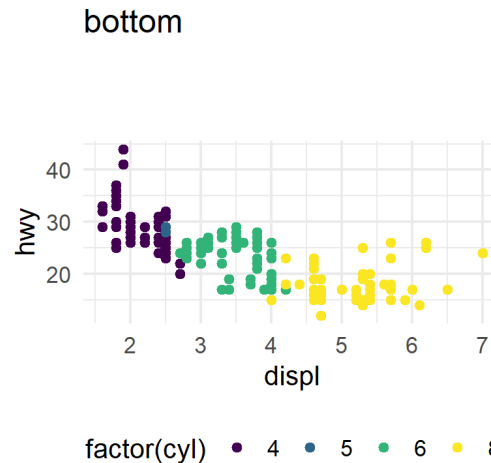
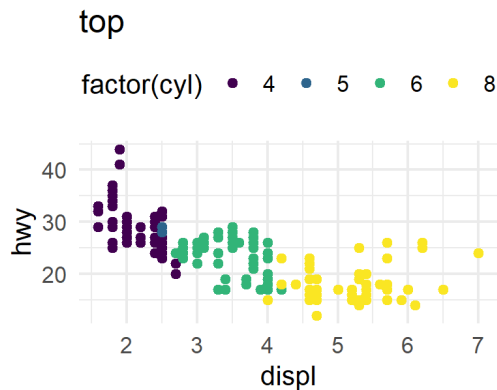
Legend Customization

```
ggplot(mpg, aes(x = displ, y = hwy, color = factor(cyl))) +  
  geom_point(size = 3) +  
  labs(title = "Customized Legend",  
        color = "Cylinders") +  
  theme_minimal() +  
  theme(  
    legend.position = "bottom",  
    legend.direction = "horizontal",  
    legend.background = element_rect(fill = "gray95", color = "black"),  
    legend.key = element_rect(fill = "white"),  
    legend.title = element_text(face = "bold", size = 12),  
    legend.text = element_text(size = 10)  
  )
```



Legend Position Options

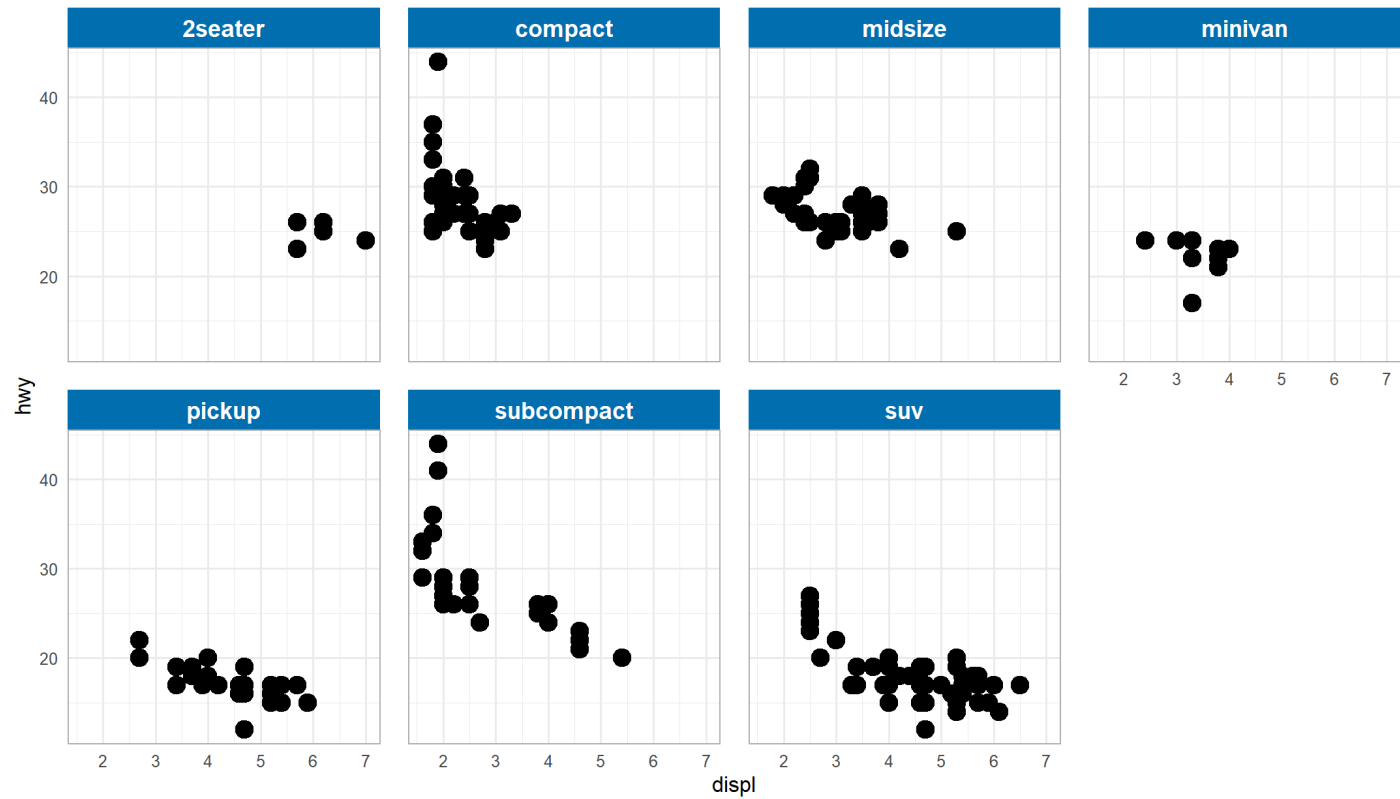
```
p_base <- ggplot(mpg, aes(x = displ, y = hwy, color = factor(cyl))) +  
  geom_point(size = 2) +  
  scale_color_viridis_d()  
  
p1 <- p_base + theme(legend.position = "top") + labs(title = "top")  
p2 <- p_base + theme(legend.position = "bottom") + labs(title = "bottom")  
p3 <- p_base + theme(legend.position = "left") + labs(title = "left")  
p4 <- p_base + theme(legend.position = "right") + labs(title = "right")  
  
(p1 + p2) / (p3 + p4)
```



Facet Customization

```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point() +  
  facet_wrap(~ class, nrow = 2) +  
  theme_minimal() +  
  theme(  
    strip.background = element_rect(fill = "#0072B2", color = NA),  
    strip.text = element_text(color = "white", size = 12, face =  
    panel.spacing = unit(1, "lines"),  
    panel.border = element_rect(color = "gray70", fill = NA)  
  ) +  
  labs(title = "Customized Facet Strips")
```


Customized Facet Strips

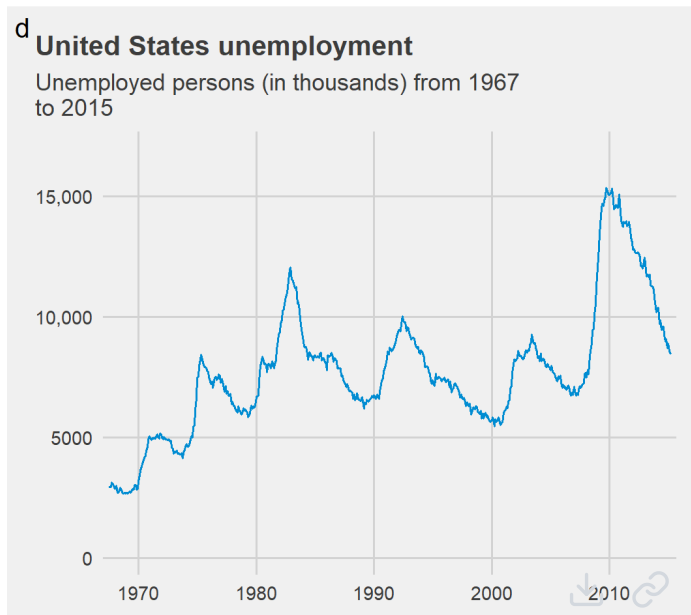
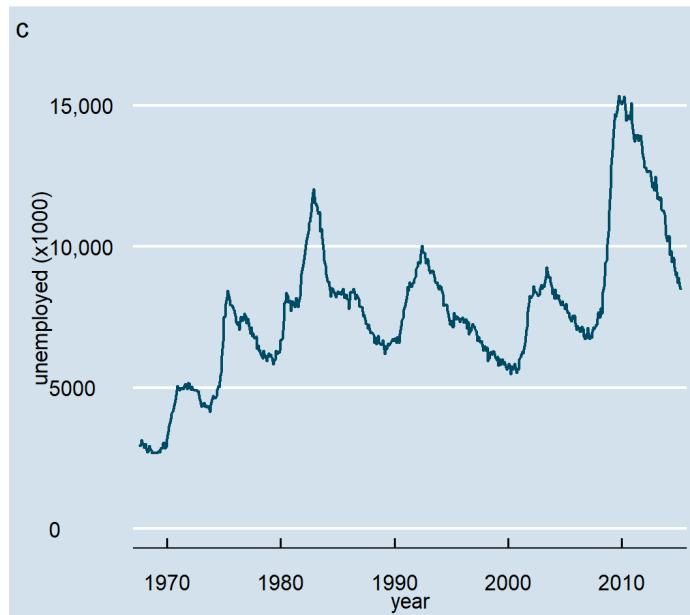
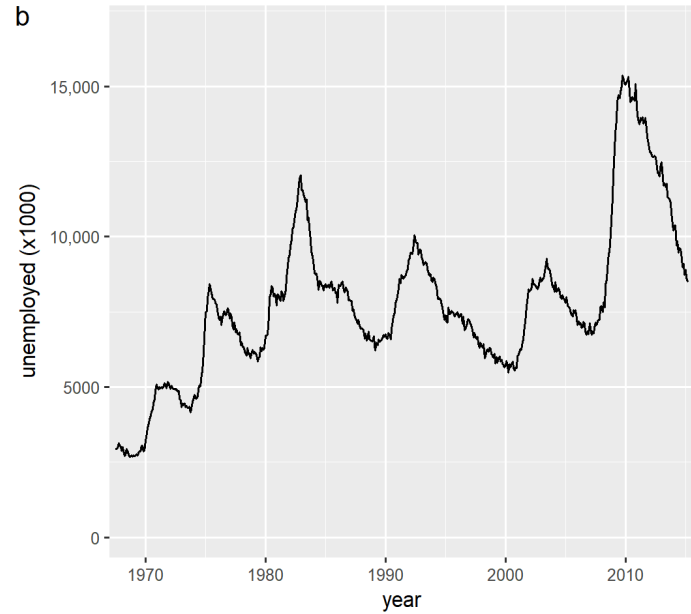
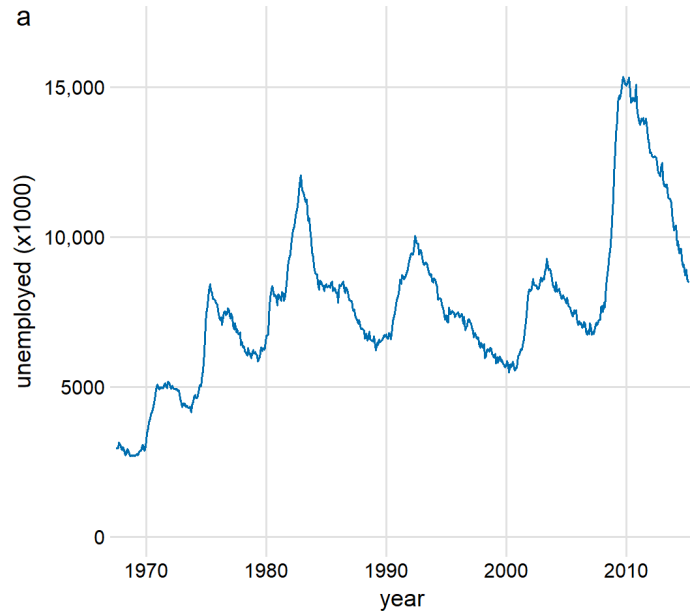


Data viz in Wild

Chen

Tongle

Everett and Dobjivi on deck for next
week

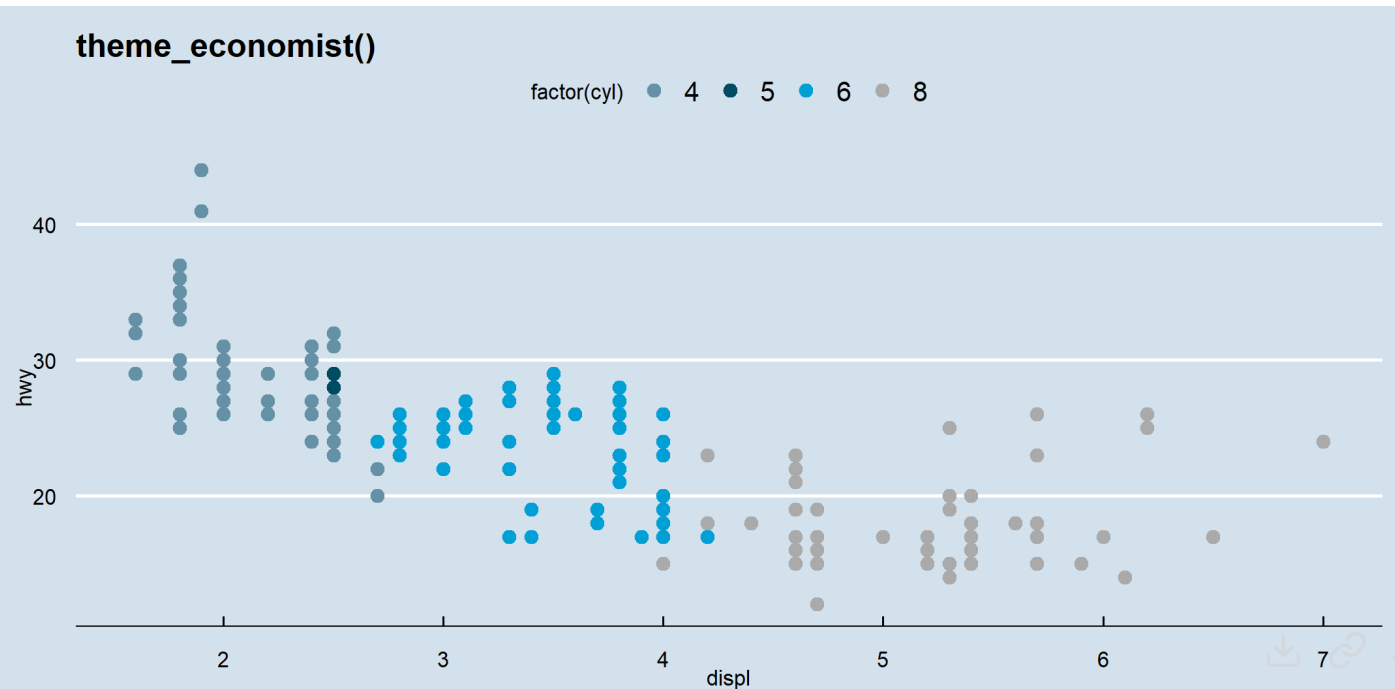


ggthemes Package

```
library(ggthemes)
```

```
p <- ggplot(mpg, aes(x = displ, y = hwy, color = factor(cyl))) +  
  geom_point(size = 3)
```

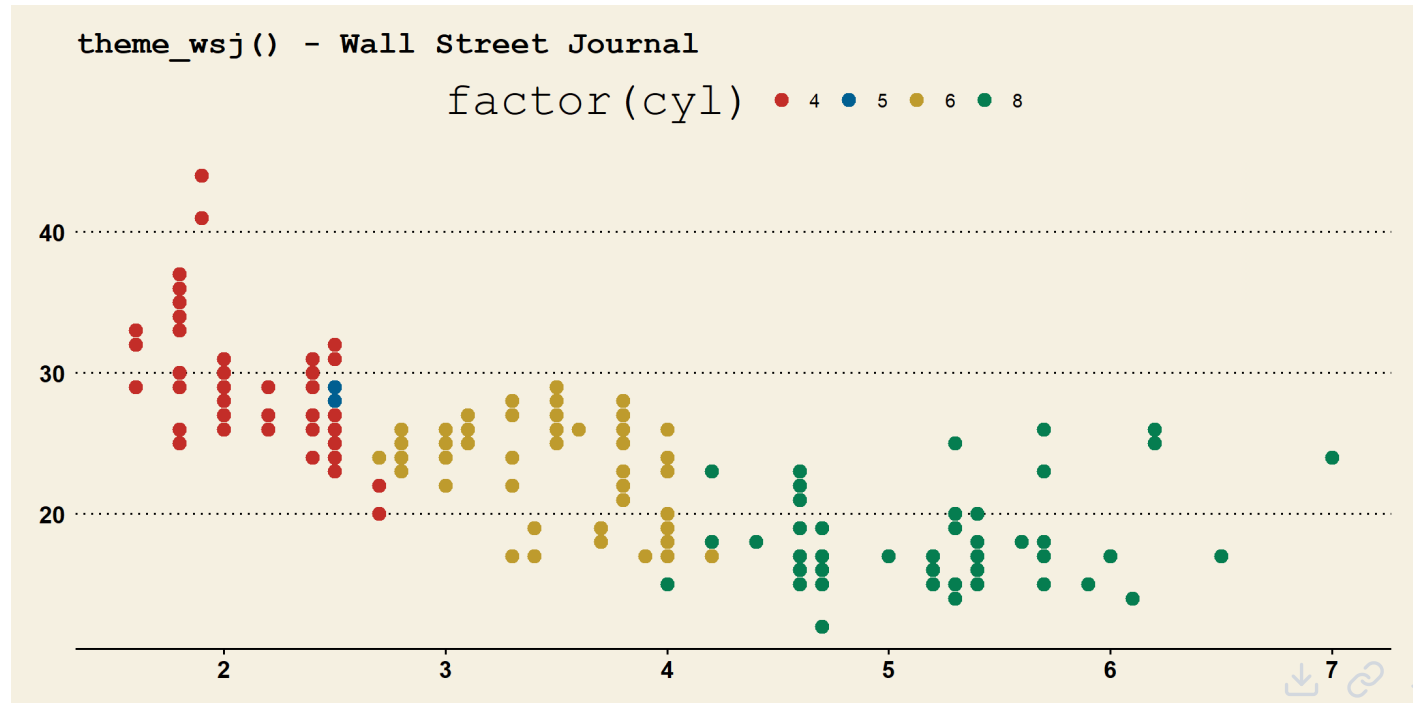
```
p + theme_economist() +  
  scale_color_economist() +  
  labs(title = "theme_economist()")
```



More ggthemes

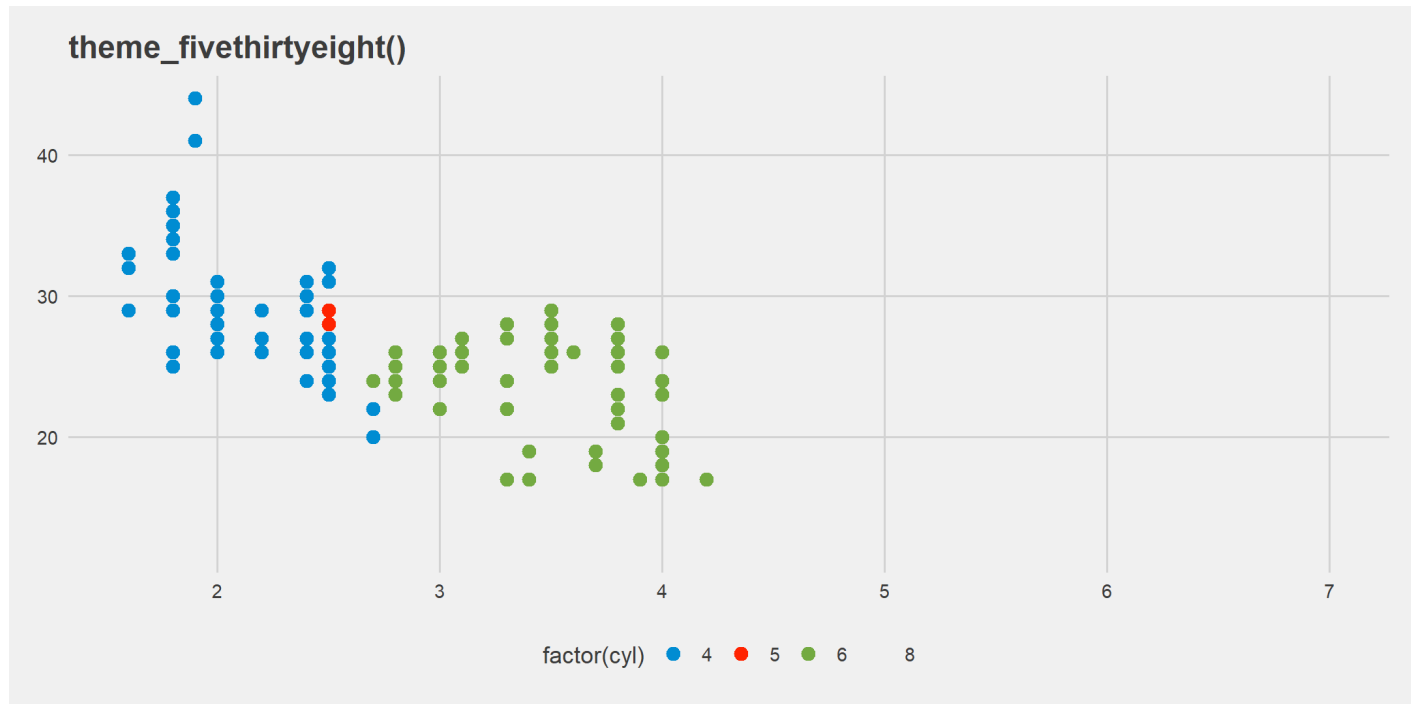
Examples

```
p + theme_wsj() +  
  scale_color_wsj() +  
  labs(title = "theme_wsj() - Wall Street Journal") +  
  theme(plot.title = element_text(size = 16))
```



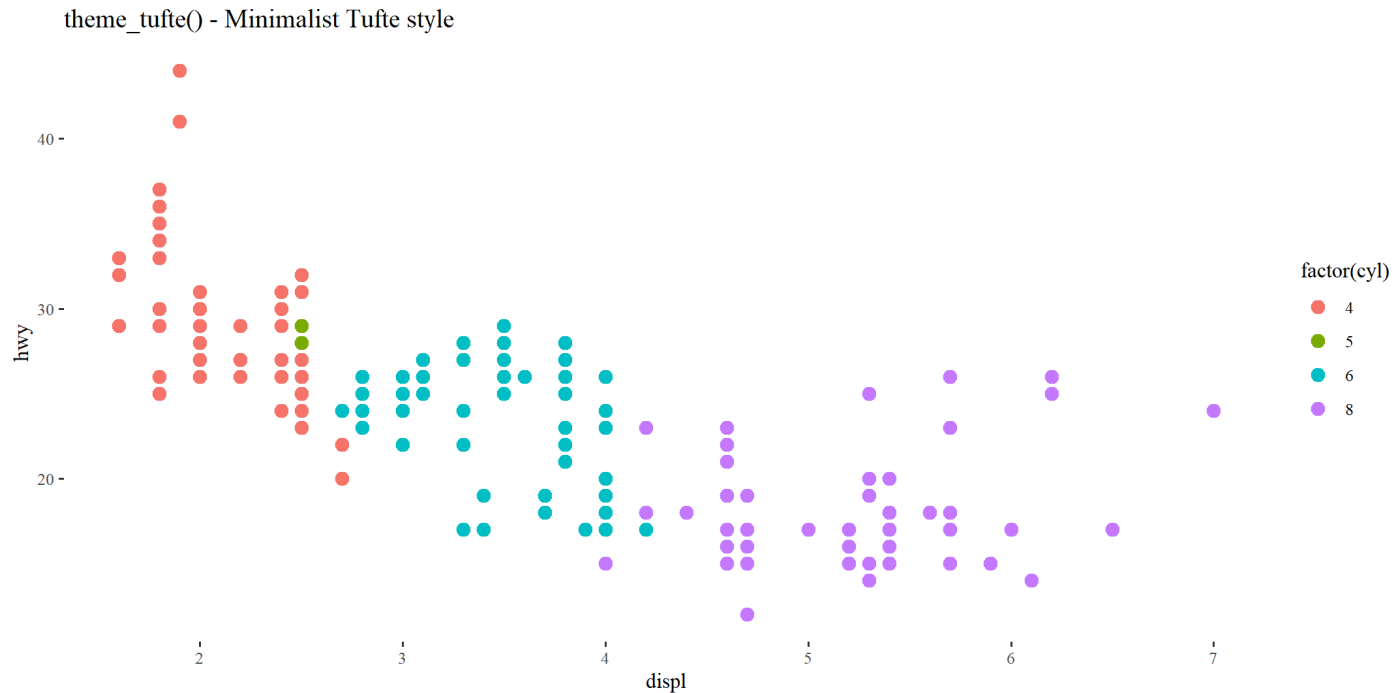
FiveThirtyEight Theme

```
p + theme_fivethirtyeight() +  
  scale_color_fivethirtyeight() +  
  labs(title = "theme_fivethirtyeight()") +  
  theme(plot.title = element_text(size = 16))
```



Tufte Theme (Minimal)

```
p + theme_tufte() +  
  labs(title = "theme_tufte() - Minimalist Tufte style")
```



BBC

The BBC uses ggplot for most of its graphics. They've developed a package with a theme and some functions to help make it match their style more.

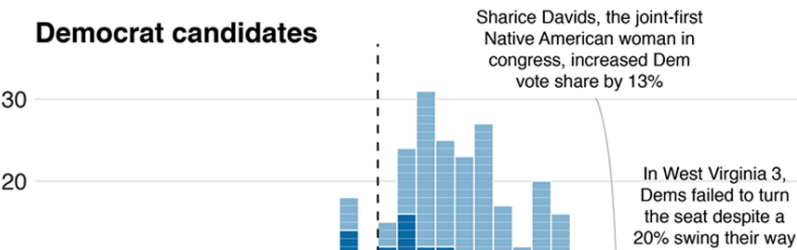
See the repo [here](#)

Their [Journalism Cookbook](#) is really nice too

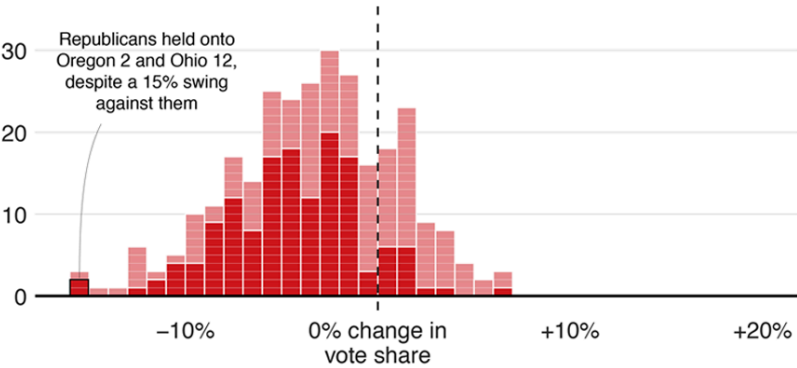
Blue wave

Won seat Didn't win

Democrat candidates



Republican candidates

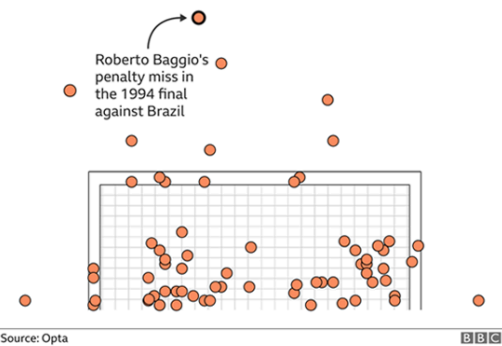


Source: AP, 19:01 ET

BBC

Where penalties are saved

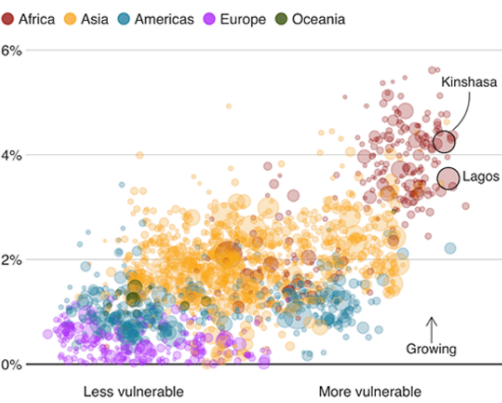
World Cup shootout misses and saves, 1982-2014



BBC

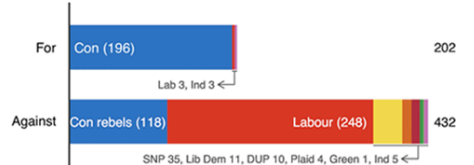
Fast-growing cities face worse climate risks

Population growth 2018-2035 over climate change vulnerability



Source: Verisk Maplecroft. Circle size represents current population.

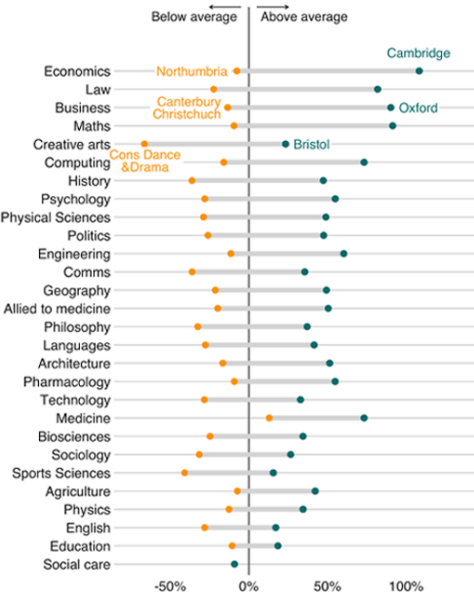
MPs rejected Theresa May's deal by 230 votes



Source: Commons Votes Services. Excludes 'tellers', the Speaker and deputies

Earnings vary across units even within subjects

Impact on men's earnings relative to the average degree



Source: Institute for Fiscal Studies

BBC

Similarly, the Urban Institute Visual Guide

See the repo [here](#)

So, I created one!

- Based on UO's visual guide [here](#)

[demo]

Creating Your Own Theme

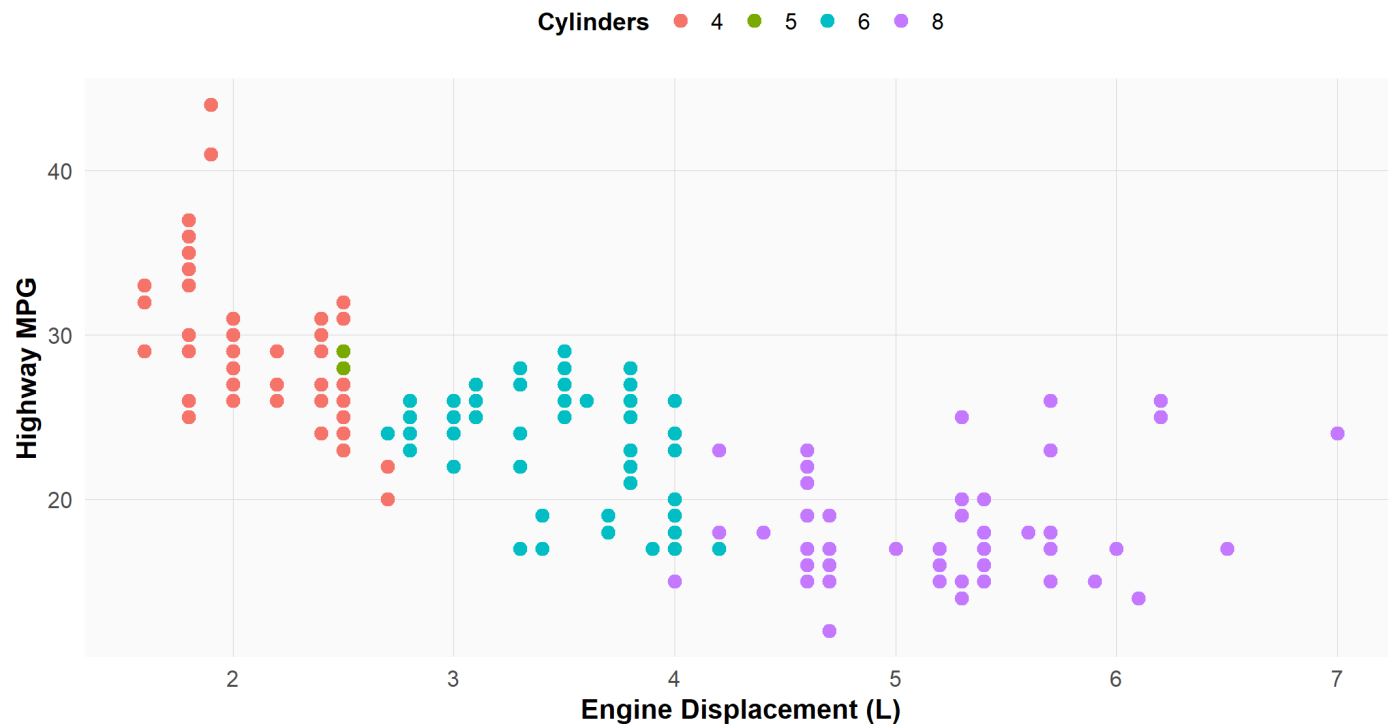
```
theme_maithreyi <- function(base_size = 14) {  
  theme_minimal(base_size = base_size) +  
  theme(  
    # Text elements scaled relative to base_size  
    plot.title = element_text(  
      size = base_size * 1.3,  
      face = "bold",  
      color = "#2c3e50",  
      hjust = 0  
    ),  
  
    plot.subtitle = element_text(  
      size = base_size * 0.9,  
      face = "italic",  
      color = "gray40",  
      hjust = 0  
    ),  
  
    axis.title = element_text(  
      size = base_size * 1.0,  
      face = "bold"  
    ),  
  )  
}
```

Using Your Custom Theme

```
ggplot(mpg, aes(x = displ, y = hwy, color = factor(cyl))) +  
  geom_point(size = 3) +  
  labs(title = "My Custom Theme",  
        subtitle = "Engine displacement vs. Highway MPG",  
        x = "Engine Displacement (L)",  
        y = "Highway MPG",  
        color = "Cylinders") +  
  theme_maithreyi()
```

My Custom Theme

Engine displacement vs. Highway MPG



ggthemeassist

- Another great place to start with making major modifications/creating your own custom theme
- Can't do everything, but can do a lot
- See [here](#)

[demo]

theme() for everything else

- You can basically change your plot to look however you want through `theme`
- Generally a bit more complicated
- I've used ggplot for *years* and only really now gaining fluency with it

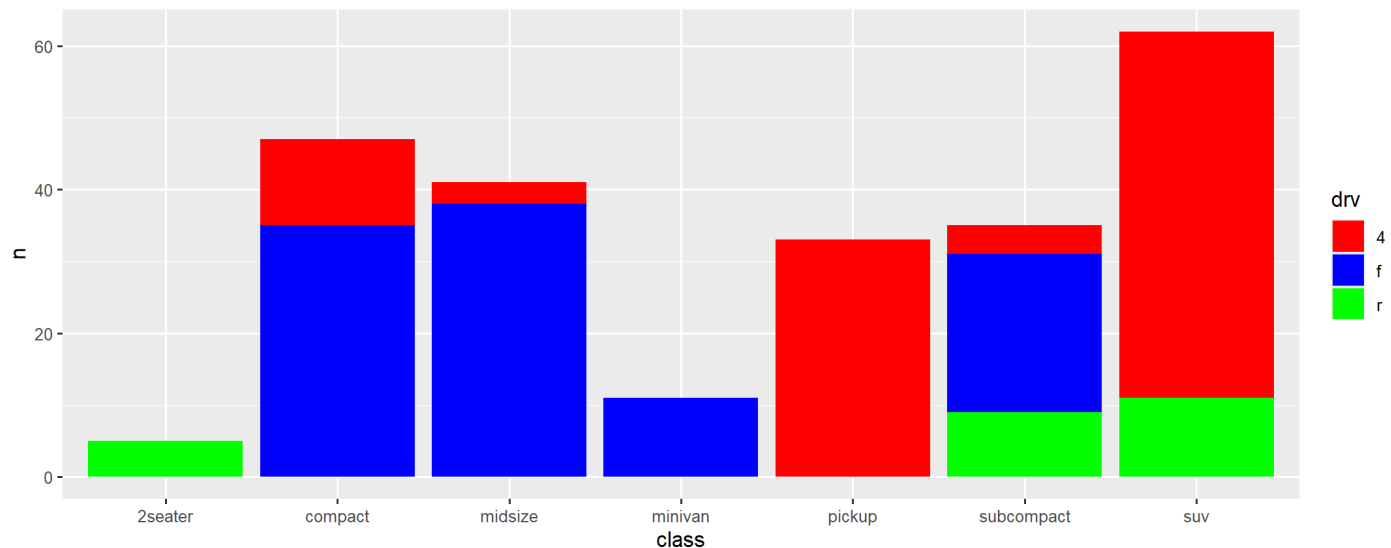
Putting It
All Together

Best Practices: Comparing Categories

1. **Order thoughtfully:** By value, time, or logic (not alphabetically)
2. **Use appropriate chart type:** Bars for counts, dots for many categories
3. **Consider faceting:** For multiple grouping variables
4. **Highlight what matters:** Use color strategically
5. **Annotate directly:** Labels often better than legends
6. **Maintain consistency:** Same colors across related charts

Example: Poor Practice

```
mpg %>%  
  count(class, drv) %>%  
  ggplot(aes(x = class, y = n, fill = drv)) +  
  geom_col() +  
  scale_fill_manual(values = c("red", "blue", "green")) +  
  theme_gray()
```



Problems: Stacked bars hard to compare, random colors, alphabetical ordering

Last Quick example

The *google_trends* dataset comes from a [fivethirtyeight story](#) about how the media covered hurricanes and Trump.

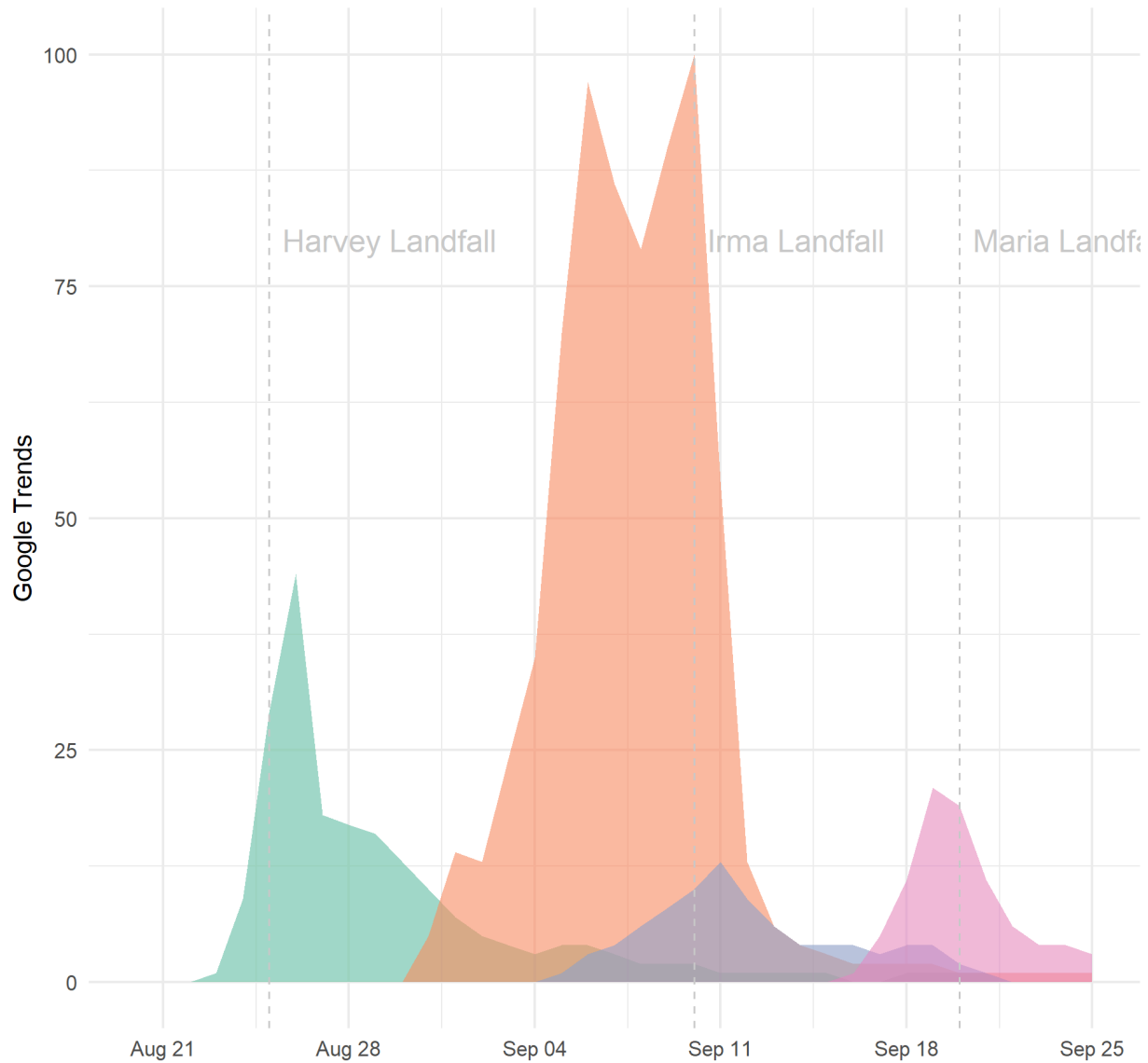
```
library(fivethirtyeight)
g <- google_trends %>%
  pivot_longer(starts_with("hurricane"),
               names_to = "hurricane",
               values_to = "interest",
               names_pattern = "_(.+)_")

landfall <- tibble(
  date = lubridate::mdy(
    c("August 25, 2017", "September 10, 2017", "September 20, 2017")
  ),
  hurricane = c("Harvey Landfall", "Irma Landfall", "Maria Landfall")
)
```

Let's start by visualizing the change in trends for each hurricane over time in one plot with three scales. We can map color to a discrete scale here. We can add vertical lines to show when each of them made landfall!

```
p <- ggplot(g, aes(date, interest)) +  
  geom_ribbon(aes(fill = hurricane, ymin = 0, ymax = interest),  
            alpha = 0.6) +  
  geom_vline(aes(xintercept = date), landfall,  
            color = "gray80",  
            lty = "dashed") +  
  geom_text(aes(x = date, y = 80, label = hurricane), landfall,  
            color = "gray80",  
            nudge_x = 0.5,  
            hjust = 0) +  
  labs(x = "",  
       y = "Google Trends",  
       title = "Hurricane Google trends over time",  
       caption = "Source: https://github.com/fivethirtyeight/data",  
       scale_fill_brewer("Hurricane", palette = "Set2")
```

Hurricane Google trends over time



We can use `ggthemeassist` to make a whole bunch of changes!

```
p + theme(  
  panel.grid.major = element_line(colour = "gray30"),  
  panel.grid.minor = element_line(colour = "gray30"),  
  axis.text = element_text(colour = "gray80"),  
  axis.text.x = element_text(colour = "gray80"),  
  axis.text.y = element_text(colour = "gray80"),  
  axis.title = element_text(colour = "gray80"),  
  legend.text = element_text(colour = "gray80"),  
  legend.title = element_text(colour = "gray80"),  
  panel.background = element_rect(fill = "gray10"),  
  plot.background = element_rect(fill = "gray10"),  
  legend.background = element_rect(fill = NA, color = NA),  
  legend.position = c(0.2, -0.1),  
  legend.direction = "horizontal",  
  plot.margin = margin(10, 10, b = 20, 10),  
  plot.caption = element_text(colour = "gray80", vjust = 1),  
  plot.title = element_text(colour = "gray80")  
)
```

Saving Plots

The ggsave() function

Basic usage:

```
p <- ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point()  
  
ggsave("my_plot.png", p, width = 8, height = 6, dpi = 300)
```

Common ggsave() arguments

- **filename**: Path and filename (extension determines format)
- **plot**: Plot object (defaults to last plot)
- **width**, **height**: Dimensions in **units**
- **units**: "in" (default), "cm", or "mm"
- **dpi**: Resolution (300 for print, 96 for screen)
- **device**: File type ("png", "pdf", "jpeg", "tiff", etc.)

Different file formats

```
# PNG for web/presentations
ggsave("figure1.png", width = 8, height = 6, dpi = 300)

# PDF for publications (vector, scalable)
ggsave("figure1.pdf", width = 8, height = 6)

# JPEG (smaller file size, lossy compression)
ggsave("figure1.jpg", width = 8, height = 6, dpi = 300)

# TIFF for publications (lossless)
ggsave("figure1.tiff", width = 8, height = 6, dpi = 300)

# SVG (vector, for web)
ggsave("figure1.svg", width = 8, height = 6)
```

Aspect ratios for different uses

```
# Widescreen presentation (16:9)
ggsave("presentation.png", width = 10, height = 5.625, dpi = 150)

# Square for Instagram
ggsave("instagram.png", width = 6, height = 6, dpi = 300)

# Journal figure (often 3.5" or 7" wide)
ggsave("journal.pdf", width = 7, height = 5)

# Poster (large format)
ggsave("poster.pdf", width = 24, height = 36, dpi = 300)
```

Setting defaults

```
# Create custom save function
save_plot <- function(filename, plot = last_plot()) {
  ggsave(
    filename = filename,
    plot = plot,
    width = 8,
    height = 6,
    dpi = 300,
    bg = "white" # Ensures white background
  )
}

# Use it
p <- ggplot(mpg, aes(x = displ, y = hwy)) + geom_point()
save_plot("my_figure.png", p)
```

Saving plots for publication

```
# High resolution, multiple formats
p <- ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  theme_minimal(base_size = 12)

# For manuscript
ggsave("figure1_main.pdf", p, width = 7, height = 5)
ggsave("figure1_main.png", p, width = 7, height = 5, dpi = 600)

# For supplement (might be different size)
ggsave("figureS1.pdf", p, width = 8.5, height = 11)
```

The preview trick

```
# From TJ Mahr - preview exact output size
ggpreview <- function(..., device = "png") {
  fname <- tempfile(fileext = paste0(".", device))
  ggsave(filename = fname, device = device, ...)
  system2("open", fname) # Mac
  # Use shell.exec(fname) on Windows
  invisible(NULL)
}

# Preview at exact journal dimensions
ggpreview(width = 3.5, height = 3.5, dpi = 300)
```

Editing in external programs

Sometimes you need to make final tweaks outside R:

- **Inkscape** (free): Vector editing of PDFs/SVGs
- **Adobe Illustrator**: Professional vector editing
- **GIMP** (free): Raster image editing
- **PowerPoint**: Quick annotations and combinations

I rarely do this, but if I do, I tend to use Inkscape, which is free.

```
ggplot(mpg, aes(displ, hwy)) +  
  geom_point(color = "gray80") +  
  geom_point(color = "#FD7A43", data = filter(mpg, cyl == 4))  
  
ggsave("~/Desktop/example-plot.pdf", width = 6.5, height = 6.5)
```


Compound figures

Please follow along

Options

My favorite: `{patchwork}`

- `{cowplot}`
- `{ggpubr}`
- `{gridExtra}`

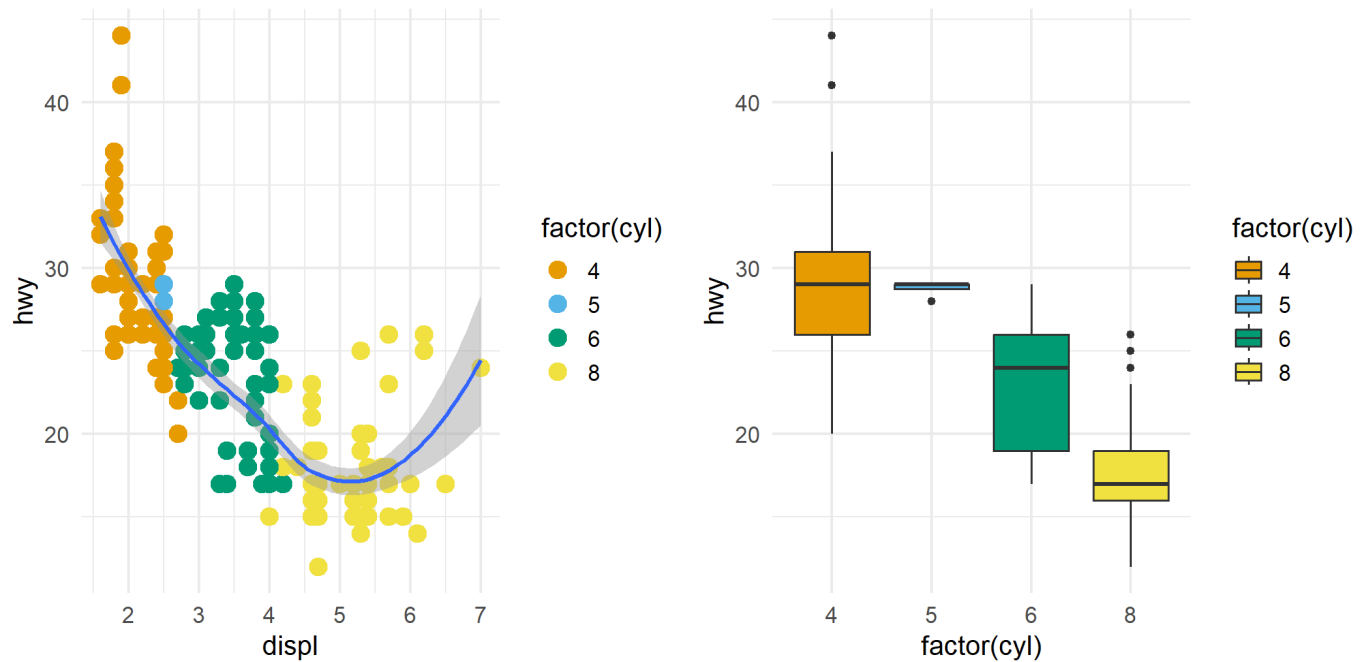
Example

- First, create two plots

```
p1 <- ggplot(mpg, aes(displ, hwy)) +  
  geom_point(aes(color = factor(cyl))) +  
  geom_smooth() +  
  scale_color_okabeIto()  
  
p2 <- ggplot(mpg, aes(factor(cyl), hwy)) +  
  geom_boxplot(aes(fill = factor(cyl))) +  
  scale_fill_okabeIto()
```

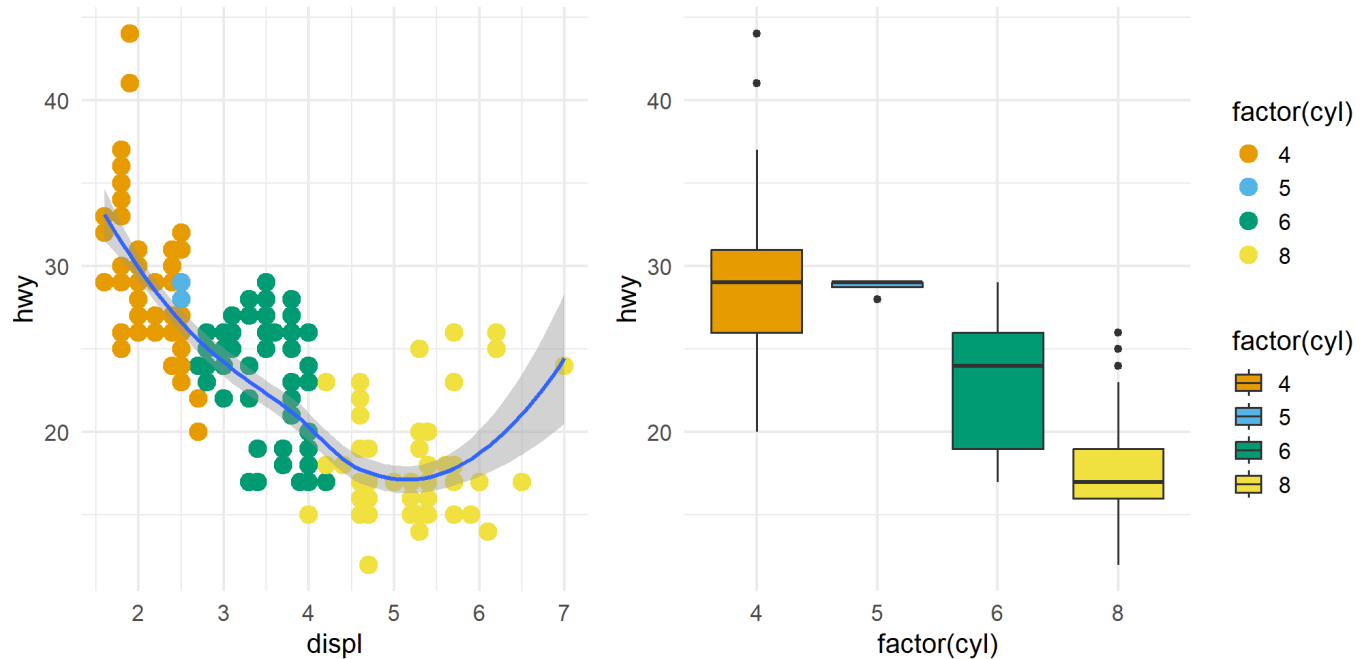
Side by side

```
library(patchwork)
p1 + p2
```



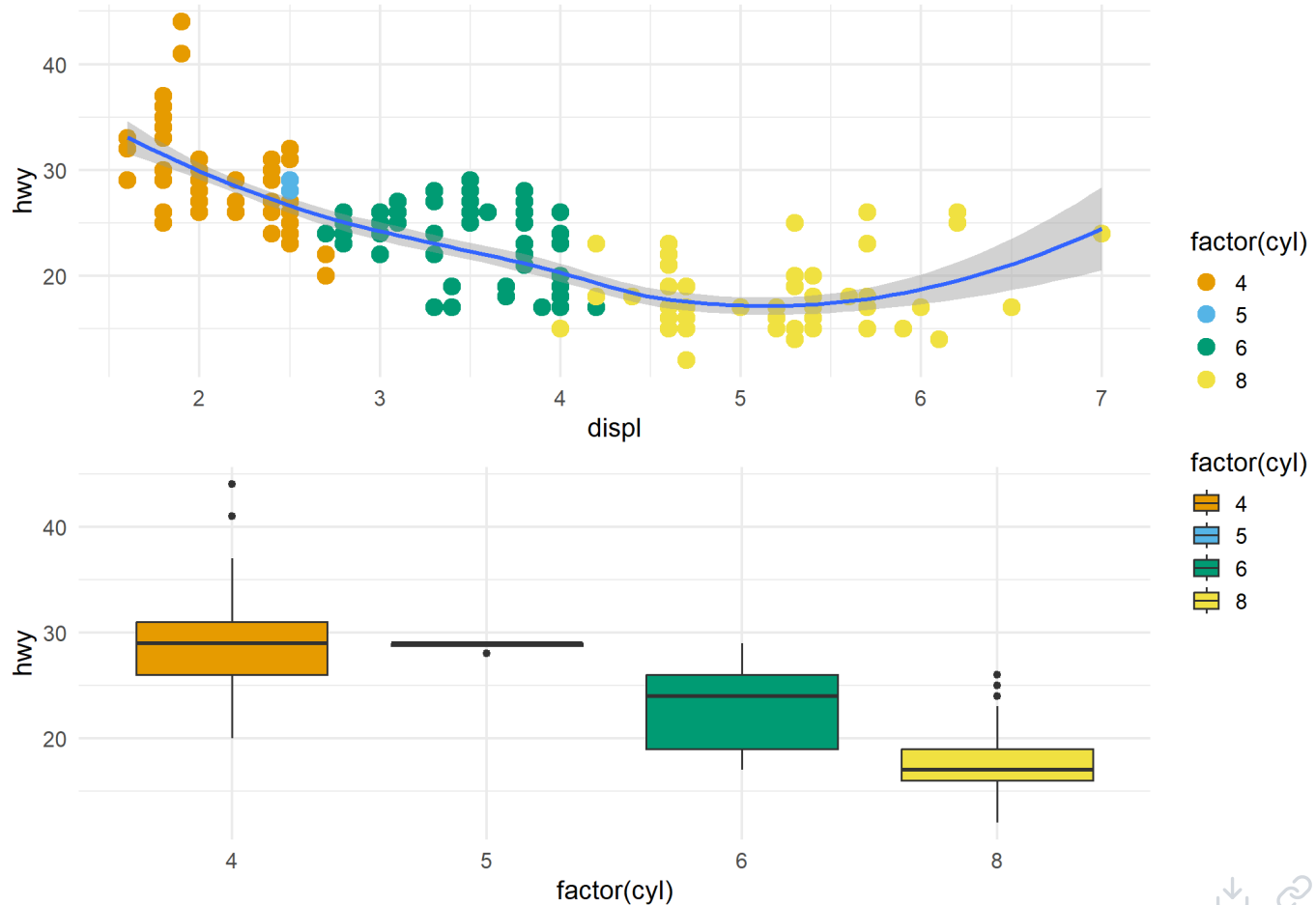
Collect legends

```
p1 + p2 + plot_layout(guides = "collect")
```



Stack vertically

```
p1 / p2 + plot_layout(guides = "collect")
```

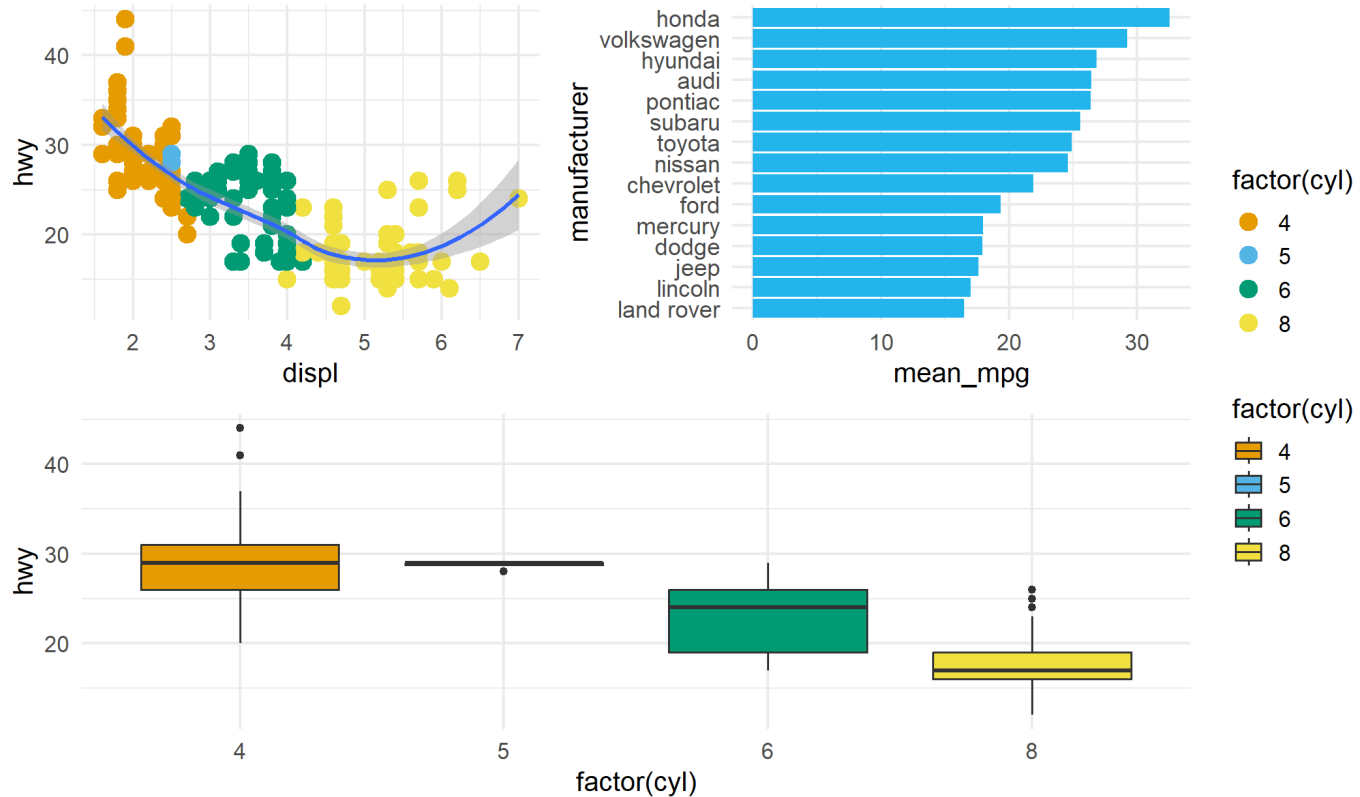


Add a third plot

```
p3 <- mpg %>%  
  group_by(manufacturer) %>%  
  summarize(mean_mpg = mean(hwy, na.rm = TRUE)) %>%  
  mutate(manufacturer = fct_reorder(manufacturer, mean_mpg)) %>%  
  ggplot(aes(mean_mpg, manufacturer)) +  
    geom_col(fill = "#25B6EE") +  
    theme(axis.text.y = element_text(size = 12))
```

Put box plot on bottom

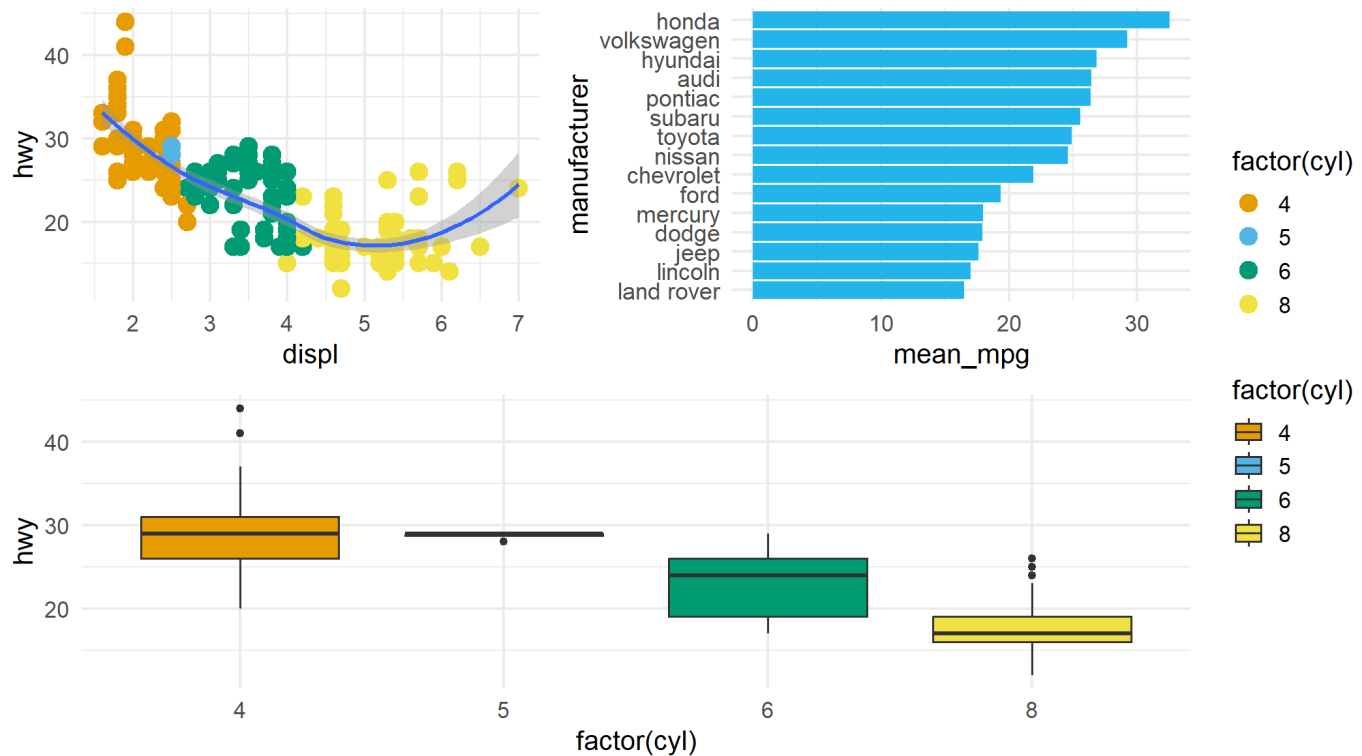
```
(p1 + p3) / p2 + plot_layout(guides = "collect")
```



Overall title

```
(p1 + p3) / p2 + plot_layout(guides = "collect") +  
  plot_annotation("Some cool plots")
```

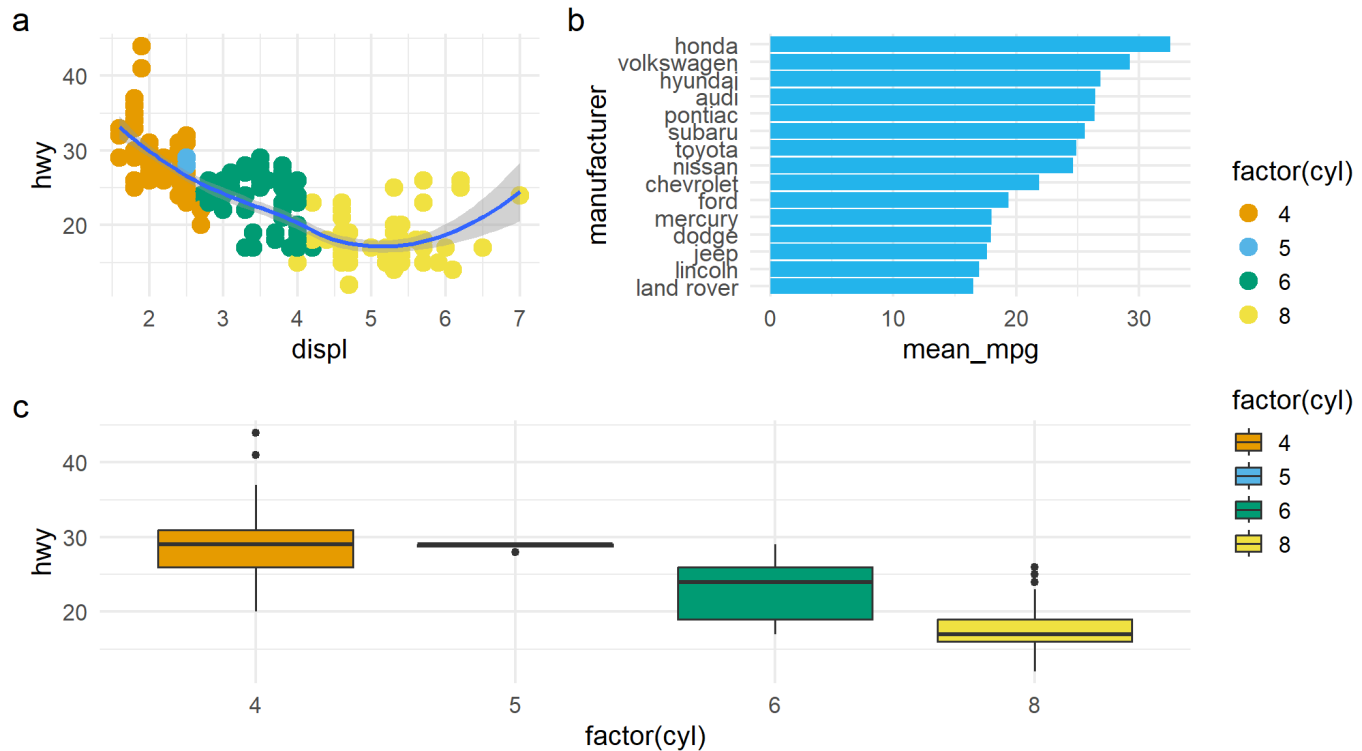
Some cool plots



Tags

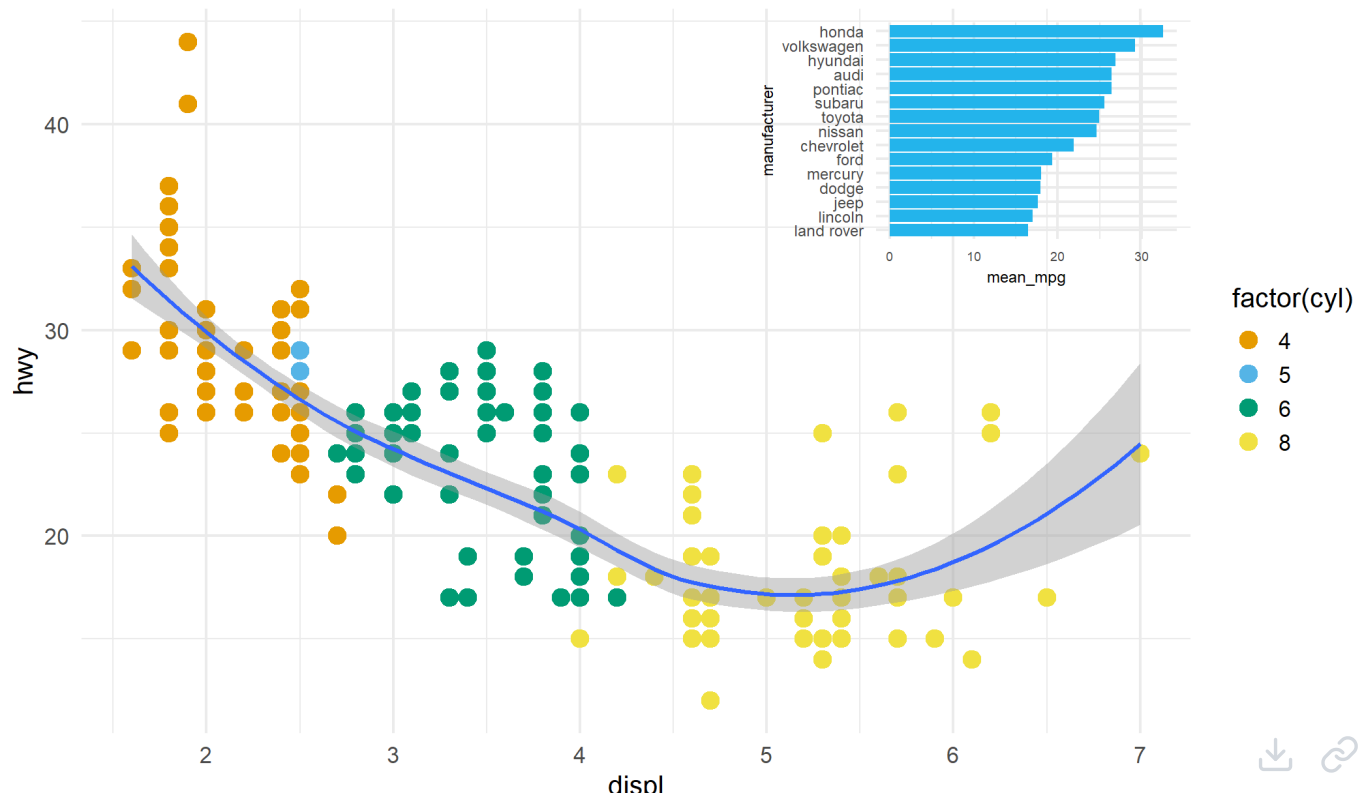
```
(p1 + p3) / p2 + plot_layout(guides = "collect") +  
  plot_annotation("Some cool plots", tag_levels = "a")
```

Some cool plots



Insets

```
p3_small_txt <- p3 +  
  theme(axis.text.y = element_text(size = 8),  
        text = element_text(size = 8))  
  
p1 + inset_element(p3_small_txt, 0.6, 0.6, 1, 1)
```

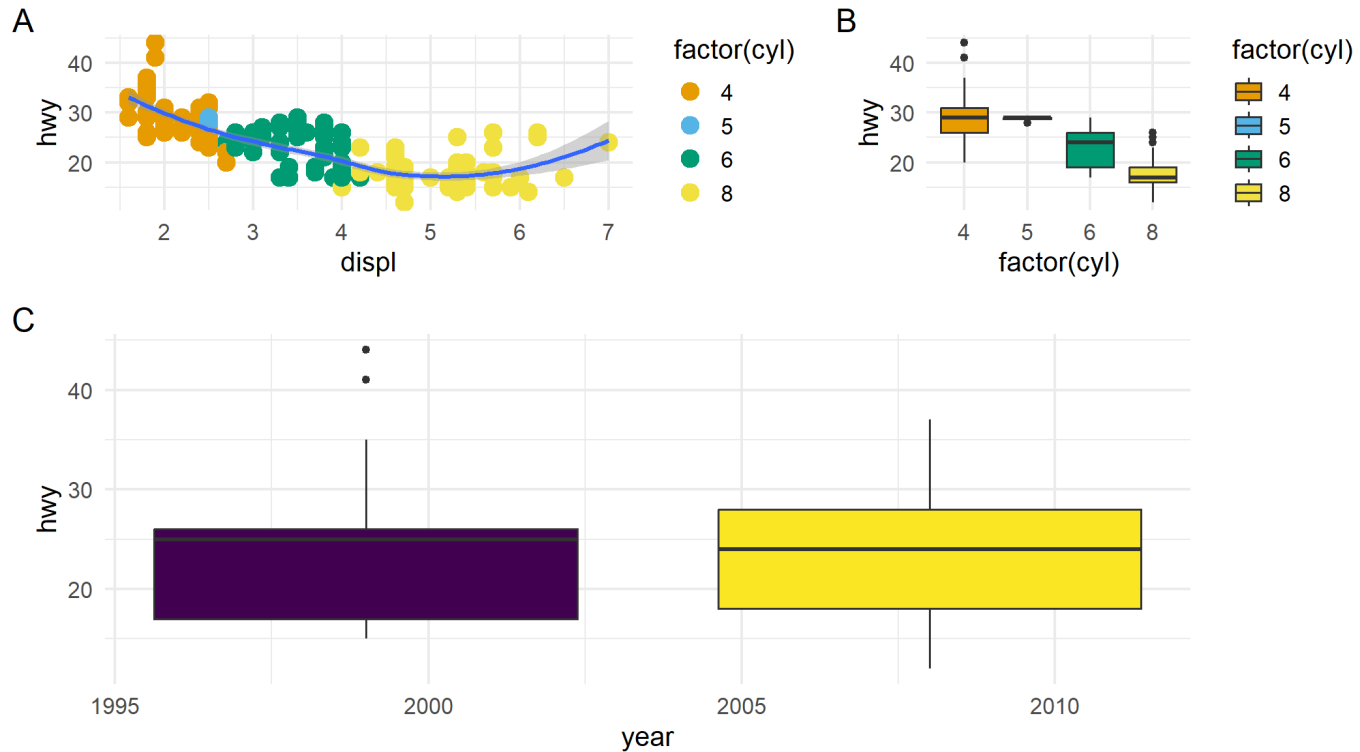


Complex Layouts with patchwork

```
p4 <- ggplot(mpg, aes(x = year, y = hwy)) +  
  geom_boxplot(aes(group = year, fill = factor(year))) +  
  scale_fill_viridis_d() +  
  theme(legend.position = "none")  
  
layout <- "  
AAB  
CCC  
CCC  
"  
  
p1 + p2 + p4 +  
  plot_layout(design = layout) +  
  plot_annotation(title = "Complex Custom Layout",  
                  tag_levels = "A")
```

Complex Layouts with patchwork

Complex Custom Layout



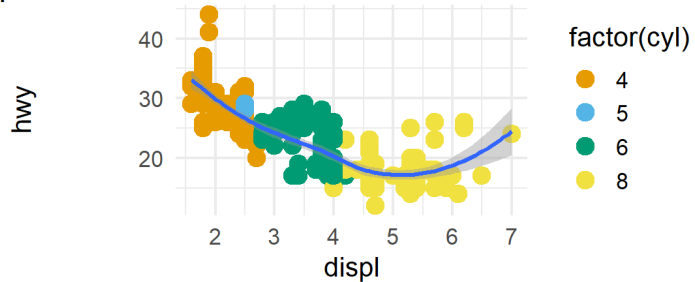
Nested Layouts

```
top_row <- p1 + p2
bottom_plot <- p3

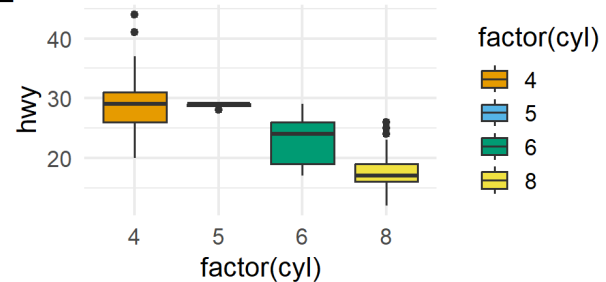
top_row / bottom_plot +
  plot_layout(heights = c(1, 1.5)) +
  plot_annotation(title = "Nested Layout Example",
                  tag_levels = "1")
```

Nested Layout Example

1



2



3



Lab 3 and Lab 4 Review

+ Discussion (if time permits)

Lab PS-3

Will be posted tomorrow

Lab 5

Next time

Wrap up visualizing changes over time/distributions and Intro to Websites, Flex dashboards, CSS customizations

Note: Lab-PS3- Last Lab problem set in this class will be posted tomorrow